第四章：系統塑模與統一架構 (System Modeling & Unified Architecture)

課程大綱與核心觀念 (Lecture Outline & Key Concepts)

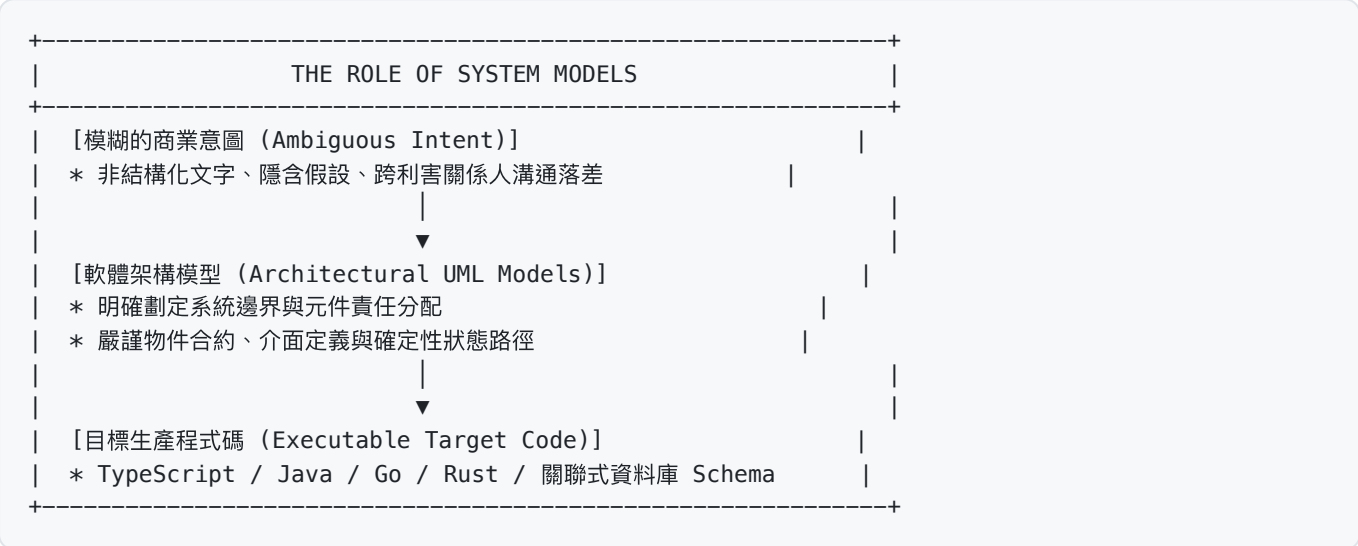
- **4.1 系統塑模基礎 (Foundations of System Modeling)**：為何需要軟體塑模？自然語言的模糊性局限、盲人摸象大師寓言，以及軟體架構四大正交塑模視角（外部視角、互動視角、結構視角、行為視角）。
- **4.2 統一塑模語言 (UML) 與統一流程**：1990 年代物件導向「方法大戰 (Method Wars)」、UML 三巨頭（Grady Booch, Jim Rumbaugh, Ivar Jacobson）歷史整合、OMG 國際標準規範、UML 2.5 十四種圖表分類樹，以及以架構為中心、使用案例驅動之漸進迭代思維。
- **4.3 功能與使用案例模型 (Use Case Modeling)**：系統邊界矩形、參與者 (Actors) 分類學、主動動詞命名語義、關聯矩陣（`<<include>>` 強制包含子程序 vs. `<<extend>>` 條件擴充點 vs. 參與者一般化繼承）、嵌套深度安全上限、大學選課系統案例、十欄位工業級使用案例規格書 (Use Case Specification)、七大塑模反模式，以及 AI 提示詞工程。
- **4.4 結構與領域類別模型 (Class Modeling)**：類別三層結構（名稱、屬性、方法）、可見度修飾詞（`+`, `-`, `#`, `~`）、參數方向性、物件關聯分類學（一般關聯、聚合、組合、一般化繼承、依賴）、導覽性與重數多重性 (Multiplicity)、關聯類別、外送平台領域模型實戰、專家檢核清單與 AI 提示詞工程。
- **4.5 互動與循序圖模型 (Sequence Diagrams)**：物件隨時間協同運作、二維畫布（水平參與者 vs. 垂直向下時間軸）、生命線 (Lifeline) 與啟動條 (Activation)、五大通訊訊息語法（同步、非同步、回傳、建立、銷毀）、複合片段 (Combined Fragments: `alt`, `opt`, `loop`, `par`)、飯店訂房案例、BCE (Boundary-Control-Entity) 架構模式，以及 AI 提示詞工程。
- **4.6 流程與活動圖模型 (Activity Diagrams)**：程序邏輯與 Token 流程控制、活動圖相較傳統流程圖之四大優勢、分岔與結合 (Fork/Join 並行同步) vs. 決策與合併 (Decision/Merge 互斥條件) 深度辨析、泳道 (Swimlanes/Partitions) 職責分工、物件流 (Object Flow) 與資料狀態、中斷區域與例外處理、外送平台履行流程實戰，以及 AI 提示詞工程。
- **4.7 行為與狀態機圖模型 (State Machine Modeling)**：狀態相依行為（同一事件，不同反應）、四大狀態變遷觸發因子 (Signal, Call, Time, Change)、五元素變遷語法 (`Trigger [Guard] / Effect`)、動作 (Action 原子不可中斷) vs. 活動 (Activity 耗時可中斷) 核心語義辨析、`entry/` 與 `exit/` 機制、複合狀態 (Composite State) 與淺/深歷史假狀態 (`H`, `H*`)、正交區域並行性、溫控與外送訂單生命週期案例，以及 AI 提示詞工程。
- **4.8 PlantUML 文字宣告式塑模 (Architecture-as-Code)**：架構即代碼哲學、徹底克服傳統 GUI 拖拉軟體無版本控制與 PR 無法 diff 之工程瓶頸、五大核心模型語法矩陣、Skinparam 樣式定制、跨模型語法速查表，以及現代 Generative AI 提示詞架構合成工作流。
- **4.9 模型選擇決策框架與課程總結 (Decision Framework & Recap)**：六大模型架構決策矩陣、十大核心概念填空自我檢測題，以及國內外權威經典文獻指引。
- **附錄：課堂互動測驗 (CCQ 1–8) 詳細解答與架構復盤解析**。

4.1 系統塑模基礎 (Foundations of System Modeling)

在現代軟體工程中，如果未經架構塑模就直接開始撰寫程式碼，就如同建造一座四十層的商業大樓卻沒有結構藍圖。雖然自然語言（如使用者故事、需求訪談記錄）能夠直觀表達商業願景，但文字天生具備歧義性、不完整性與語意落差。系統塑模 (System Modeling) 提供了形式化、圖形化的抽象機制，讓工程團隊在動手投入昂貴的編碼成本前，能夠精準設計、溝通並驗證軟體架構。

4.1.1 什麼是模型？

模型 (Model) 是對真實系統的一種「刻意抽象化表示」：它強調與當前設計目的高度相關的本質特徵，同時過濾掉無關緊要的實現細節。正如電子工程師依賴電路原理圖（而不是顯微鏡下的銅導線實體掃描）來分析邏輯閘，軟體工程師透過架構圖來推演模組邊界、資料流向與狀態變遷。



4.1.2 盲人摸象與軟體架構的四大塑模視角

沒有任何單一圖表能夠同時呈現整個軟體系統的全部面向。若試圖將資料庫表格、網路通訊協定、使用者介面點擊與演算法細節塞進同一張圖中，只會產出毫無可讀性的視覺混亂。因此，軟體工程師師法經典「盲人摸象」的哲理，從**四個彼此正交的專業視角**來剖析系統：

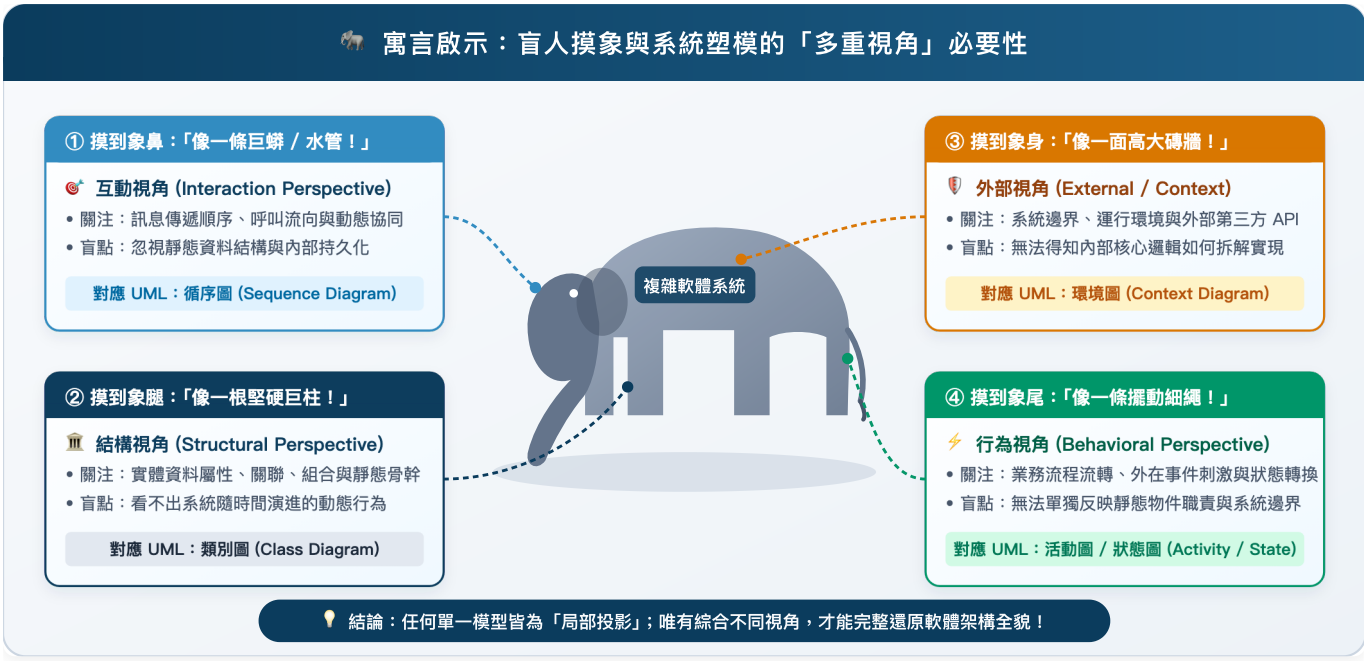


圖 4.1.1：盲人摸象寓言與軟體架構四大視角。每一種 UML 圖表專注於捕捉軟體架構的特定視角，組合起來方能呈現系統的全貌。

1. **外部視角 (External Perspective - 環境與邊界)**： 描繪系統與其所處運作環境的關聯，劃定清晰的系統邊界矩形，區分內部開發程式碼範疇與外部人類使用者、第三方 SaaS 雲端服務或硬體感測器。
2. **互動視角 (Interaction Perspective - 動態協同運作)**： 描繪外部參與者與系統之間，以及系統內部各個執行期物件之間，如何跨越記憶體與網路邊界傳遞訊息、調用函式。
3. **結構視角 (Structural Perspective - 靜態組織架構)**： 描繪領域實體、類別屬性、方法簽名、介面合約與實體間的靜態關係（關聯、聚合、組合、繼承），完全獨立於執行時間軸。
4. **行為視角 (Behavioral Perspective - 動態狀態與流程)**： 描繪系統如何對內部與外部事件做出反應，捕捉實體的生命週期狀態變遷、條件分支、例外中斷與並行工作流程。

觀念檢核測驗 (CCQ 1) — 四大塑模視角

問題

軟體架構師希望定義**領域實體資料如何組織、類別之間如何繼承與關聯**，且完全獨立於運行時期的執行順序。架構師應採取哪一種塑模視角？

- A. 外部視角 (External Perspective)
- B. 互動視角 (Interaction Perspective)
- C. 結構視角 (Structural Perspective)
- D. 行為視角 (Behavioral Perspective)

Interactive Activity (線上作答)



▶ 點擊展開參考答案與解析

4.2 統一塑模語言 (UML) 與統一流程

在 1990 年代中期之前，物件導向軟體工程界經歷了一場著名的「**方法大戰 (Method Wars)**」。市場上充斥著數十種互不相容的標記法，其中最具代表性的包括 Grady Booch 的 Booch Method、Jim Rumbaugh 的 OMT（物件塑模技術），以及 Ivar Jacobson 的 OOSE（物件導向軟體工程）。工程師每換一家公司就必須重新學習一套私有的圖表語法，造成嚴重的溝通障礙。

4.2.1 UML 三巨頭與統一歷程

1994 年，Rational Software 公司促成了這三位大師的劃時代聯手，工程界尊稱為 **UML 三巨頭 (The Three Amigos)**：

先驅大師	代表性塑模方法論	對 UML 的核心歷史貢獻
Grady Booch	Booch Method	物件導向設計、巨觀結構架構與工程符號化
James Rumbaugh	OMT (Object Modeling Technique)	豐富靜態物件關聯、狀態圖分析與資料建模
Ivar Jacobson	OOSE (Object-Oriented SE)	發明使用案例 (Use Cases)、參與者商業目標導向之需求工程技術

三位大師去蕪存菁，於 1997 年正式發表 UML 1.0，隨後被 **OMG (Object Management Group)** 採納為全球軟體產業的統一國際標準（ISO/IEC 19505）。

4.2.2 UML 2.5 十四種圖表分類樹

現代 UML 2.5 規格書共定義了 14 種官方圖表，對稱劃分為兩大分支：

分類樹分支	UML 2.5 官方圖表類型	軟體架構核心觀察視角
結構圖 (Structure Diagrams) *(靜態架構)*	• **類別圖 Class Diagram** (核心) • 物件圖 Object Diagram • 元件圖 Component Diagram • 部署圖 Deployment Diagram • 套件圖 Package Diagram • 複合結構圖 Composite Structure • 剖面輪廓圖 Profile Diagram	塑模軟體靜態結構、實體屬性、關聯繼承、微服務模組邊界與實體硬體伺服器部署拓撲。
行為圖 (Behavior Diagrams) *(動態行為)*	• **使用案例圖 Use Case Diagram** (範疇) • **活動圖 Activity Diagram** (工作流) • **狀態機圖 State Machine** (生命週期)	塑模系統執行時期的動態運作、利害關係人商業目標、演算法並行流程與物件生命週期變遷。
互動圖 (Interaction Diagrams) *(行為圖子分支)*	• **循序圖 Sequence Diagram** (時間時序) • 溝通圖 Communication Diagram • 計時圖 Timing Diagram • 互動概觀圖 Interaction Overview	聚焦物件與物件之間「隨時間推移之訊息調用鏈 (Message Passing)」、分散式 API 交互協定。

> 📌 **工程實踐的 80/20 法則**：在現代敏捷與企業級開發中，超過 80% 的架構溝通與設計工作，主要由五大核心模型完成：**使用案例圖 (Use Case)**、**類別圖 (Class)**、**循序圖 (Sequence)**、**活動圖 (Activity)** 與 **狀態機圖 (State Machine)**。

🙋‍♂️ 觀念檢核測驗 (CCQ 2) — UML 奠基先驅與三巨頭

問題

在 UML 「三巨頭 (Three Amigos)」中，哪一位大師因於 1986 年發明**使用案例 (Use Cases)**、將軟體架構錨定於使用者目標而享譽工程界？

- A. Grady Booch
- B. James Rumbaugh
- C. Ivar Jacobson

- D. Martin Fowler

Interactive Activity (線上作答)

▶ 點擊展開參考答案與解析

4.3 功能與使用案例模型 (Use Case Modeling)

使用案例模型是界定軟體系統責任範圍的最主要工具。它迫使工程團隊走出內部實作細節，站在外部參與者的立場思考：「系統究竟提供什麼具備商業價值的服務？」

4.3.1 使用案例圖視覺語法結構

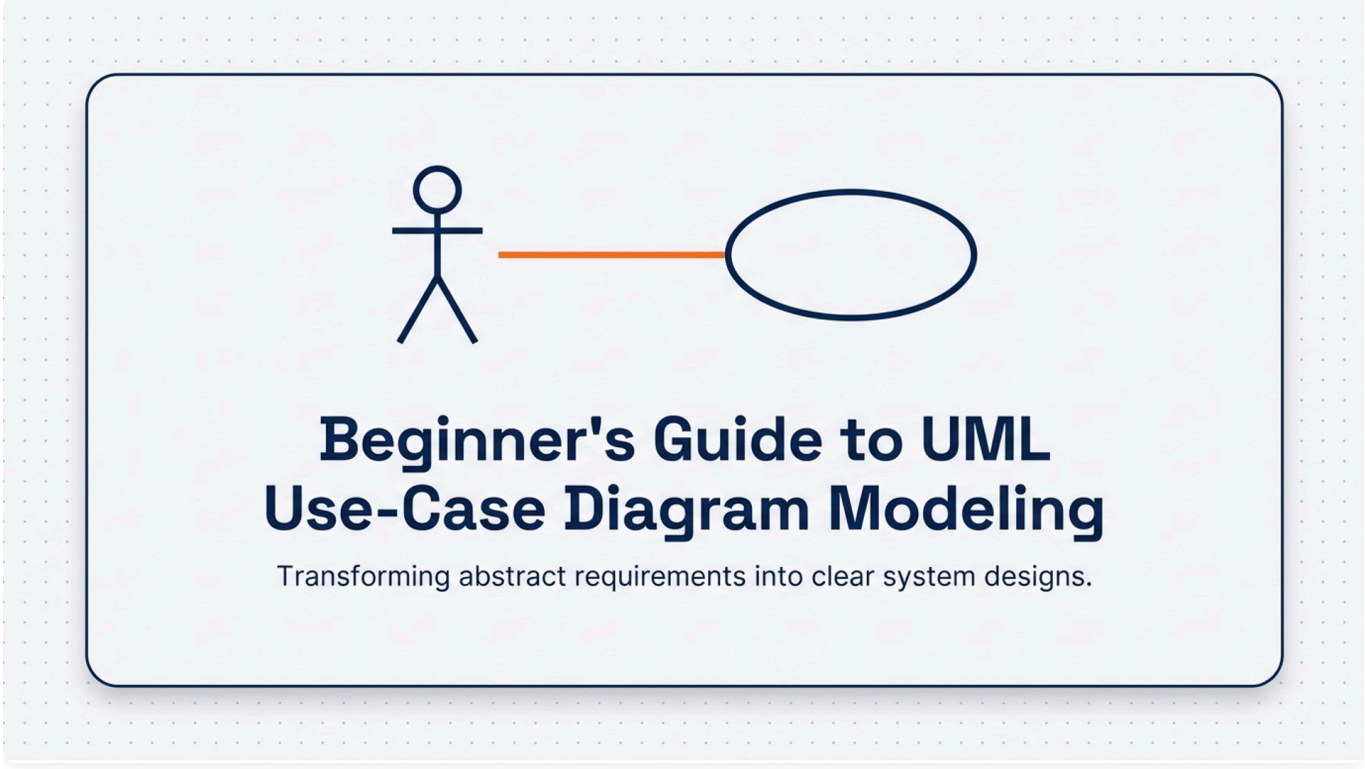


圖 4.3.1：使用案例塑模視覺解剖學。以系統邊界為界限，清楚劃分外部參與者與內部功能服務。

標準 UML 使用案例圖由四大視覺元素組成：

1. **系統邊界 (System Boundary - 矩形框)**：標示軟體系統的實體邊界，框內代表團隊必須編碼交付的功能，框外代表外部實體。

2. **參與者 (Actors - 人形符號)**：位於系統外部、直接與系統互動的實體（人類使用者、外部後端 API、自動化硬體）。
3. **使用案例 (Use Cases - 橢圓形)**：系統內部執行的完整業務邏輯，能為參與者產生具備可衡量價值的結果。
4. **關聯線 (Association Lines - 實線)**：連接參與者與其參與之使用案例。

4.3.2 商業價值 vs. 技術操作迷思

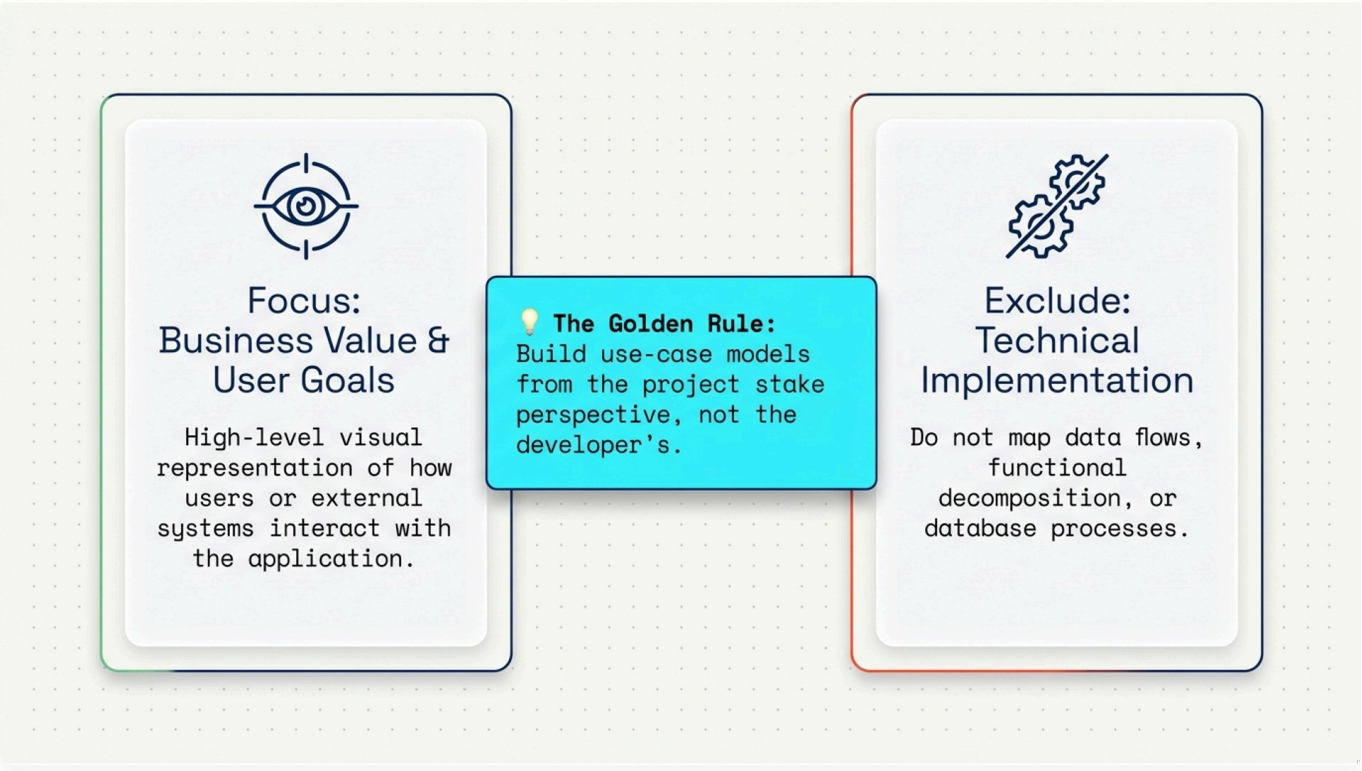


圖 4.3.2：商業價值與使用者目標導向。使用案例應當表述完整的商業目標，切忌退化為細微的技術點擊步驟。

新手架構師最常犯的錯誤是將使用案例圖畫成流程圖（例如把「輸入帳號」、「驗證密碼」、「點擊登入按鈕」畫成三個使用案例）：

- **反模式 (功能分解陷阱)**：拆分成「輸入信用卡號」、「檢查有效期限」、「扣款」。
- **最佳實踐 (商業目標導向)**：以單一具備業務價值的目標封裝：**「結帳並處理扣款 (Process Checkout Payment)」**。

4.3.3 參與者分類學：主要、次要與排程觸發

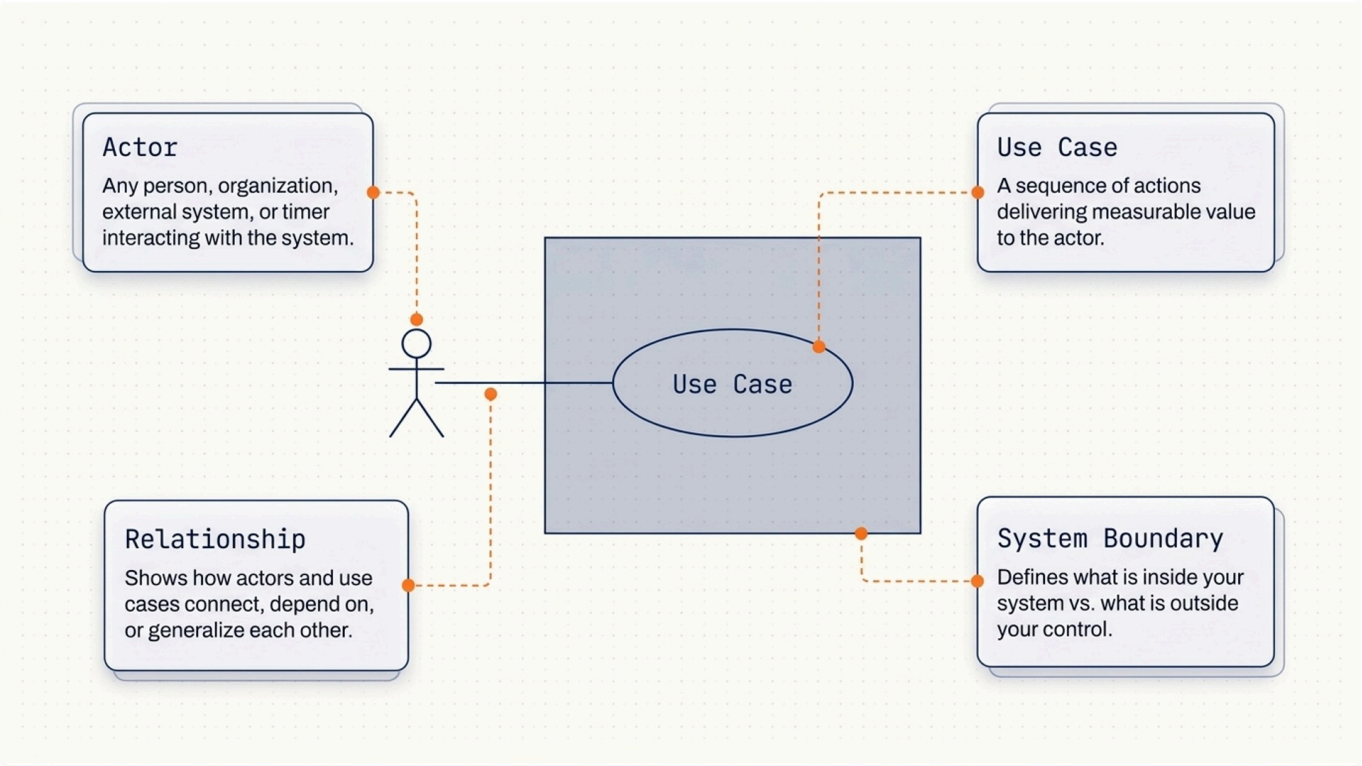


圖 4.3.3：參與者分類學。依據發起目標的主動性與輔助角色進行分類。

- **主要參與者 (Primary Actors)**：主動發起使用案例以達成個人或業務目標的實體（如顧客、學生、醫生），慣例繪製於邊界框**左側**。
- **次要/支援參與者 (Secondary / Supporting Actors)**：被動接收系統呼叫以提供基礎服務的外部系統（如金流閘道、第三方簡訊平台、統一編號驗證服務），慣例繪製於邊界框**右側**。
- **排程/自動化觸發者 (Automated Schedulers)**：系統內部的計時觸發器（如每日排程計費 Daemon）。

4.3.4 關聯矩陣：包含、擴充與一般化

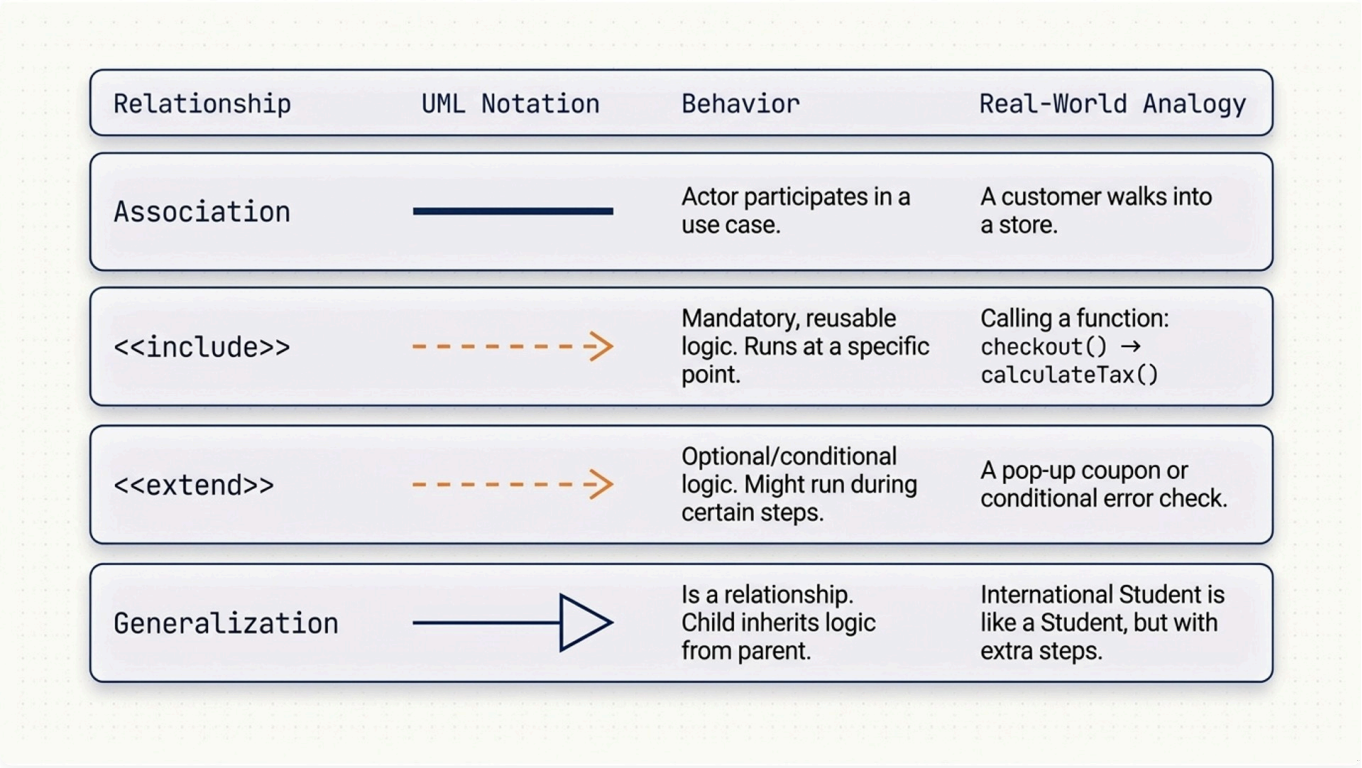


圖 4.3.4：使用案例關聯規則矩陣。嚴格界定 Association、<>、<> 與 Generalization 之箭頭方向與語義。

關聯類型	視覺符號語法	箭頭指向規則	架構核心語意
關聯 (Association)	實線 `——`	無或開放箭頭	參與者與使用案例之間的通訊互動連線。
包含 (`<<include>>`)	虛線開放箭頭 `- - - >`	基礎 → 包含目標	**必經共享子流程。** 基礎案例執行時「無條件強制執行」包含案例。
擴充 (`<<extend>>`)	虛線開放箭頭 `- - - >`	擴充 → 基礎案例	**條件式選用擴充。** 僅在滿足指定擴充點 (Extension Point) 時才觸發插入執行。
一般化 (Generalization)	實線空心三角形 `——▷`	子型別 → 父型別	**繼承關係。** 子參與者或特化使用案例自動繼承父層全部操作權限與結構。

4.3.5 明確命名語意：強動詞與單數角色

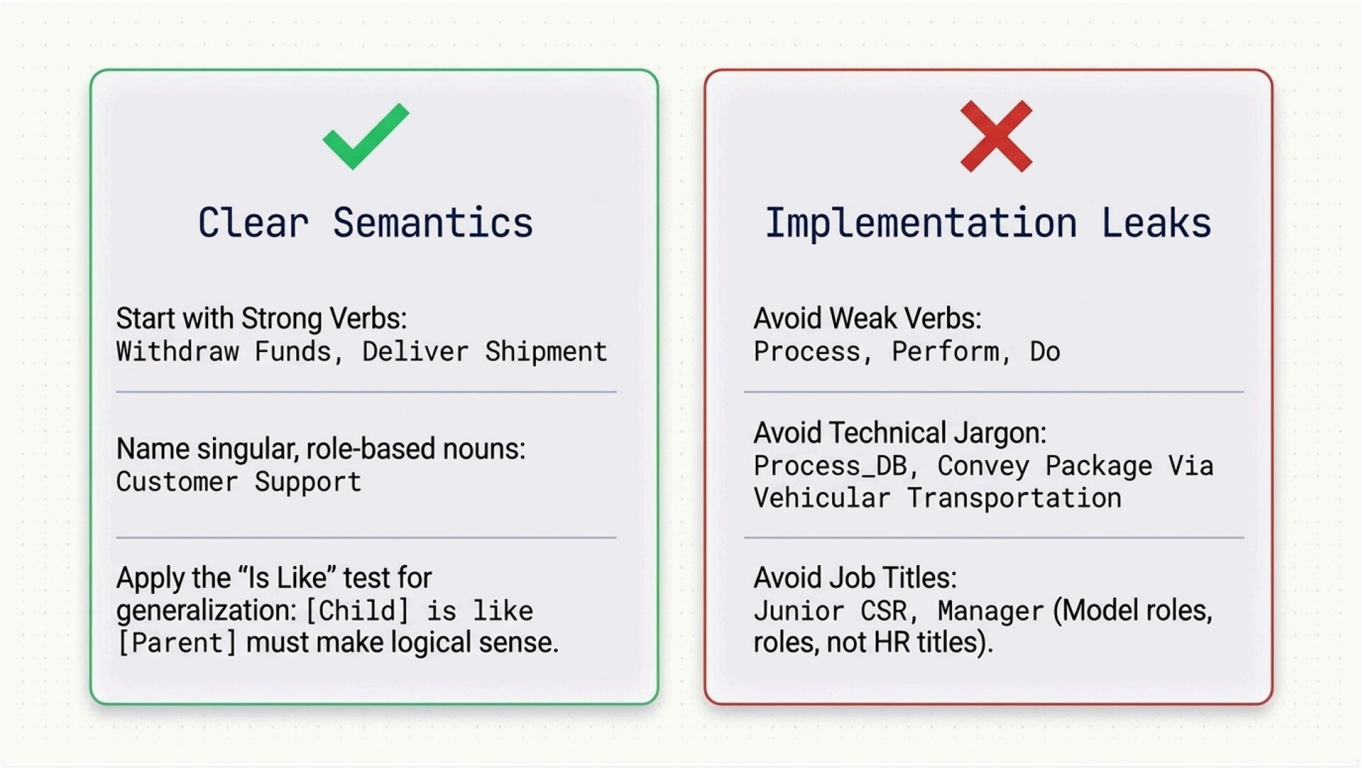


圖 4.3.5：明確命名語意原則。使用強動詞與單數角色命名，嚴禁模糊弱動詞、技術黑話與 HR 行政職稱。

評估向度	良好設計實踐 (Recommended)	致命壞味道反模式 (Anti-Pattern)
使用案例動詞	強動詞明確表達商業目標：`提領現金`、`配送包裹`、`下單訂購`	模糊弱動詞：`處理`、`執行`、`做`、`管理`
領域術語	純粹業務語言：`驗證使用者`、`產生訂單收據`	底層技術黑話：`Process_DB_Transaction`、`SQL_Insert`
參與者名稱	單數業務角色：`顧客服務人員`、`計費主管`、`外送員`	部門組織名稱或 HR 職級：`客服部`、`二級助理工程師`
一般化繼承	符合「是一種 (is-a / is-like)」常理：`國際學生 繼承 學生`	為了少畫幾條線而將無關角色硬塞繼承關係

4.3.6 安全嵌套上限與三大防護原則

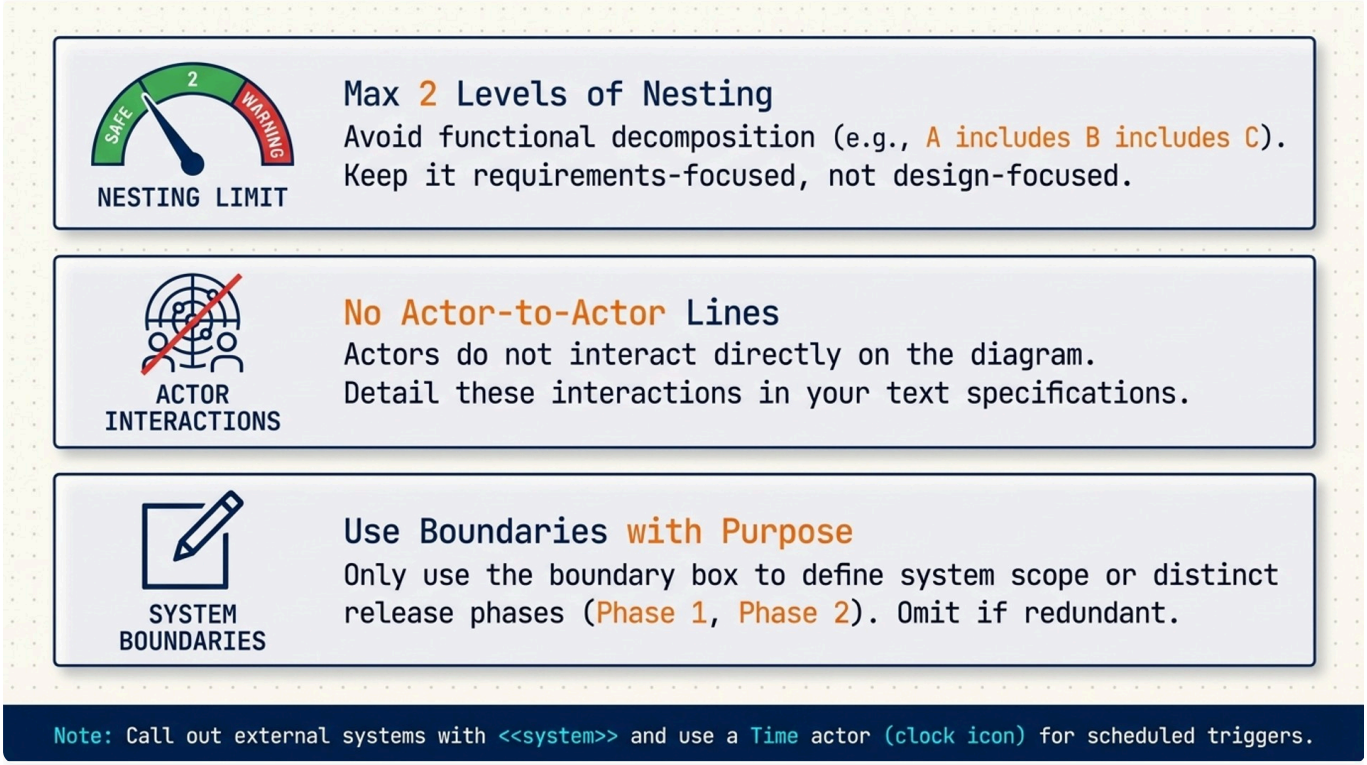


圖 4.3.6：嵌套深度安全限制。嚴禁多層級的 include 鏈結，關聯深度應嚴格限制在 1–2 層以內。

三大架構防護原則確保使用案例圖維持專業清晰：

1. **最多兩層嵌套：** 保持 include 與 extend 鏈結極度扁平（如 A includes B, B includes C 會讓圖表退化成雜亂的流程圖）。
2. **嚴禁參與者互連：** 參與者只能跨越邊界與使用案例通訊，人與人之間的溝通應寫在情境敘述文字中。
3. **有目的地使用系統邊界框：** 邊界框明確宣告交付範疇或劃分敏捷疊代版本（第一期 MVP vs. 第二期擴充）。

4.3.7 大學選課系統實戰案例與標準規格書

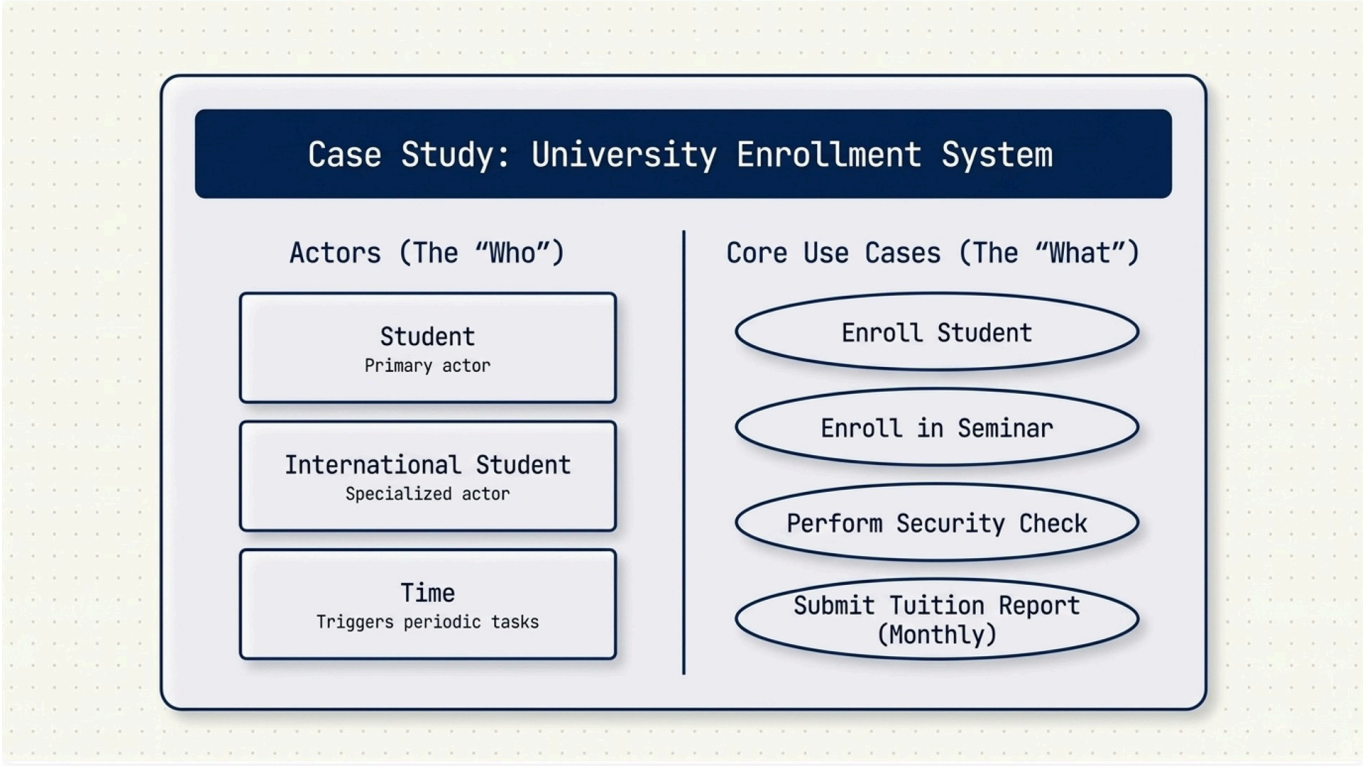


圖 4.3.7：大學學生選課案例盤點。在連線前先清晰盤點系統參與者 (The Who) 與核心服務 (The What)。

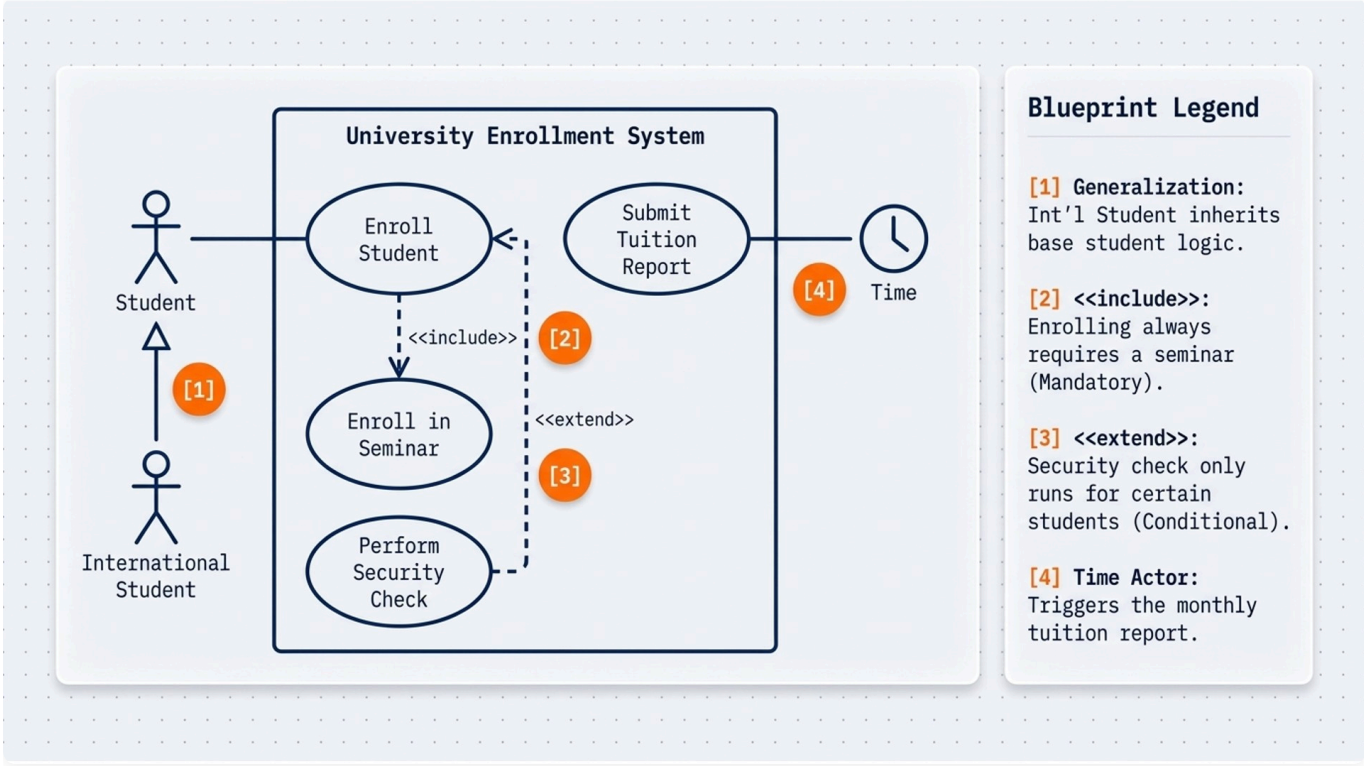
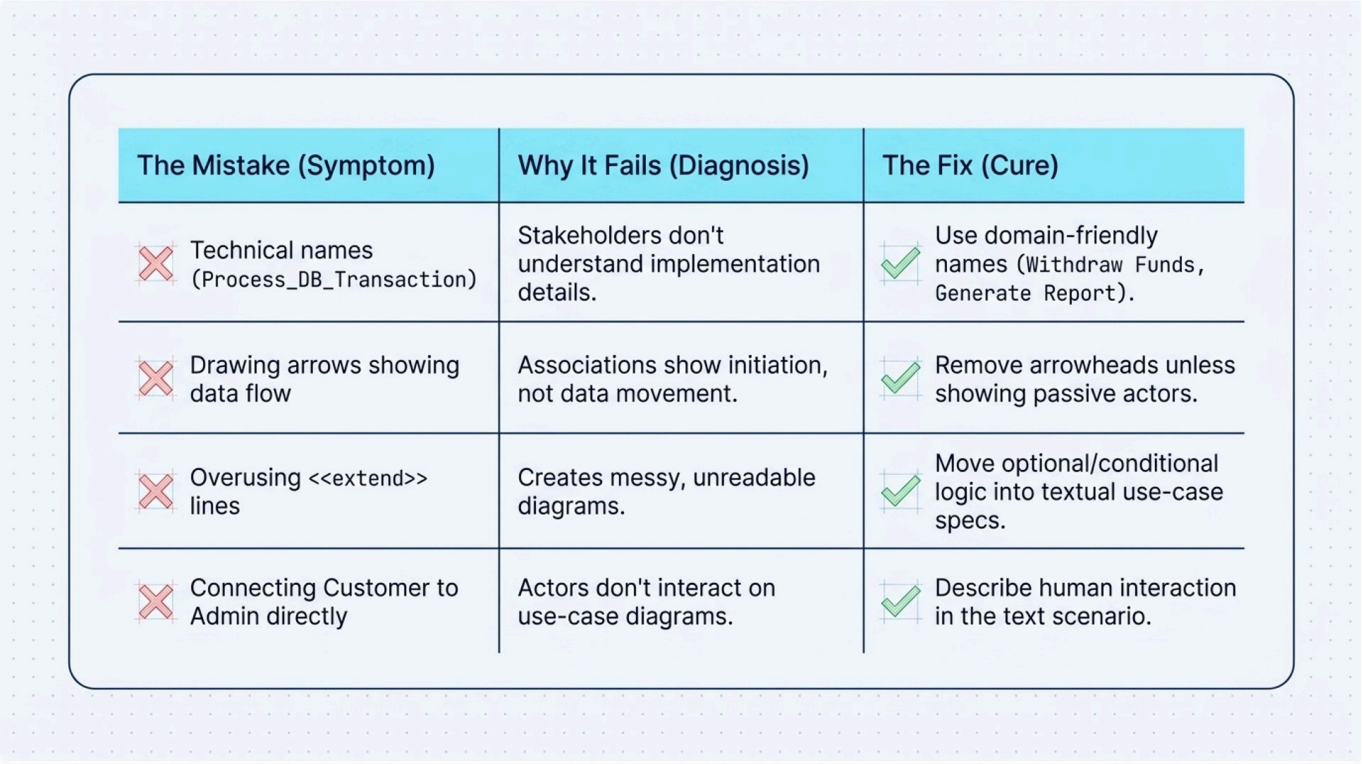


圖 4.3.8：大學選課完整使用案例模型。展示角色泛化繼承、強制包含、條件擴充與定時排程觸發。

工業級使用案例規格書 (Use Case Specification)：學生選課

規格書欄位	具體工程規格細節與合約定義
案例代號與名稱	**UC-04: 學生選課 (Enroll Student in Course)**
主要參與者	學生 (Student)
利害關係人與目標	• **學生**：成功選修學位必修或選修課程，取得畢業學分。 • **教務處註冊組**：確保選課人數未超額、先修擋修規則嚴格落實。 • **開課系所**：確保教學班級規模符合師生比與教室安全容量。
前置條件 (Preconditions)	1. 學生已成功登入教務系統，且無欠費停權處分。 2. 當前時間處於該年級之正式開放選課時段內。
成功保證 (Postconditions)	學生正式列入課程修課名單、教室佔有名額扣減 1、產生學分應繳帳單。
觸發條件 (Trigger)	學生在選課介面點擊「確認加選此課程」按鈕。
主要成功情境 *(Happy Path)*	1. 學生瀏覽全校開課清單並檢視尚有名額之課程。 2. 學生選定目標班別並送出加選申請。 3. 系統驗證該生已修畢且通過該課程之所有先修條件。 4. 系統檢查目標課程當前剩餘名額 (Quota > 0)。 5. 系統保留名額，將學生帳號寫入修課名冊 (Course Roster)。 6. 系統非同步發送結算訊息至計費系統增加學分費。 7. 系統回傳加選成功收據畫面，顯示當前已選學分數。
擴充與例外分支 *(Extensions)*	**3a. 先修課程未通過：** 1. 系統顯示未符合之先修科目代碼與成績。 2. 系統提示學生可申請先修豁免單，終止本次加選。 **4a. 課程名額已額滿 (Quota = 0)：** 1. 系統提示本班級名額已滿。 2. 系統啟動 `<<extend>>` UC-05: 加入候補排隊名單。 3. 若學生確認，將其加入候補隊列 (Waiting List)。 **6a. 出納計費系統斷線：** 1. 系統將計費事件寫入本地交易 Outbox 表。 2. 依舊判定加選成功，記錄稽核日誌並背景重試計費。

4.3.8 使用案例七大常見致命反模式



The Mistake (Symptom)	Why It Fails (Diagnosis)	The Fix (Cure)
✗ Technical names (`Process_DB_Transaction`)	Stakeholders don't understand implementation details.	✓ Use domain-friendly names (`Withdraw Funds`, `Generate Report`).
✗ Drawing arrows showing data flow	Associations show initiation, not data movement.	✓ Remove arrowheads unless showing passive actors.
✗ Overusing `<<extend>>` lines	Creates messy, unreadable diagrams.	✓ Move optional/conditional logic into textual use-case specs.
✗ Connecting Customer to Admin directly	Actors don't interact on use-case diagrams.	✓ Describe human interaction in the text scenario.

圖 4.3.9：使用案例塑模常見反模式。包括箭頭反向、無邊界框、功能分解與參雜 UI 點擊。

1. **擴充箭頭指錯方向**：誤將 `<<extend>>` 箭頭從基底指向擴充。請牢記：基底案例對擴充案例應完全一無所知！
2. **功能分解陷阱**：把使用案例當成程序代碼逐行拆解。
3. **缺少系統邊界框**：橢圓漂浮在空中，無法定義專案範圍。
4. **參與者畫在框內**：將外部系統誤認為內部模組。
5. **過度使用一般化繼承**：為使用案例建立複雜的深層繼承樹。
6. **參雜 UI 細節操作**：寫出「點擊送出按鈕」而非「提交訂單」。
7. **孤兒使用案例**：完全沒有連接任何參與者的無效橢圓。

4.3.9 專家最佳實踐檢核清單

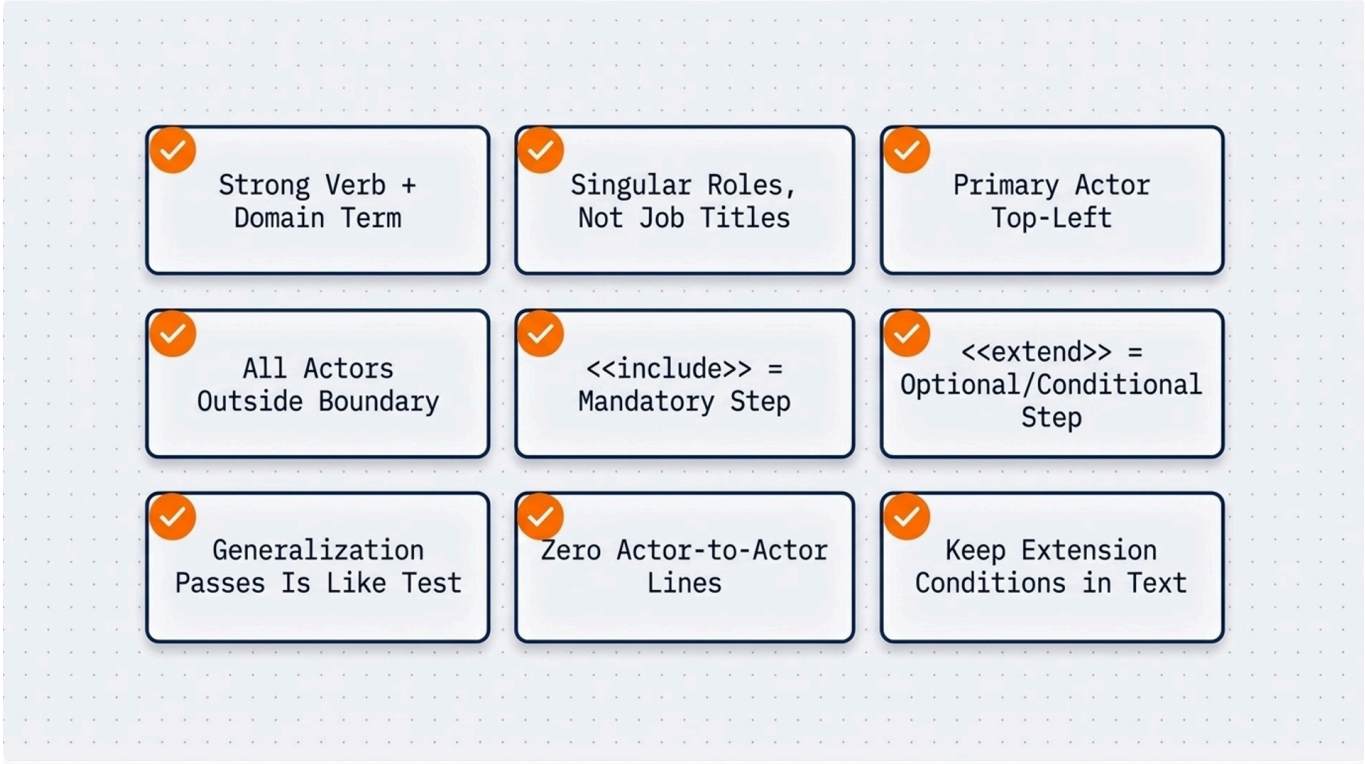


圖 4.3.10：使用案例品質檢核清單。在向團隊展示前，確認通過九項架構檢驗點。

檢查項目	核心品質檢核原則	驗證標準
1	**強動詞 + 名詞**	每個使用案例是否皆以具備商業價值的及物動詞與名詞命名？
2	**單數業務角色**	參與者是否採用單數角色名稱，而非行政 HR 職級？
3	**主要參與者置左**	主要發起參與者是否統一排列於邊界框左側？
4	**參與者置於框外**	所有參與者是否皆嚴格繪製於系統邊界矩形之外？
5	**嚴格包含 (Include)**	`<<include>>` 是否嚴格保留給「強制必經」之共享子流程？
6	**嚴格擴充 (Extend)**	`<<extend>>` 是否嚴格保留給「條件可選」之擴充行為？
7	**繼承常理檢驗**	參與者一般化繼承是否皆通過常理「是一種」檢驗？
8	**禁止人連人**	參與者之間是否完全零連線（禁止人連人）？
9	**擴充條件寫在文字**	細節擴充條件與例外規則是否完整記錄於文字規格書中？

4.3.10 溝通第一思維

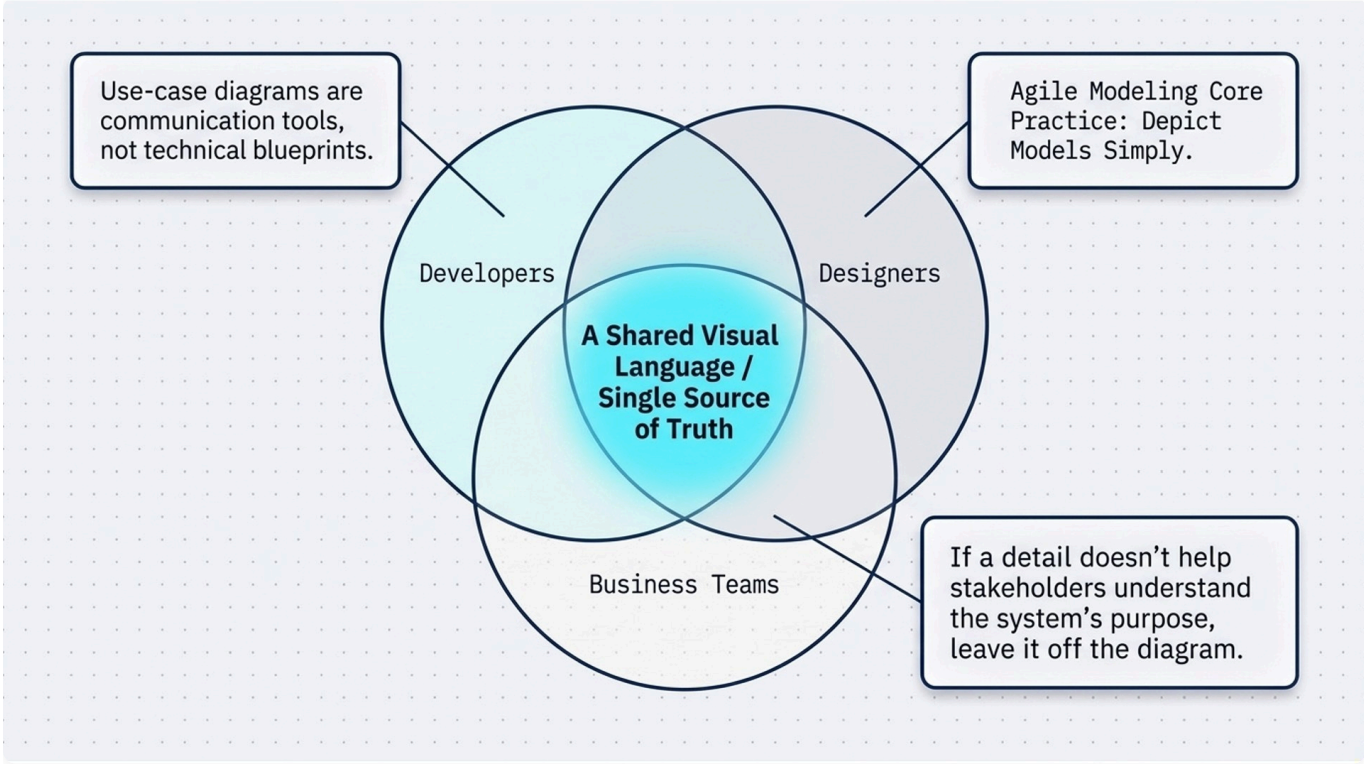


圖 4.3.11：溝通第一思維。使用案例模型是連結非技術商業利害關係人、UX/系統設計師與後端工程師之關鍵橋樑。

4.3.11 生成式 AI 提示詞指引與外送平台使用案例模型實戰

引導 AI 產出工業級使用案例模型時，必須給予明確邊界、語法規則與情境描述：

```
你是一位資深系統分析架構師。
請根據以下需求描述，產生合規且簡練的 PlantUML 使用案例圖。

塑模約束條件：
– 使用 <<extend>> 與 <<include>> 建構模組化關係。
– 嚴禁功能分解（禁止出現「點擊按鈕」等細節）。
– 以 rectangle "Food Delivery System" 劃定邊界。
– 外部金流閘道置於邊界外作為次要參與者。

需求描述：
顧客瀏覽菜單、將餐點加入購物車並結帳下單。下單時
系統強制授權扣款金流；可選套用優惠券。
餐廳管理菜單並審核訂單；外送員承接配送任務。
```

外送平台使用案例模型圖 (Food Delivery Use Case Model)

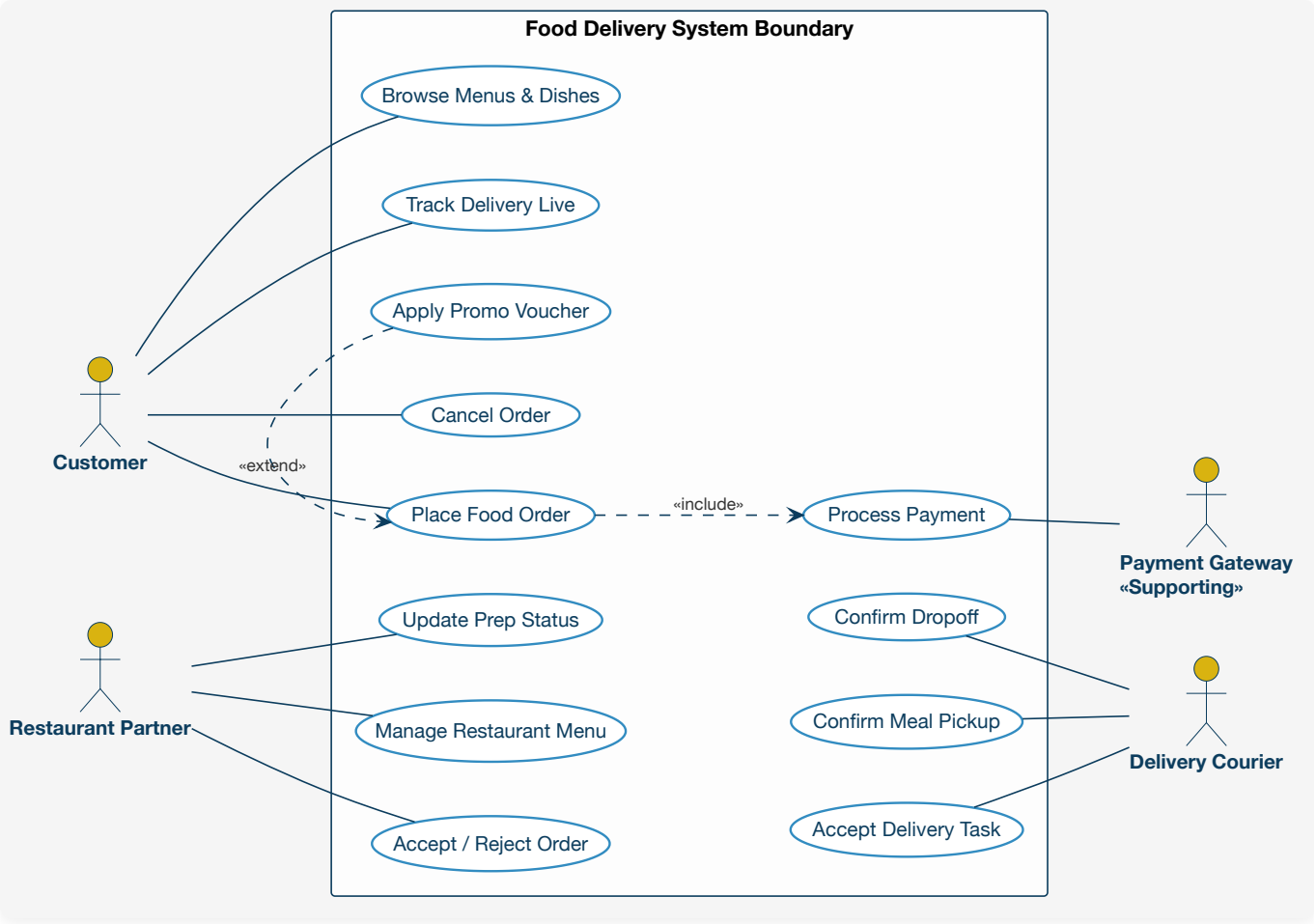


圖 4.3.12：外送平台使用案例圖。劃定系統邊界、顧客核心交易、餐廳接單、外送員承接任務、強制金流授權與可選優惠券擴充。

```
@startuml
left to right direction
actor Customer
actor "Restaurant Partner" as Restaurant
actor "Payment Gateway\n«Supporting»" as Payment
actor "Delivery Courier" as Courier

rectangle "Food Delivery System" {
  usecase "Browse Menus" as UC_Browse
  usecase "Place Food Order" as UC_PlaceOrder
  usecase "Apply Promo Voucher" as UC_Voucher
  usecase "Track Delivery Live" as UC_Track
  usecase "Manage Menu" as UC_Menu
  usecase "Accept / Reject Order" as UC_AcceptOrder
  usecase "Process Payment" as UC_Payment
  usecase "Accept Delivery Task" as UC_AcceptTask
}

Customer -- UC_Browse
Customer -- UC_PlaceOrder
Customer -- UC_Track
Restaurant -- UC_Menu
Restaurant -- UC_AcceptOrder

UC_PlaceOrder ..> UC_Payment : <<include>>
UC_Voucher .> UC_PlaceOrder : <<extend>>
UC_Payment -- Payment
UC_AcceptTask -- Courier
@enduml
```

4.3.12 診斷式案例實戰：抓出使用案例模型的致命破綻 (Diagnostic Case Study)

即使是看似直觀的使用案例圖，若缺乏嚴謹的架構邊界意識，極易退化為充滿破綻的低階 UI 操作清單。

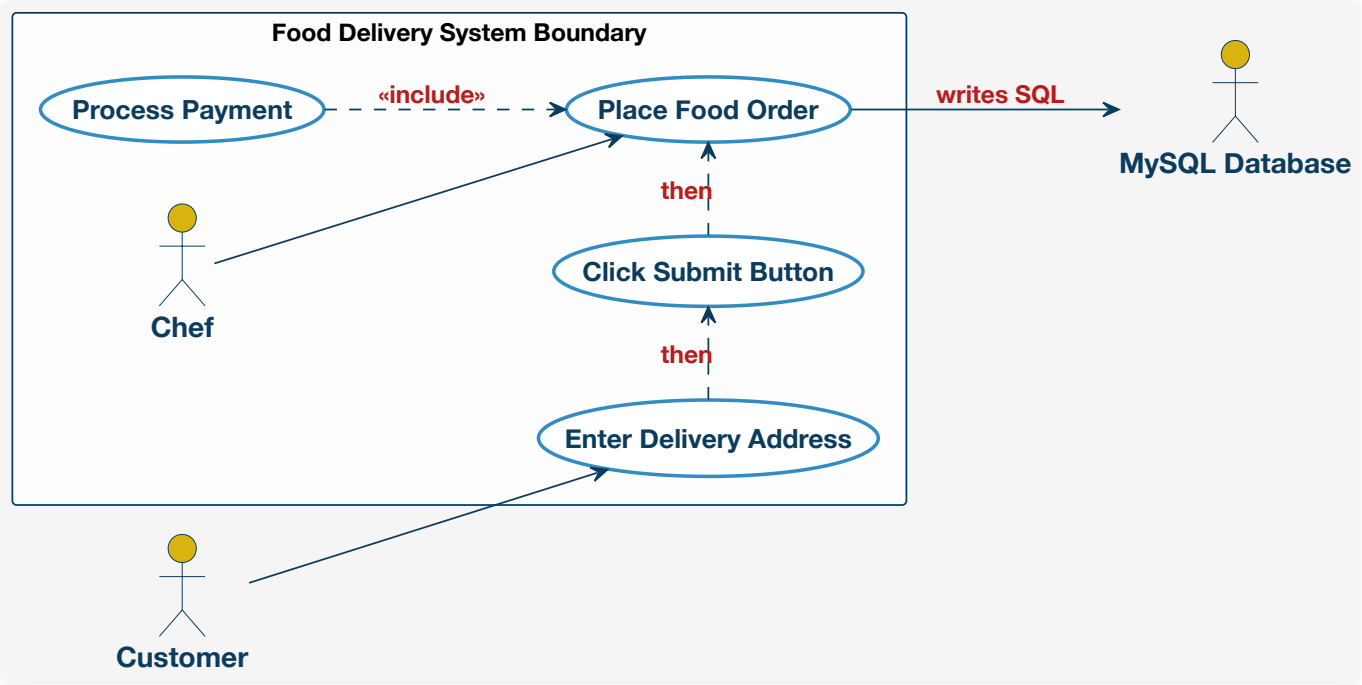


圖 4.3.11：具備四大典型缺陷的使用案例圖草稿。

架構診斷審查：四大致命反模式

1. **系統邊界混淆 (Boundary Collapse)：**
   - *破綻：* 外部合作夥伴 `PaymentGateway`（第三方金流閘道）被錯誤置於系統邊界矩形框內部。
   - *修正：* 系統邊界矩形框代表「我們團隊負責編寫與部署的軟體範疇」。所有外部系統、第三方 API 與終端使用者皆屬於外部參與者 (Actors)，必須嚴格置於系統邊界框之外。
2. **CRUD 功能碎片化 (CRUD Fragmentation)：**
   - *破綻：* 將「輸入密碼」、「點擊登入按鈕」與「驗證帳號」拆解為三個獨立的使用案例橢圓。
   - *修正：* 使用案例必須代表完整、為參與者帶來獨立商業價值的目標（User Goal），嚴禁將單一操作畫面上的表單輸入與按鈕點擊拆為細碎案例。應收斂為單一的 `顧客登入` 或包含於交易流程中。
3. **Include 依賴語意濫用 (Include Dependency Inversion / Abuse)：**
   - *破綻：* 箭頭由細節操作指向主業務，或將非必要之輔助操作標記為 `<<include>>`。
   - *修正：* `<<include>>` 箭頭必須由「基底案例」指向「被包含之子案例」，且代表不可或缺之強制流程（例如：下單強制包含扣款）。
4. **參與者主客顛倒 (Actor Inversion)：**
   - *破綻：* 未區分主要參與者（啟動者）與次要參與者（輔助者），甚至將系統內部的後端排程服務誤畫為人形參與者。
   - *修正：* 左側繪製發起業務目標的主要參與者，右側繪製提供被動支援的次要外部系統。

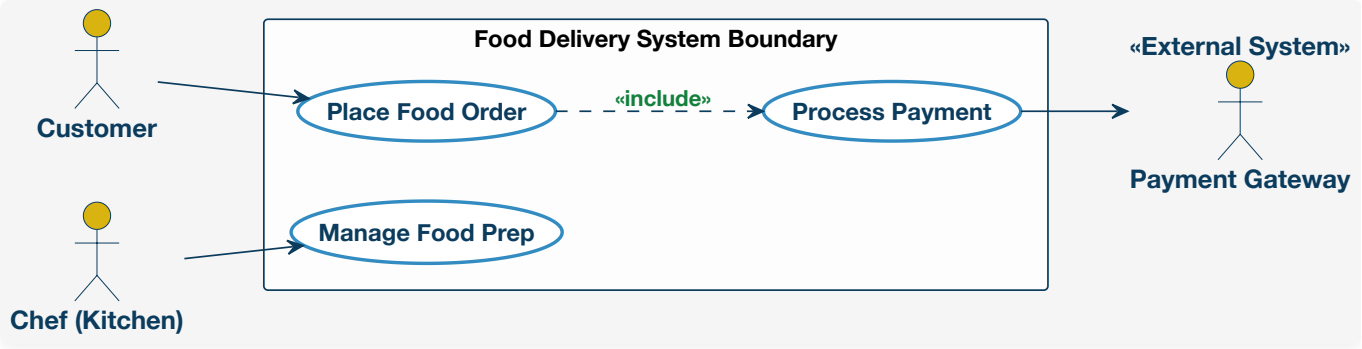


圖 4.3.12：修正後之標準使用案例模型。邊界清晰、參與者職責分明，且全面以商業目標為導向。

4.3.13 互動實作：運用 AI 演進使用案例模型 (Pair Hands-On Activity)

- **實作目標：** 在既有的外送平台提示詞中引入一項全新的業務變更需求，並審查 AI 產出的 PlantUML 規格：
  i. *選項 A (預約排程訂餐)：* 允許顧客預約未來 7 天內的特定時段送達（使用 `<<extend>>` 與擴充點守衛）。
  ii. *選項 B (在線客服即時交談)：* 新增顧客與平台客服、外送員之即時交談使用案例（外部次要參與者正確置於邊界外）。
  iii. *選項 C (多人團購分帳)：* 允許主揪發起多人購物車並各自結帳。
- **架構檢核清單：**
  - 邊界紀律：新增的外部輔助人員或服務是否嚴格置於系統矩形邊界之外？
  - 關聯精確性：選擇性功能是否正確採用 `<<extend>>`，且箭頭指向基底案例？

- 目標層次：案例名稱是否維持動詞+名詞的高階商業目標，未退化為按鈕點擊？

🤔 觀念檢核測驗 (CCQ 3) — 使用案例關聯：包含 vs. 擴充

問題

在 UML 使用案例模型中，若使用案例 A 與使用案例 B 之間存在 `<<include>>` 關聯（A 指向 B），這在軟體架構上代表什麼意義？

- A. 使用案例 B 是選擇性的，僅在特定異常狀況下才會由 A 觸發執行。
- B. 每當使用案例 A 執行時，使用案例 B 所定義的行為「必定且強制」會被納入執行。
- C. 使用案例 A 繼承了使用案例 B 的所有屬性與操作方法。
- D. 使用案例 B 是外部第三方系統，不屬於軟體邊界範疇。

Interactive Activity (線上作答)



▶ 點擊展開參考答案與解析

4.4 結構與領域類別模型 (Class Modeling)

如果說使用案例圖定義了軟體「做什麼」，那麼**類別圖 (Class Diagram)** 則奠定了系統「由什麼實體組成」的靜態架構骨幹。類別圖定義領域詞彙、對應關聯式資料庫結構，並規範物件導向程式碼的編譯期合約。

4.4.1 類別結構解剖：三層隔間語法

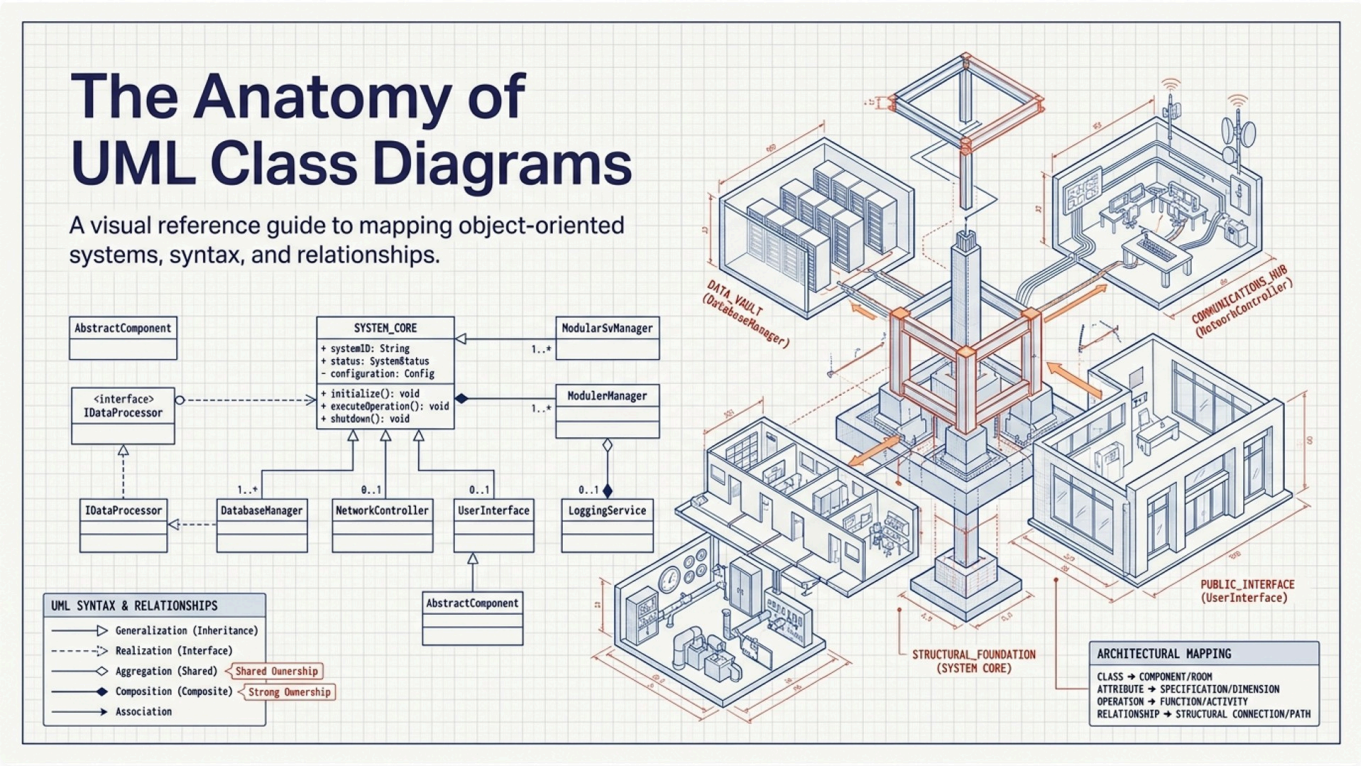


圖 4.4.1：UML 類別圖解剖學。標準類別方塊分為三層：類別名稱、屬性變數與方法簽名。

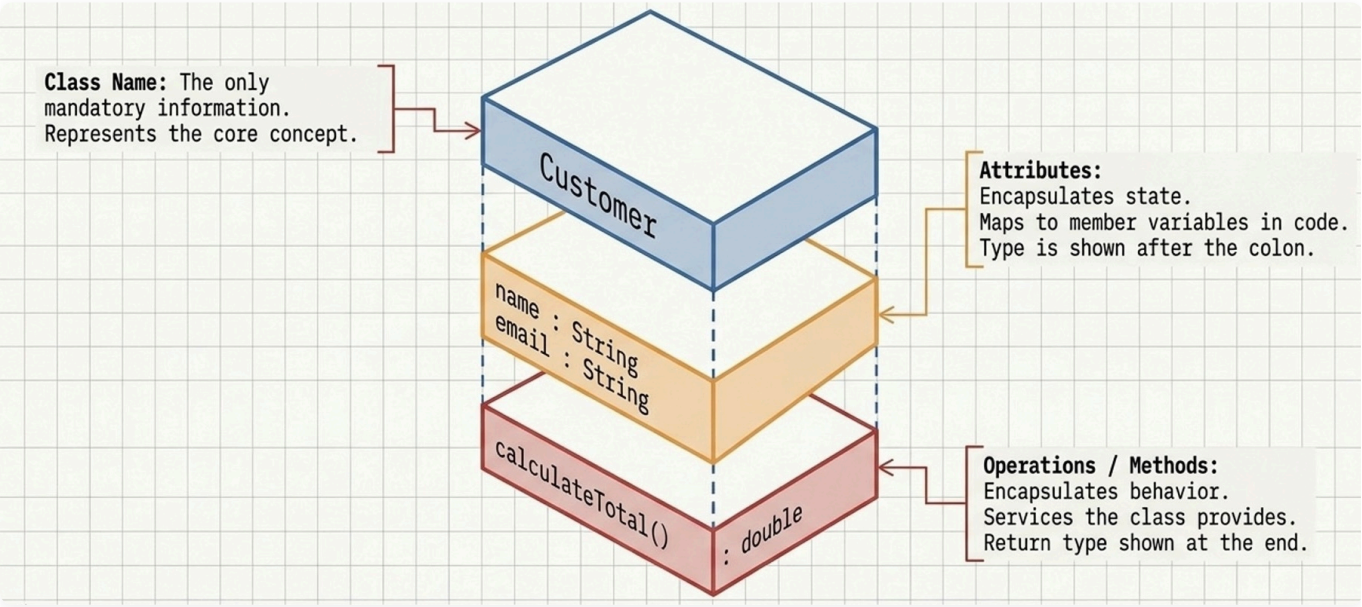


圖 4.4.2：類別方塊細部語法。包含屬性型別標註、預設值、方法參數方向性與可見度標記。

標準 UML 類別方塊被劃分為三個水平隔間：

1. **頂層隔間 (類別名稱)**：宣告類別識別名稱。抽象類別以*斜體*表示或標註 `{abstract}`；介面則加上 `<<interface>>` 造型飾詞。
2. **中層隔間 (屬性清單)**：描述實體狀態資料，標準語法為：`[可見度] 屬性名稱 : 型別 [重數] [= 預設值] [{特性修飾}]` *(範例：`- balance : Money = 0.00 {readOnly}`)*

3. **底層隔間 (操作方法清單)**：宣告可被外部調用之行為合約，標準語法為：`[可見度] 方法名稱 ( [方向] 參數名 : 參數型別 , ... ) : 回傳型別` *(範例：`+ processPayment(in amount: Decimal, in token: String) : PaymentResult`)*

4.4.2 可見度修飾詞與資訊隱藏

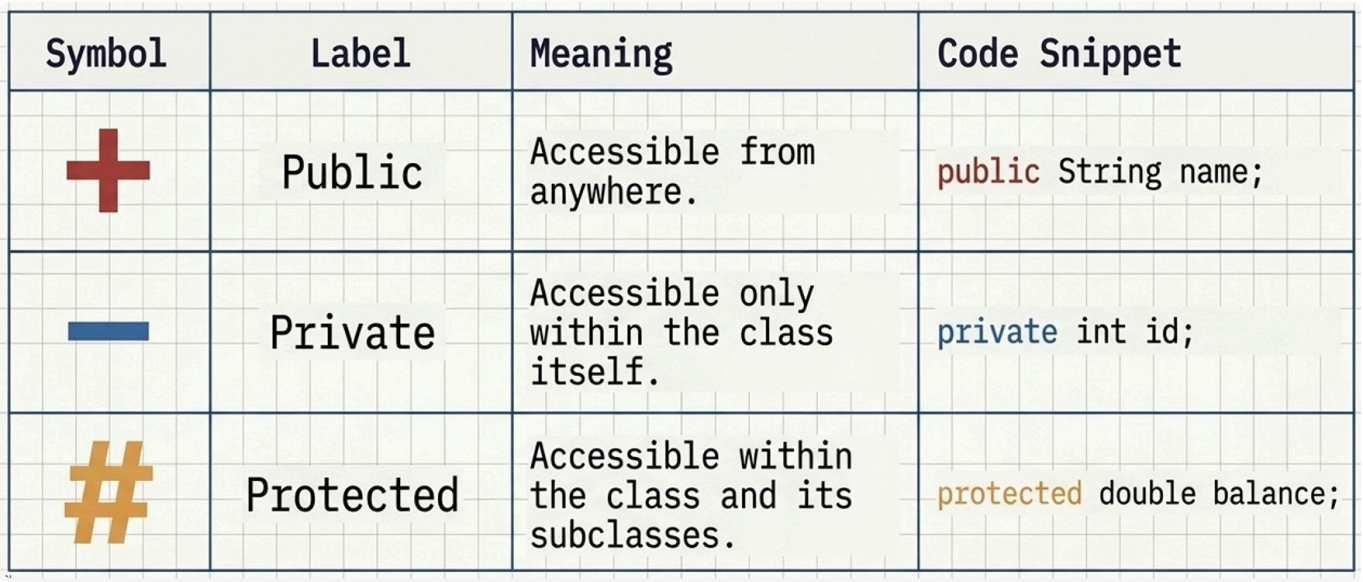


圖 4.4.3：可見度修飾詞矩陣。定義 Public (+)、Private (-)、Protected (#) 與 Package (~) 之封裝邊界。

符號標記	可見度名稱	程式語言對應語法	軟體架構封裝範疇
`+`	**公開 (Public)**	`public`	任何外部物件皆可直接公開調用。
`-`	**私有 (Private)**	`private`	嚴格封裝於類別內部，外部不可見。
`#`	**受保護 (Protected)**	`protected`	僅該類別及其衍生之繼承子類別可存取。
`~`	**套件 (Package)**	`default` (套件私有)	僅同一個套件或命名空間內的類別可存取。

4.4.3 物件關聯分類學與連接符號速查

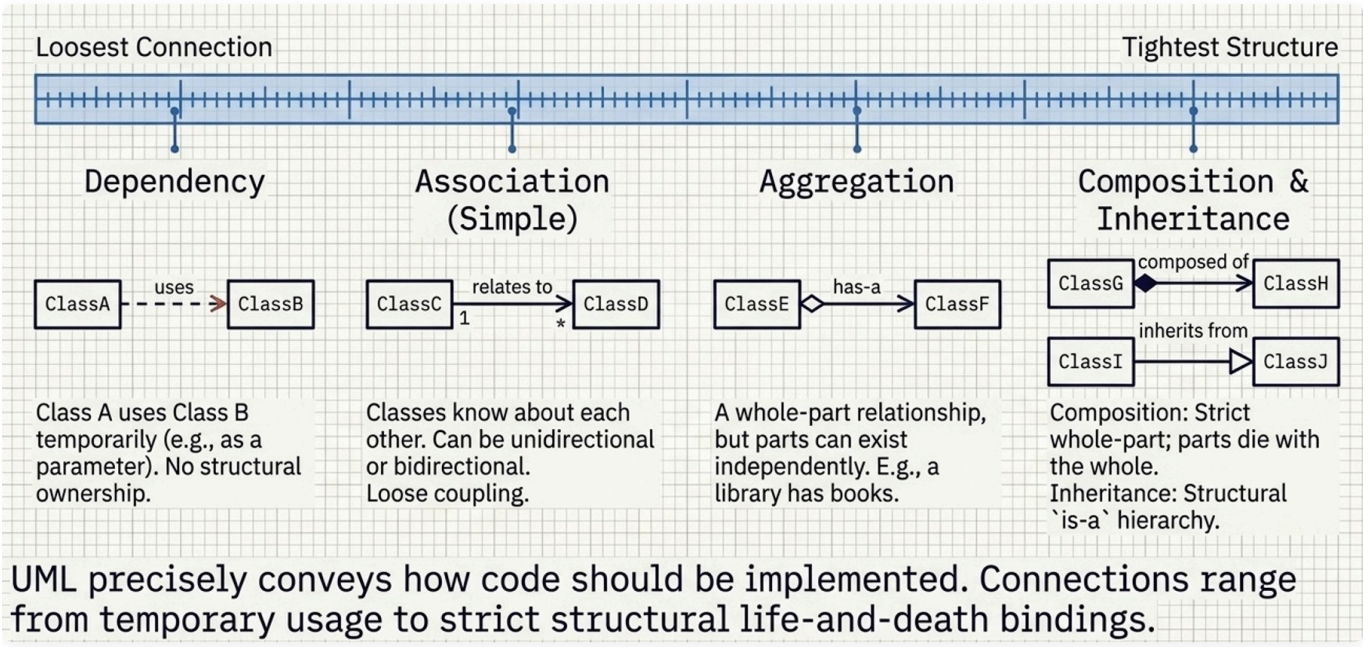


圖 4.4.4：物件導向關聯分類樹。由暫時性微弱依賴至永久生命週期綁定之光譜排列。

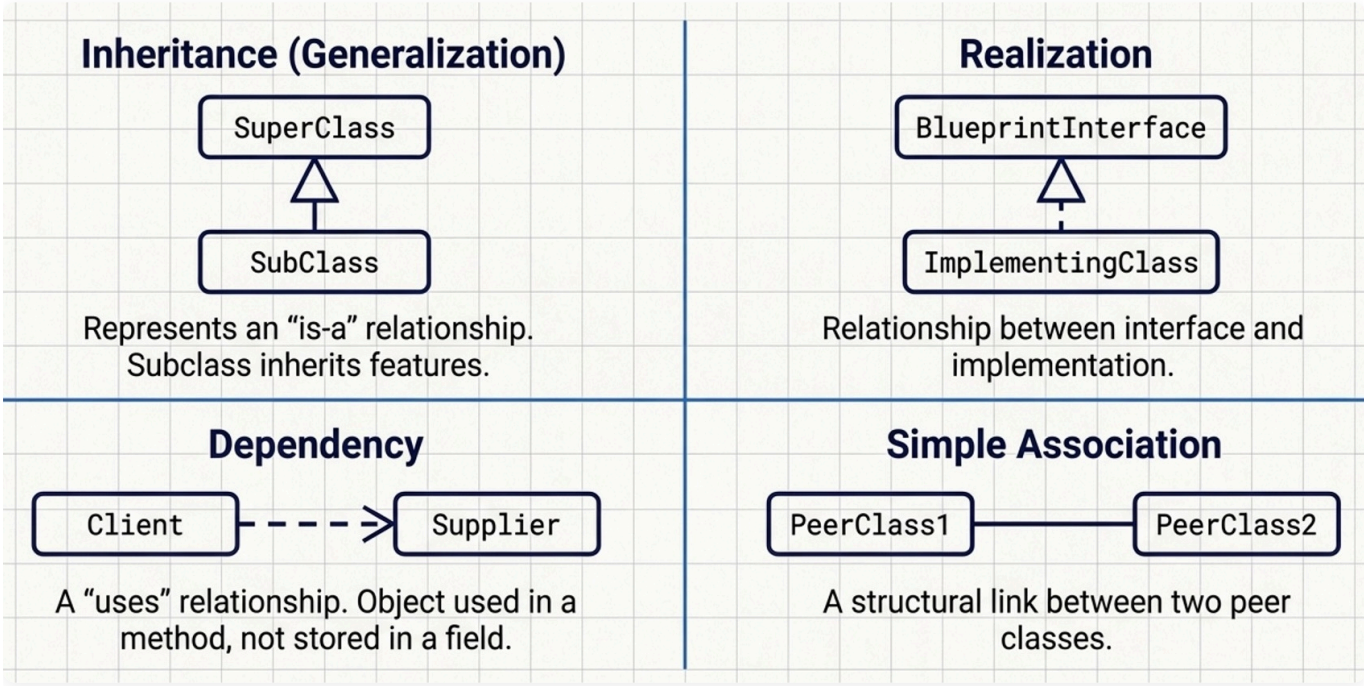


圖 4.4.5：UML 類別連接線速查手冊。關聯、聚合、組合、一般化繼承、介面實作與依賴之端點幾何符號標準。

關係類型	視覺語法符號	耦合強弱程度	生命週期相依性與架構意涵
相依 (Dependency)	虛線箭頭 `- - ->`	最弱 (暫時性)	暫時使用。物件透過方法參數、區域變數傳遞。
一般關聯 (Association)	實線 `———`	中等 (同儕同伴)	結構性持有。透過實例屬性常駐引用對方。
聚合 (Aggregation)	空心菱形 `———◇`	中強 (整體與部分)	**弱所有權 (共享部分)。** 部分可脫離整體而獨立存活。
組合 (Composition)	實心菱形 `———◆`	最強 (物理包含)	**強所有權 (獨佔部分)。** 整體消亡時部分一併銷毀。
一般化 (Generalization)	實線空心三角形 `—▷`	繼承關係 (`is-a`)	子類別繼承父類別的所有屬性、方法與關聯合約。
實現 (Realization)	虛線空心三角形 `- -▷`	介面實作	具體類別履行介面所定義的所有抽象操作合約。

聚合 vs. 組合：生命週期所有權測試

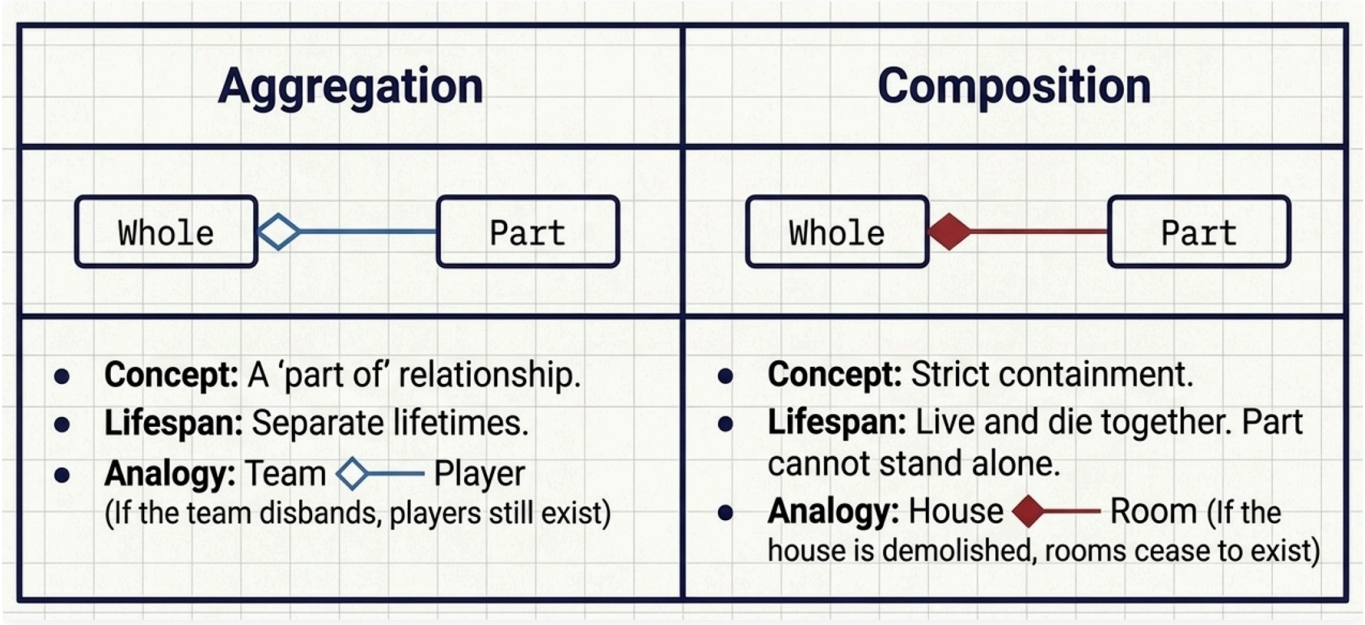


圖 4.4.6：聚合 (Aggregation) 與組合 (Composition) 深度辨析。判定標準為「生命週期所有權」：當整體容器消亡時，內部元件是否隨之同歸於盡？

- **組合 (Composition, 實心菱形 ◆)**：代表排他性實體所有權。在任何時間點，部分物件僅能屬於一個整體。一旦容器被銷毀，其內部部分物件必定被一併銷毀。
  - *實務範例*：訂單 (`Order`) 與其訂單細項 (`OrderItem`)。在資料庫中刪除一筆訂單，所有關聯的訂單明細必須 Cascade Delete 同步清除。
- **聚合 (Aggregation, 空心菱形 ◇)**：代表共享的弱擁有權。部分物件具備獨立的生命週期。

- *實務範例*：訂單（`Order`）與外送員（`Courier`）。外送員在送完訂單或訂單被取消後，依然獨立存在於系統中接取下一筆訂單。

4.4.4 多重性限制 (Multiplicity) 與導覽性

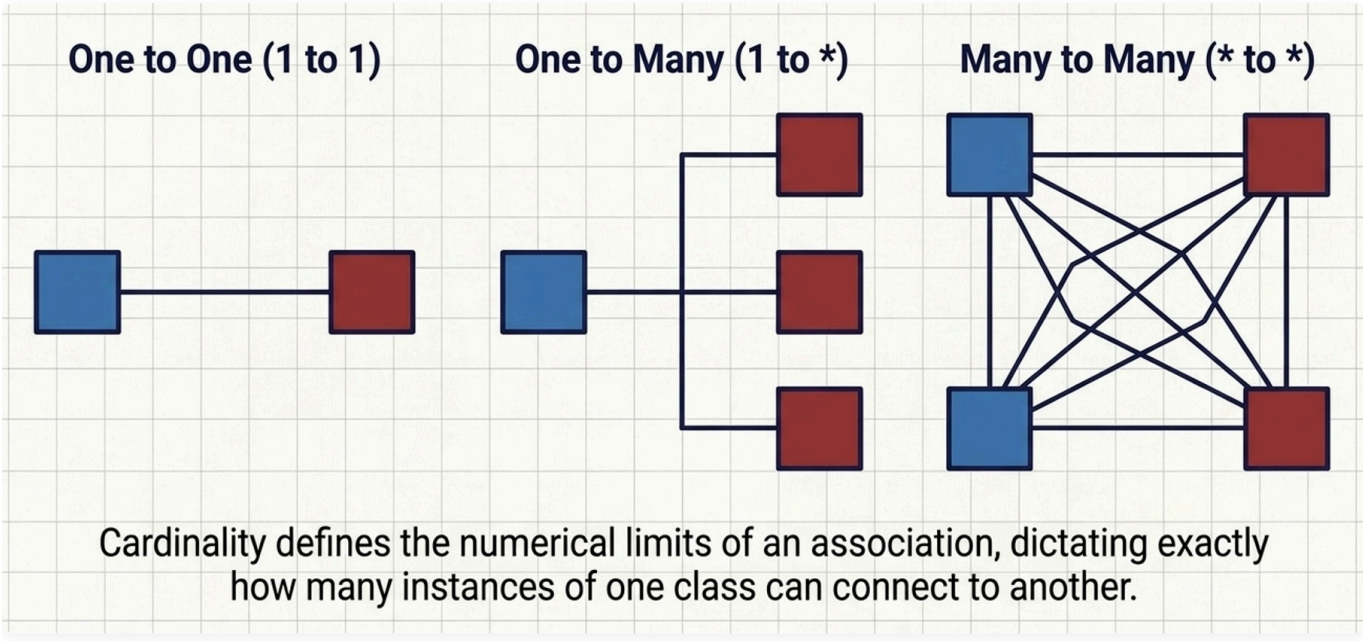


圖 4.4.7：多重性限制與導覽性。標註於關聯線端點之數量下限與上限。

- `1`：剛好一個（Mandatory，不可為 null）。
- `0..1`：零或一個（Optional，可為 null 的指標或引號）。
- `*` 或 `0..*`：零或多個（Unbounded 集合、List、Set）。
- `1..*`：至少一個以上（Non-empty 集合）。
- `m..n`：特定範圍限制（如 `2..4`）。

4.4.5 外送平台領域模型實戰案例

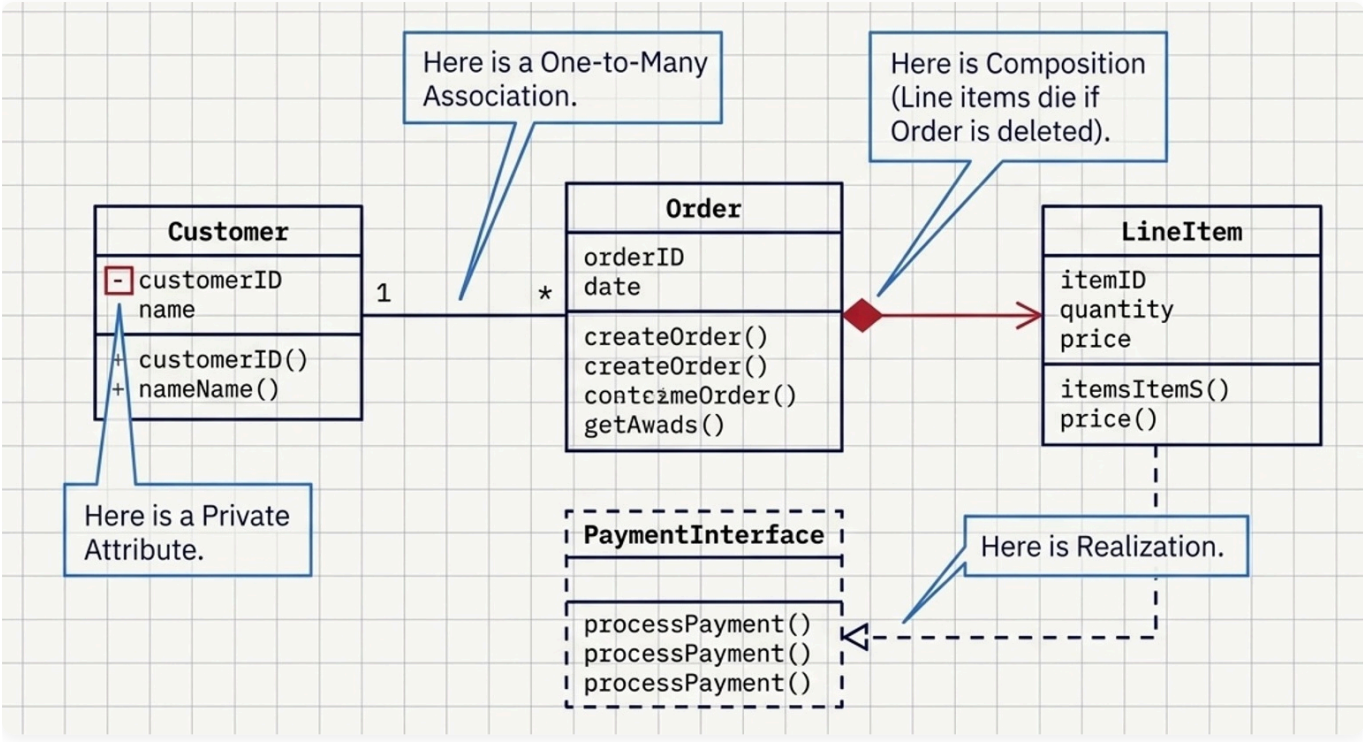


圖 4.4.8：外送平台領域模型架構實例。展示 Customer, Order, OrderItem, MenuItem, Restaurant, Courier 與 Payment 實體間嚴謹的組合、聚合與重數約束。

在本領域模型架構中：

1. `Customer` 與 `Order` 為 `1 ── *` 關聯（顧客可累積多筆歷史訂單）。
2. `Order` 與 `OrderItem` 為 `1 ◆── 1..*` 組合（訂單至少包含一項明細，明細脫離訂單即無實體意義）。
3. `OrderItem` 與 `MenuItem` 為 `* ── 1` 關聯（多項訂單明細指向相同菜單項目）。
4. `Order` 與 `Courier` 為 `* ──◇ 0..1` 聚合（外送員配送訂單，配送結束後獨立存活）。
5. `Order` 與 `PaymentProcessor` 保持依賴解耦（訂單委託外部金流處理扣款）。

4.4.6 生成式 AI 提示詞指引與外送平台類別模型

運用大型語言模型 (LLM) 自動合成類別圖時，必須提供嚴謹的架構約束與多重性指示：

```
[角色宣告：資深軟體架構師]
任務：請為《外送平台》核心業務領域生成標準 PlantUML 類別圖。

架構設計規範：
- 類別三層隔間：完整標註類別名稱、私有屬性變數 (-) 與公開方法 (+)。
- 生命週期約束：獨佔生命週期之整體-部分關係嚴格套用組合 (*--)。
- 獨立生存約束：弱擁有或共享引用之實體嚴格套用聚合 (o--)。
- 標註端點多重性 ('1', '0..1', '1..*')。
- 介面與依賴：金流扣款等外部整合服務以 interface 與依賴 (..>) 呈現。
```

外送平台類別模型圖 (Food Delivery Class Model)

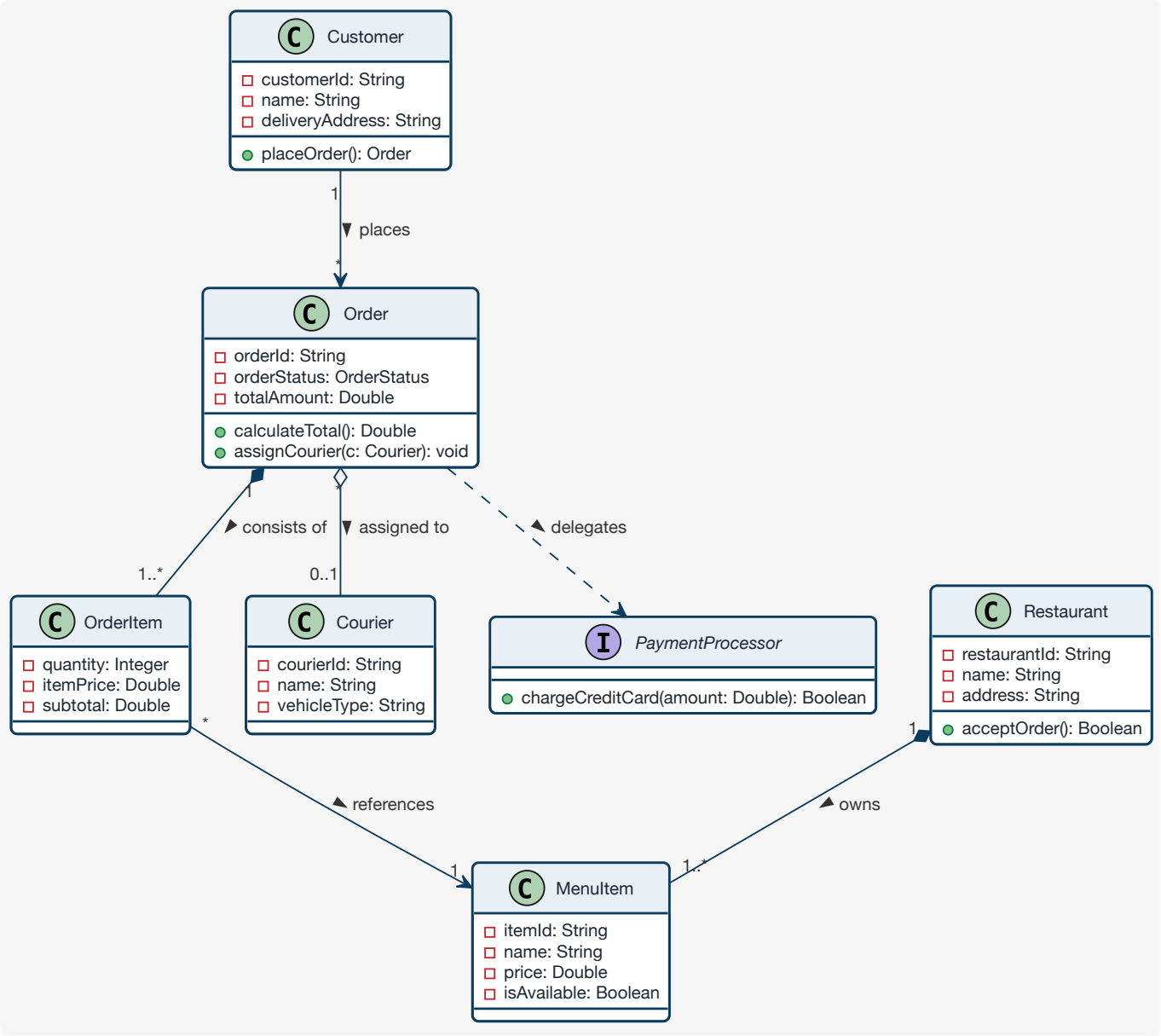


圖 4.4.9：外送平台 PlantUML 生成之標準類別圖。

```
@startuml
class Customer {
  - customerId: String
  - deliveryAddress: String
  + placeOrder(): Order
}
class Restaurant {
  - restaurantId: String
  - address: String
  + acceptOrder(): Boolean
}
class MenuItem {
  - itemId: String
  - price: Double
}
class Order {
  - orderId: String
  - totalAmount: Double
  + calculateTotal(): Double
  + assignCourier(c: Courier): void
}
class OrderItem {
  - quantity: Integer
  - subtotal: Double
}
class Courier {
  - courierId: String
  - vehicleType: String
}
interface PaymentProcessor {
  + chargeCreditCard(amount: Double): Boolean
}

Restaurant "1" *-- "1..*" MenuItem : owns >
Order "1" *-- "1..*" OrderItem : consists of >
OrderItem "*" --> "1" MenuItem : references >
Customer "1" --> "*" Order : places >
Order "*" o-- "0..1" Courier : assigned to >
Order ..> PaymentProcessor : delegates >
@enduml
```

4.4.7 診斷式案例實戰：抓出類別模型的架構破綻（棒球對決案例）

在物件導向領域模型設計中，關聯語意（繼承、聚合、組合）與多重性設定若稍有不慎，便會引發嚴重的程式碼架構災難。

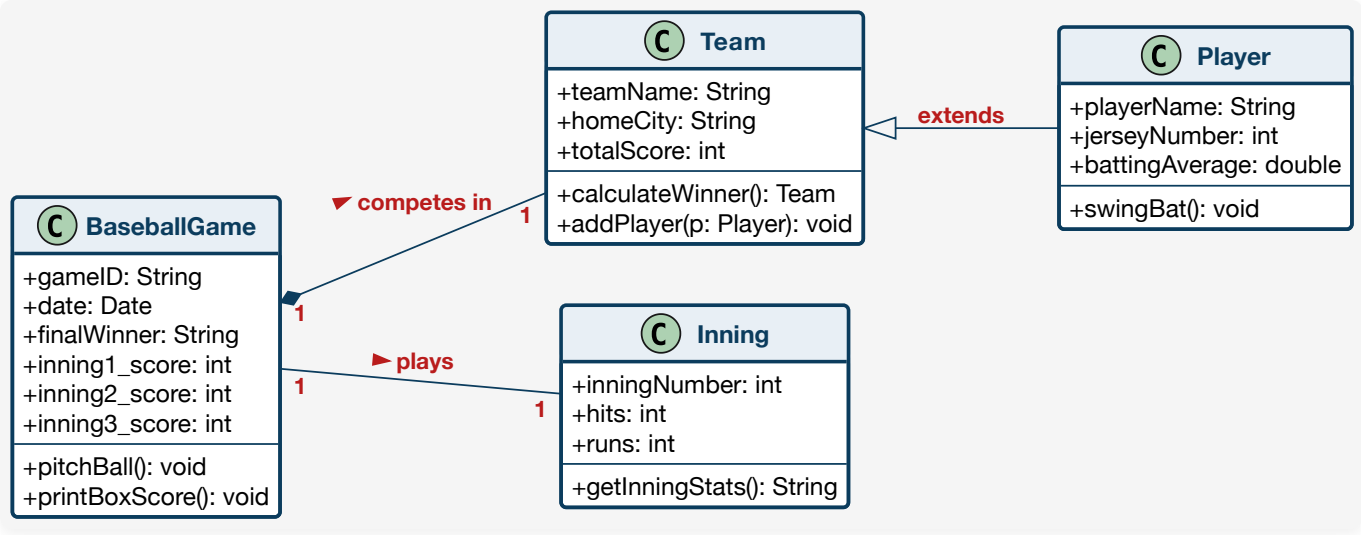


圖 4.4.10：棒球對決模型草稿。充斥著生命週期毀滅性組合、多重性倒置與封裝破壞四大嚴重破綻。

架構診斷審查：四大致命反模式

1. **破壞性組合反模式 (Destructive Composition Anti-Pattern)：**
   - *破綻：* `Team` 與 `Player` 之間被宣告為組合關聯 (`Team *-- Player`)。
   - *修正：* 組合 (◆) 代表強烈的部分與整體生命週期相依。若球隊宣告破產解散，難道旗下的所有球員必須被強制「安樂死」銷毀嗎？當然不！球員在球隊解散後可自由轉隊。球隊與球員之間是共享成員的弱擁有關係，必須使用聚合關聯 (`Team o-- Player`)。
2. **多重性倒置與幽靈成員 (Multiplicity Inversion & Ghost Members)：**
   - *破綻：* `BaseballGame` 與 `Team` 的多重性被標註為 `1` 對 `*`（一場比賽可容納無限支球隊），且球隊至球員的多重性標註為 `0..1`。
   - *修正：* 一場標準棒球對決嚴格由兩支球隊組成（主隊與客隊，多重性為 `2`）。一支球隊由多名球員組成 (`1..*` 或 `9..25`)，絕非 `0..1`。
3. **封裝破壞與職責錯置 (Information Hiding Violation & Misplaced Responsibility)：**
   - *破綻：* 類別所有屬性皆被宣告為公開 (`+`)；`pitchBall()`（投球）方法被錯誤掛在 `BaseballGame` 賽事類別上，而 `calculateWinner()` 則被掛在 `Team` 上。
   - *修正：* 實踐資訊隱藏原則，欄位應宣告為私有 (`-`)。方法必須遵循「資訊專家模式 (Information Expert)」：投球是 `Player` 的具體動作行為；而計算獲勝隊伍需比對兩隊總分，屬於掌控全局賽事的 `BaseballGame` 職責。
4. **繼承誤用 (Inheritance Misuse)：**
   - *破綻：* 將非 "is-a" 關係的實體硬套用泛化箭頭。
   - *修正：* 謹守「組合優於繼承 (Favor Composition over Inheritance)」原則。

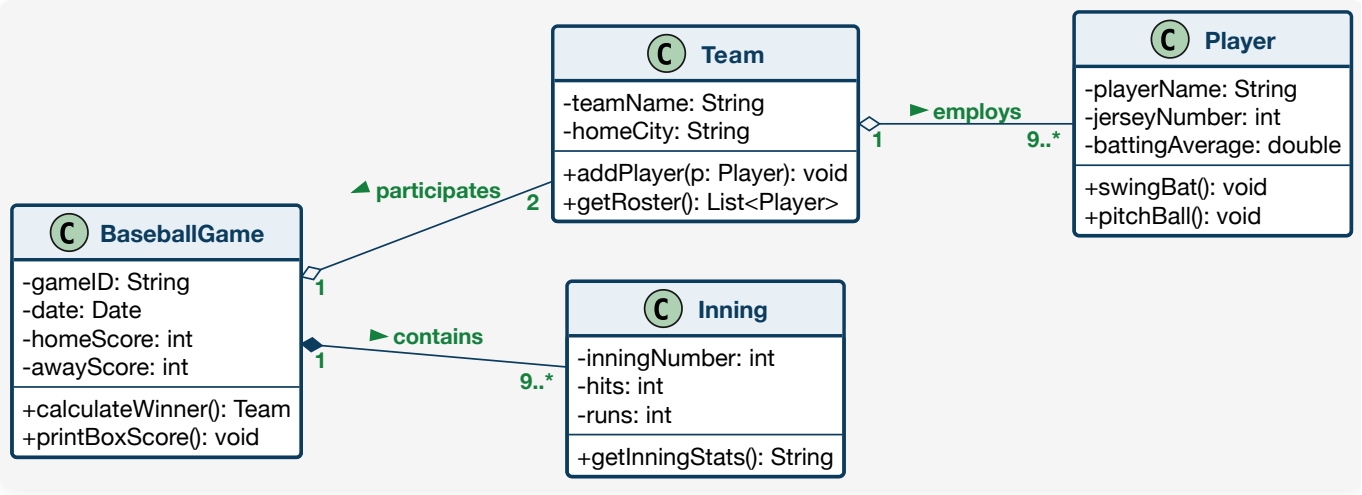


圖 4.4.11：修正後之棒球類別模型。正確套用非破壞性聚合、嚴格私有封裝與精確的業務多重性。

4.4.8 互動實作：運用 AI 演進外送平台領域類別模型

- **實作目標：** 在既有的外送平台領域類別提示詞中，引入一項業務結構變更，並審查 AI 產出的架構完整性：
  i. *選項 A (跨餐廳多店家購物車)：* 允許單筆 `Order` 包含來自不同 `Restaurant` 的餐點（需拆解為 `SubOrder` 子訂單）。
  ii. *選項 B (外送員評分與小費)：* 允許 `Customer` 在訂單完成後對 `Courier` 進行評分與自由給予小費（弱聚合關聯 `0..1`）。
  iii. *選項 C (VIP 尊榮訂閱方案)：* 引入會員訂閱制度，免收外送服務費（採用策略模式關聯而非多重繼承）。
- **架構檢核清單：**
  - 生命週期語意：新增關聯是否精準區分組合 (`*--`) 與聚合 (`o--`)？
  - 多重性邏輯：關聯端點的數量限制 (`1..*`, `0..1`) 是否符合真實業務約束？
  - 封裝與職責：欄位是否維持私有 (`-`)，方法是否分配給持有資料的專家類別？

🤦 觀念檢核測驗 (CCQ 4) — 類別關係：組合 vs. 聚合

問題

在 UML 類別圖中，**組合 (Composition, ◆)** 與 **聚合 (Aggregation, ◇)** 最關鍵的語義差異為何？

- A. 組合代表子物件可以脫離父物件獨立存活，聚合則生命週期嚴格綁定。
- B. 組合代表強烈的整體與部分關係，部分物件的生命週期與整體完全綁定（整體消滅則部分必亡）；聚合則代表弱擁有關係，部分物件具備獨立生命週期。
- C. 組合用於介面繼承，聚合用於多型實作。
- D. 組合關聯度數只能是 1:1，聚合關聯度數只能是 1:N。

Interactive Activity (線上作答)

▶ 點擊展開參考答案與解析

4.5 互動與循序圖模型 (Sequence Diagrams)

類別圖展示了靜態架構，而**循序圖 (Sequence Diagram)** 則將焦點轉移至執行期：在特定的業務情境下，執行個體 (Instances) 究竟如何隨著時間傳遞訊息完成交易？

4.5.1 二維畫布：參與者 vs. 垂直向下時間軸

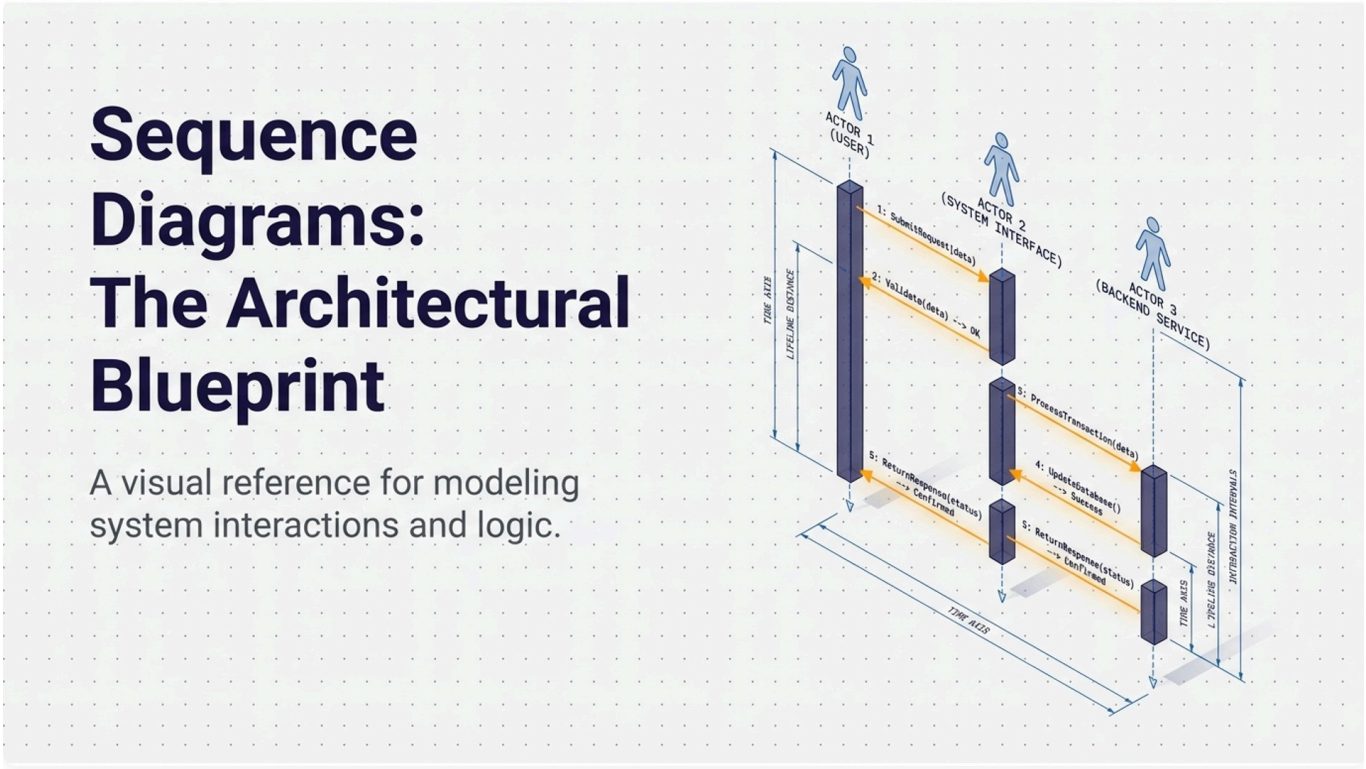


圖 4.5.1：動態互動模型綜觀。靜態類別提供方法宣告，循序圖則具體呈現執行個體在時間軸上的訊息流轉。

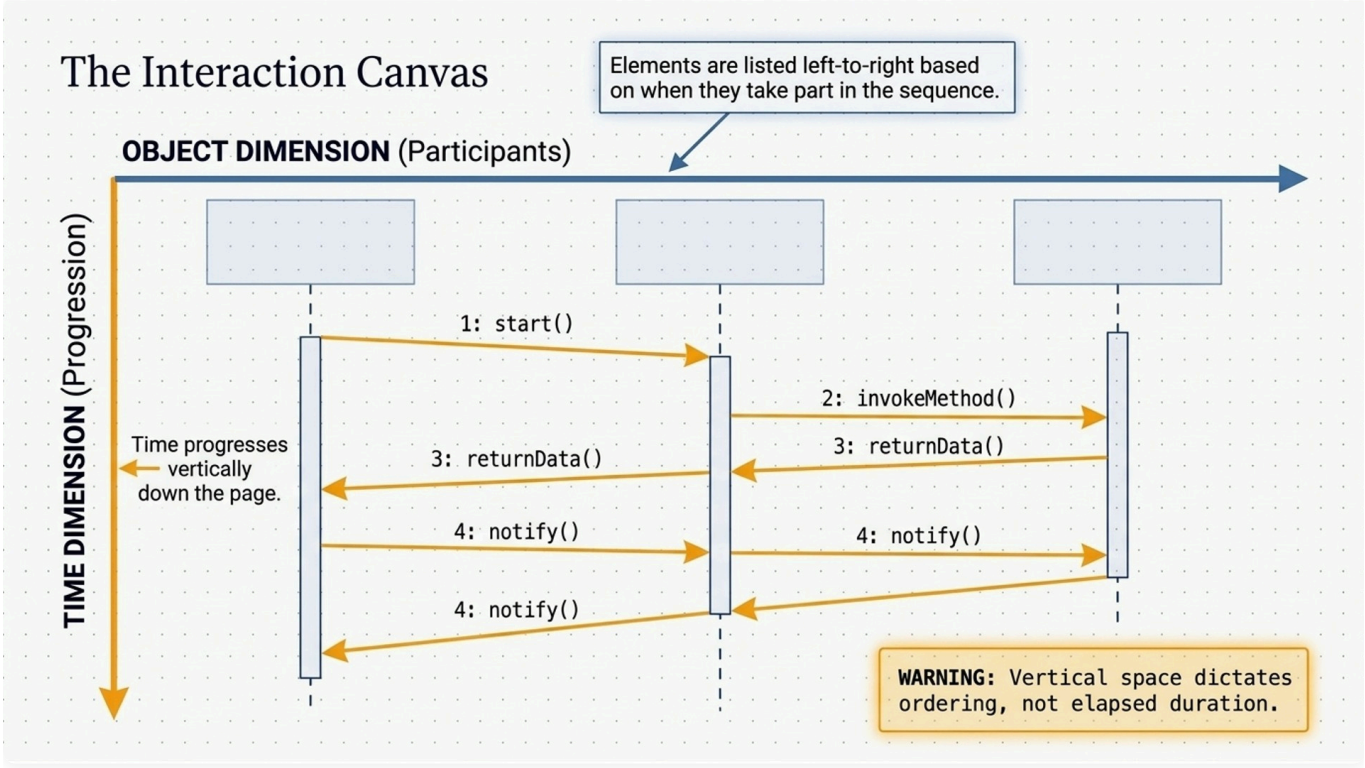


圖 4.5.2：循序圖二維畫布。水平維度排列協同運作之物件執行個體，垂直維度代表不可逆的時間流逝。

- **水平軸 (參與者與生命線)**：上方矩形方塊標註 `物件名稱 : 類別名稱`，下方延伸出一條垂直虛線，稱為**生命線 (Lifeline)**。
- **垂直軸 (時間維度)**：時間嚴格地**由上至下 (Top-to-Bottom)** 單向推進，越下方代表發生在越晚的時間點。

4.5.2 生命線、啟動條與五大訊息通訊語法

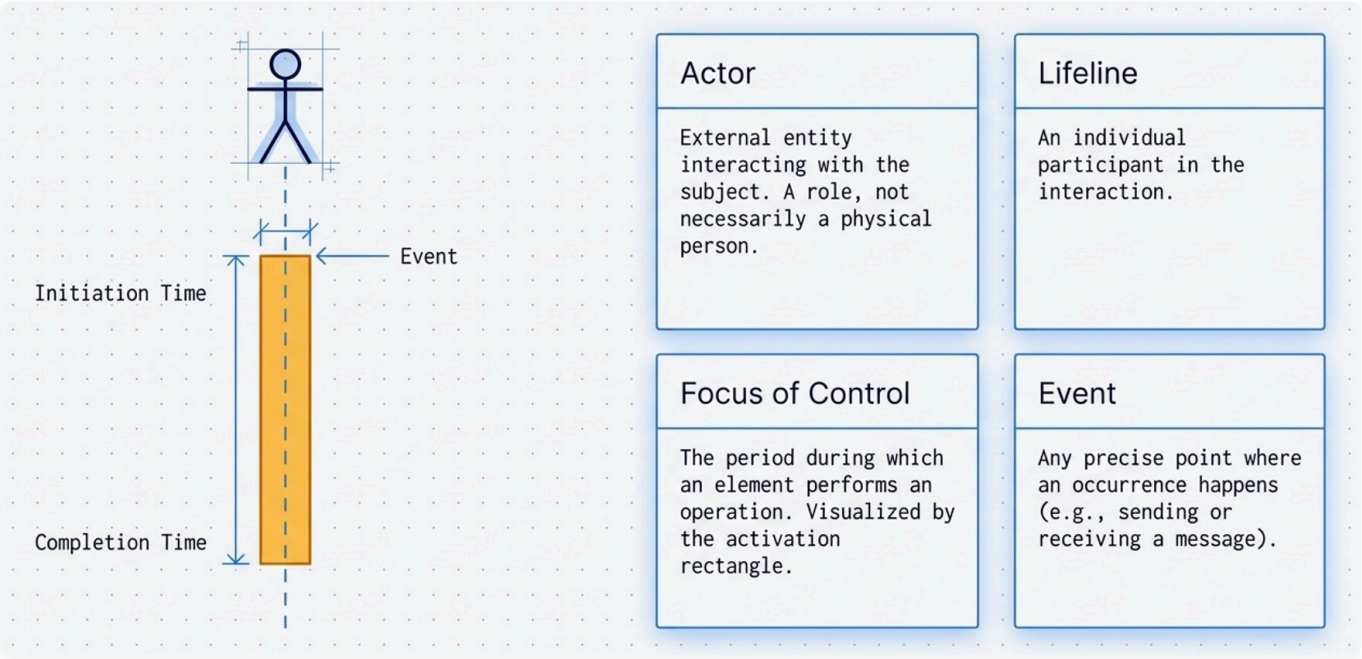


圖 4.5.3：生命線與啟動焦點條 (Activation Bar)。生命線上的細長矩形代表該物件正在執行處理邏輯或等待副程式回傳的有效作用期。

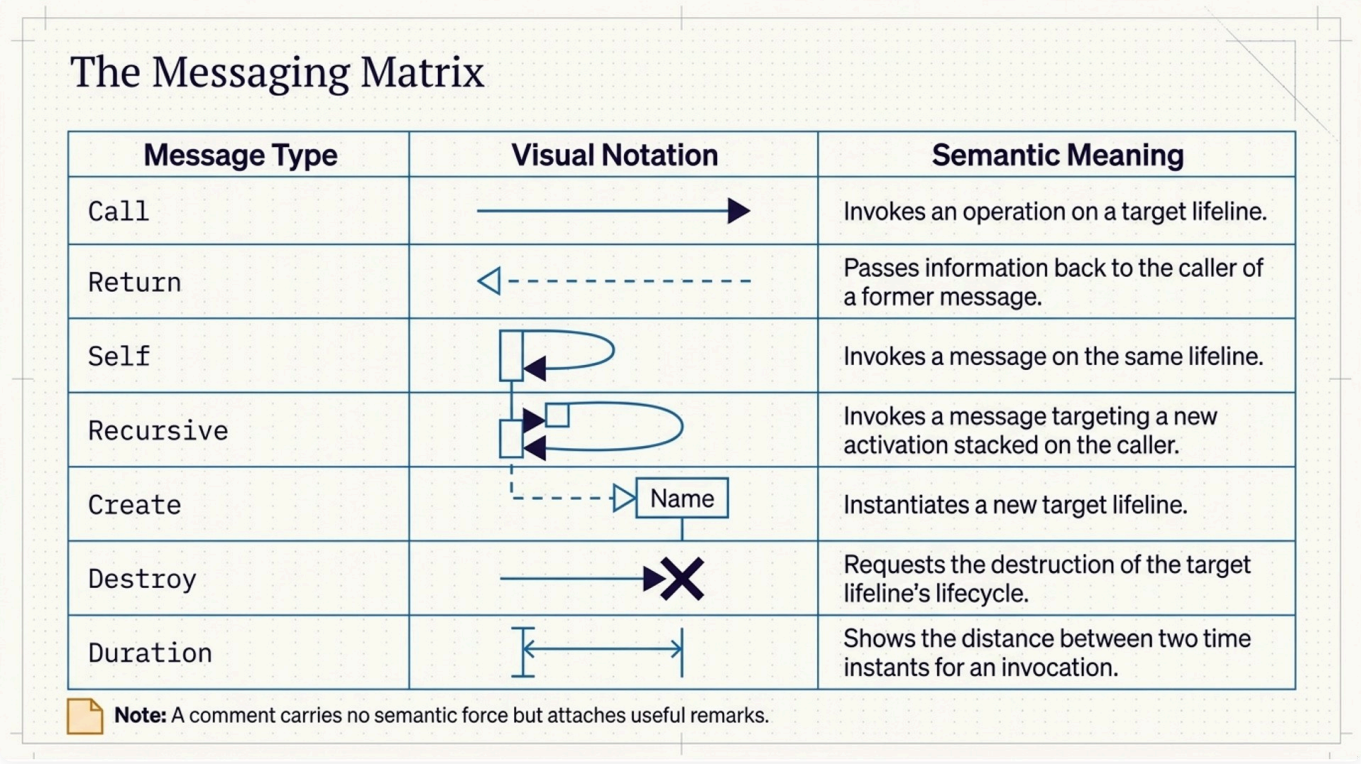


圖 4.5.4：UML 循序圖訊息通訊語法矩陣。同步呼叫、非同步訊號、回傳訊息、物件建立與消滅之圖形規範。

訊息調用類型	視覺箭頭語法	執行時期語意與執行緒行為
同步呼叫 (Sync Call)	實線實心箭頭 `——►`	**阻斷式 (Blocking)。** 呼叫端暫停並等待被呼叫端處理完成回傳。
非同步訊號 (Async Signal)	實線開放箭頭 `——>`	**非阻斷式 (Non-blocking)。** 呼叫端送出事件即繼續執行。
回傳訊息 (Return Message)	虛線開放箭頭 `<-- --`	選用顯式回傳。將控制權與運算結果資料回傳給呼叫端。
建立物件 (Create)	虛線指向實體框 `- - ►`	執行時期動態實體化新物件（`new Order()`）。
銷毀物件 (Destroy)	生命線末端大 `X` 符號	顯式釋放實體記憶體或停止垃圾回收生命週期。

4.5.3 複合片段 (Combined Fragments)：分支、迴圈與並行

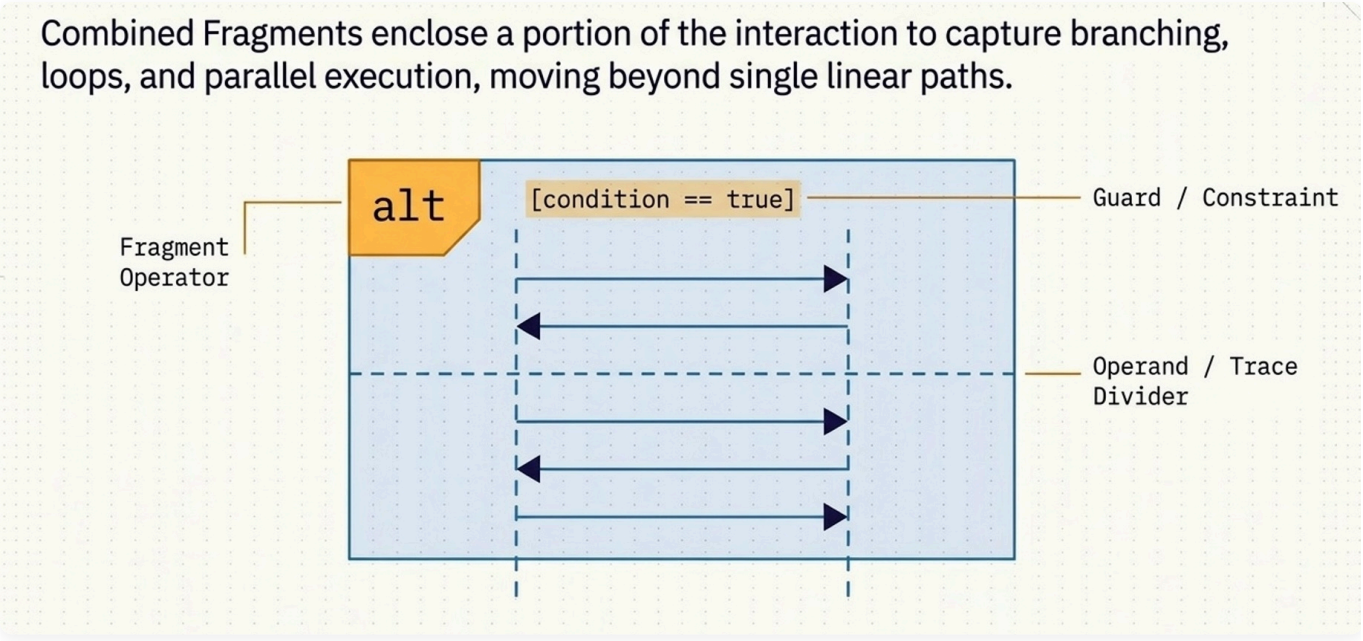


圖 4.5.5：複合片段綜觀。以標準 UML 算子方框封裝分支、迴圈與並行處理。

Operator	Definition	Code Equivalent
alt	**Alternative:** Multiple fragments; only the one with a true condition executes.	`if / else if / else`
opt	**Optional:** Executes only if the supplied condition is true. Single trace.	`if (without else)`
loop	**Loop:** Executes multiple times based on a guard condition.	`while / for loop`
par	**Parallel:** Each fragment runs concurrently.	`multi-threading / async`
region	**Critical Region:** Only one thread can execute this fragment at once.	`Mutex lock / sync`
neg	**Negative:** Shows an explicitly invalid interaction.	`Error path / Exception`
ref	**Reference:** Refers to an interaction defined on another diagram.	`Function / Module call`
sd	**Sequence Diagram:** Surrounds an entire root sequence diagram.	`Root program`

圖 4.5.6：複合片段算子與程式代碼直接對應。alt 對應 if-else，opt 對應單一 if，loop 對應 while/for，par 對應並行多執行緒。

運算子	程式語言控制結構對照	軟體架構與控制流語意
`alt`	`if (...) { ... } else { ... }`	**互斥替代分支。** 依據守衛條件 (Guard) 僅且僅會執行其中一個子框。
`opt`	`if (...) { ... }` *(無 else)*	**選用執行片段。** 當守衛條件為真時執行；為假時整段跳過。
`loop`	`for / while` `loop(min, max)`	**迴圈重複執行。** 在守衛條件維持真或指定次數範圍內反覆執行。
`par`	`Thread / Promise.all()`	**並行執行。** 框內的多個次子片段在不同執行緒上同時並行運作。

4.5.4 BCE (Boundary-Control-Entity) 架構模式實務

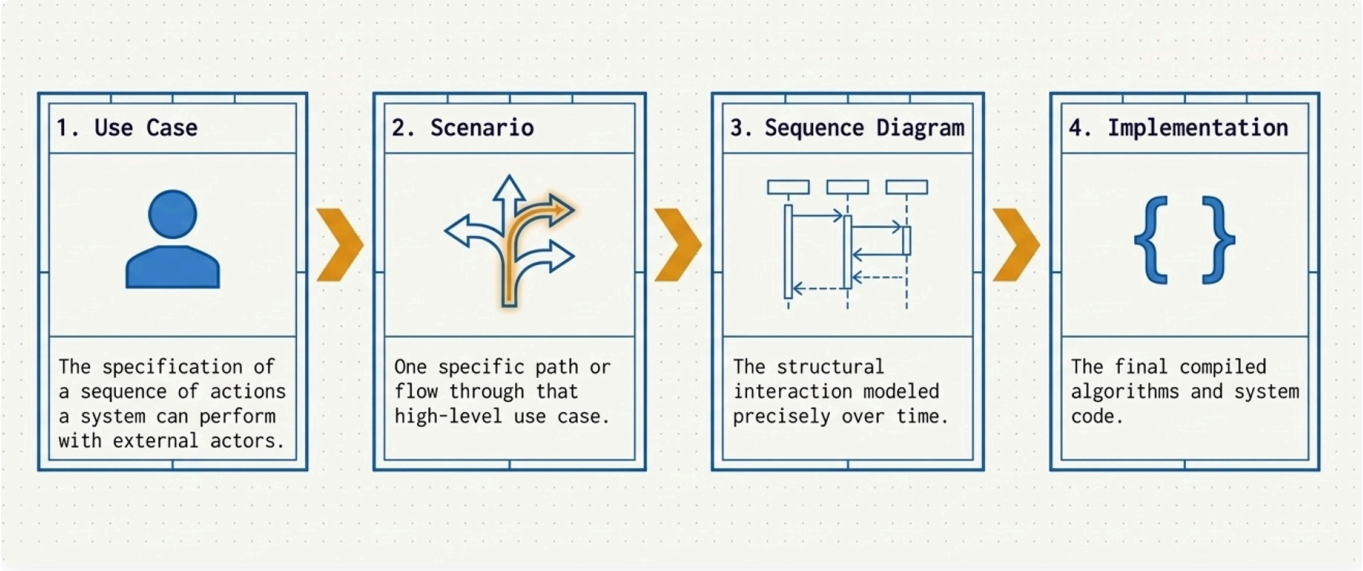


圖 4.5.7：BCE 架構模式與循序管線。前端 Boundary 將請求轉送給 Control 控制器進行邏輯協調，再由控制器操作 Entity 實體與外部服務。

在設計高內聚低耦合的循序圖時，必須遵循 BCE 架構分工：

- **邊界物件 (Boundary, `<<boundary>>`)**：負責與外部溝通（如 `CheckoutUI`、`RESTController`）。嚴禁邊界直接存取資料庫或執行交易邏輯。
- **控制物件 (Control, `<<control>>`)**：負責業務邏輯編排與交易界限（如 `OrderCheckoutService`）。控制器協調實體狀態，並調用外部邊界。
- **實體物件 (Entity, `<<entity>>`)**：代表持久化商業資料與核心規則（如 `Order`、`Customer`）。實體嚴禁感知 UI 邊界的存在。

4.5.5 外送平台結帳 BCE 循序圖實戰案例

外送平台結帳循序模型圖 (Food Delivery Sequence Model)

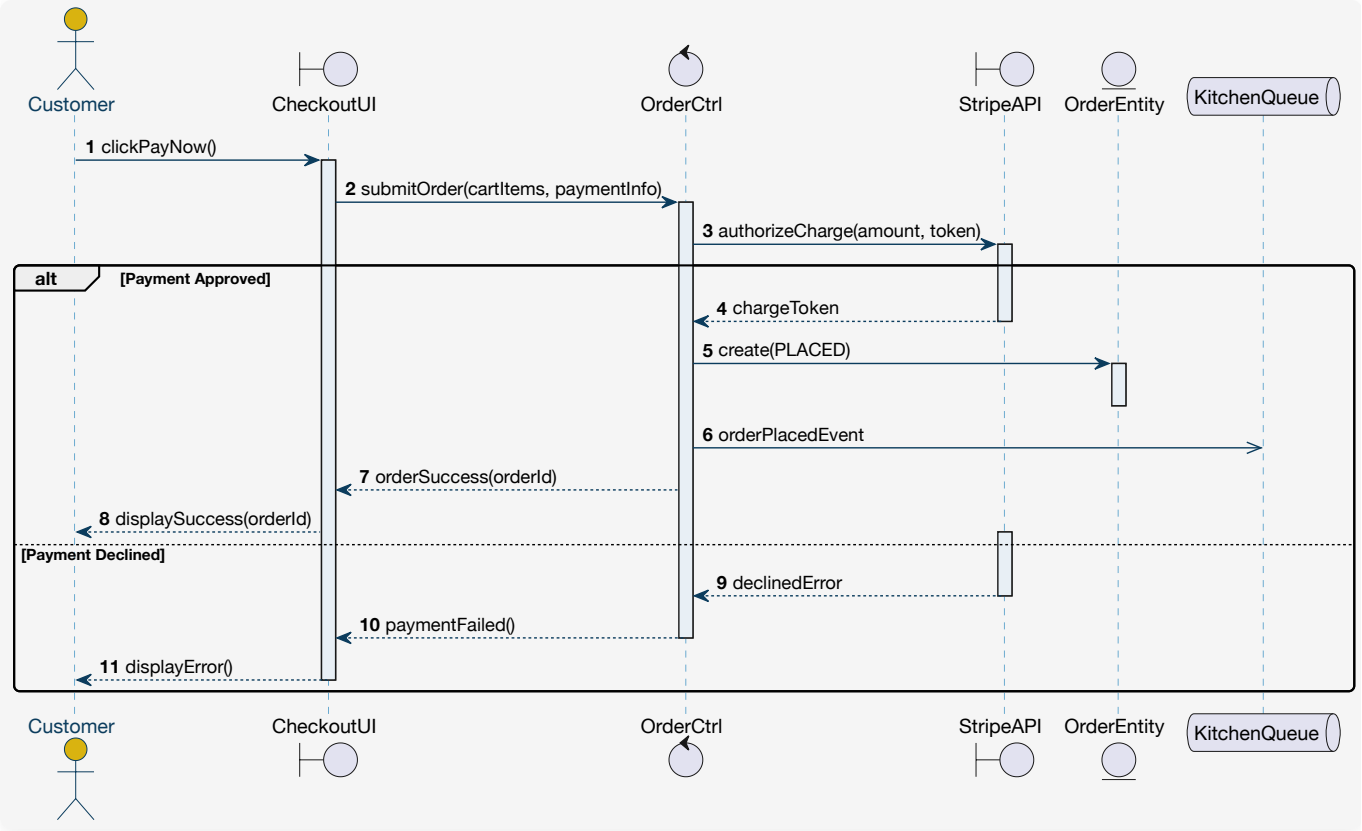


圖 4.5.8：外送平台 PlantUML 生成之標準 BCE 結帳循序圖。展示 Customer, CheckoutUI, OrderCtrl, StripeAPI, OrderEntity 與 KitchenQueue 跨層互動與 alt 成功/失敗分支。

```
@startuml
autonumber
actor Customer
boundary CheckoutUI
control OrderCtrl
boundary StripeAPI
entity OrderEntity
queue KitchenQueue

Customer -> CheckoutUI: clickPayNow()
activate CheckoutUI
CheckoutUI -> OrderCtrl: submitOrder(cart, payment)
activate OrderCtrl
OrderCtrl -> StripeAPI: authorizeCharge(amount, token)
activate StripeAPI

alt Payment Approved
  StripeAPI --> OrderCtrl: chargeToken
  deactivate StripeAPI
  OrderCtrl -> OrderEntity: create(PLACED)
  OrderCtrl ->> KitchenQueue: orderPlacedEvent
  OrderCtrl --> CheckoutUI: orderSuccess(orderId)
  CheckoutUI --> Customer: displaySuccess()
else Payment Declined
  StripeAPI --> OrderCtrl: declinedError
  OrderCtrl --> CheckoutUI: paymentFailed()
  deactivate OrderCtrl
  CheckoutUI --> Customer: displayError()
  deactivate CheckoutUI
end
@enduml
```

4.5.6 診斷式案例實戰：抓出循序圖模型的架構破綻（飯店訂房案例）

循序圖捕捉動態呼叫鏈，若忽略分層架構或控制片段語法，極易產生時序因果倒置或資料不一致之嚴重缺陷。

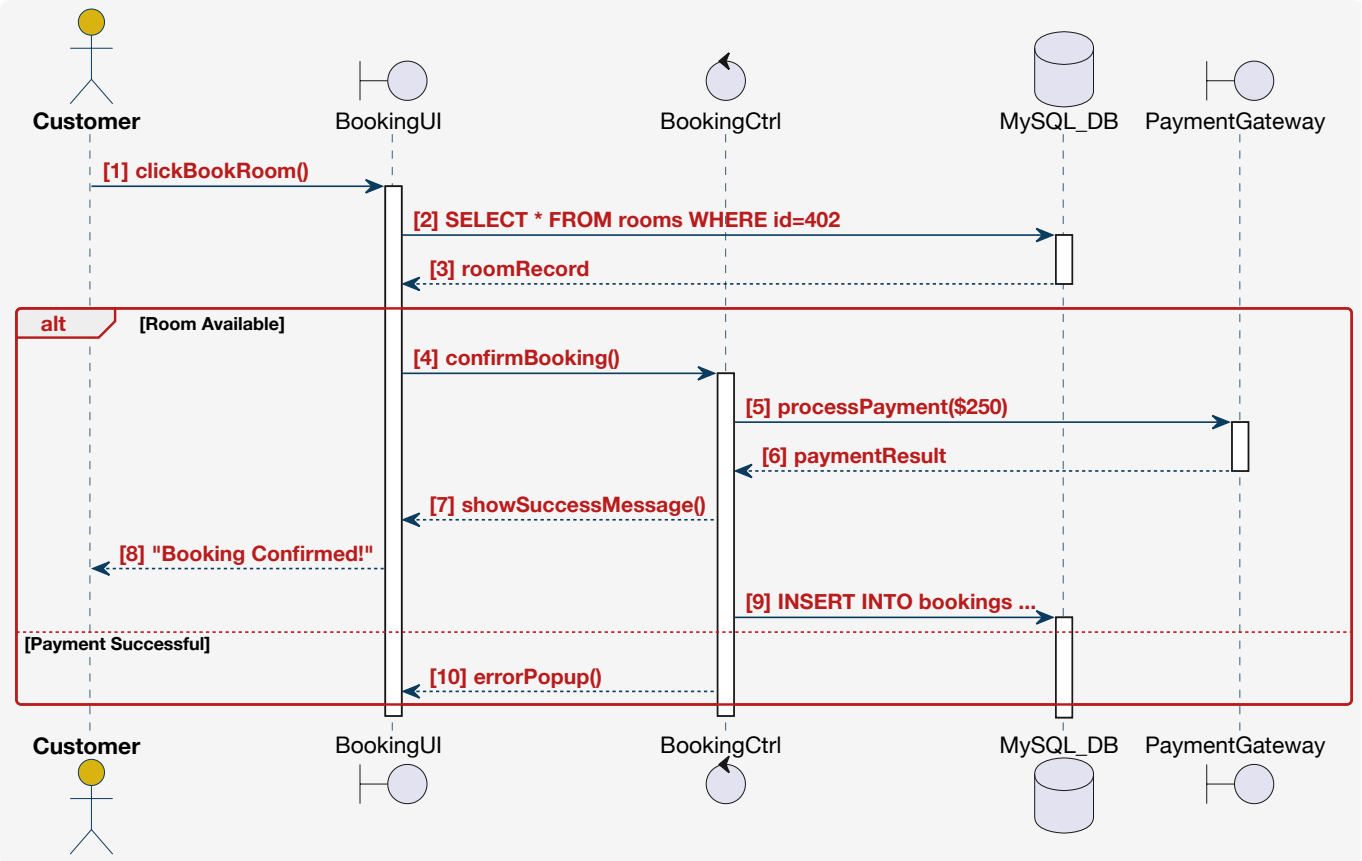


圖 4.5.9：飯店訂房循序圖草稿。包含分層架構破壞、守衛語意混淆、時序因果倒置與生命線懸掛四大嚴重破綻。

架構診斷審查：四大致命反模式

1. **分層架構與封裝破壞 (Layering & Encapsulation Violation)：**
   - *破綻：* 訊息 `[2]` 顯示前端畫面 `BookingUI` 竟直接以 SQL 語法 (`SELECT * FROM rooms...`) 直連查詢資料庫 `MySQL_DB`。
   - *修正：* 在多層架構與 BCE 原則下，表現層邊界嚴禁繞過商業邏輯控制層直通持久化資料庫。如此將導致前端與資料表綱要緊密耦合，破壞資訊隱藏。必須透過 `BookingController` 進行調度。
2. **無效之 `alt` 複合片段條件混淆 (Invalid Combined Fragment Guards)：**
   - *破綻：* 在 `alt` 條件區塊中，第一條分支守衛條件為 `[Room Available]`（房間尚有空位），但 `else` 分支守衛卻寫成 `[else Payment Successful]`（付款成功）。
   - *修正：* 同一 `alt` 區塊內的各分支必須評估相同的業務向度且具備互斥性（例如：`[Room Available]` vs. `[else / Room Unavailable]` ）。將房況庫存與付款成功混雜在同一個決策分支屬於根本性的邏輯混亂。
3. **時序因果倒置與提前確認 (Chronological Causality & Premature Confirmation)：**
   - *破綻：* 訊息 `[7]` 與 `[8]` 顯示系統在訊息 `[9]` 的資料庫寫入 (`INSERT INTO bookings`) 完成「之前」，就提前向顧客畫面顯示 `Booking Confirmed!`。
   - *修正：* 若後續資料庫寫入因主鍵衝突或連線中斷失敗，顧客手中卻握有確認成功的假訊息，將造成災難性的業務糾紛。交易與資料庫寫入必須在向使用者確認成功之前完成提交 (Commit)。
4. **生命線啟動條懸掛未關閉 (Dangling Lifeline Activation)：**
   - *破綻：* `MySQL_DB` 在訊息 `[9]` 啟動後，啟動條一路無限向下延伸，從未關閉。

- *修正：* 啟動條代表物件獲取 CPU 執行緒或等待子程序返回之聚焦期間。資料庫查詢或寫入完畢後，必須立即終止啟動條焦點。

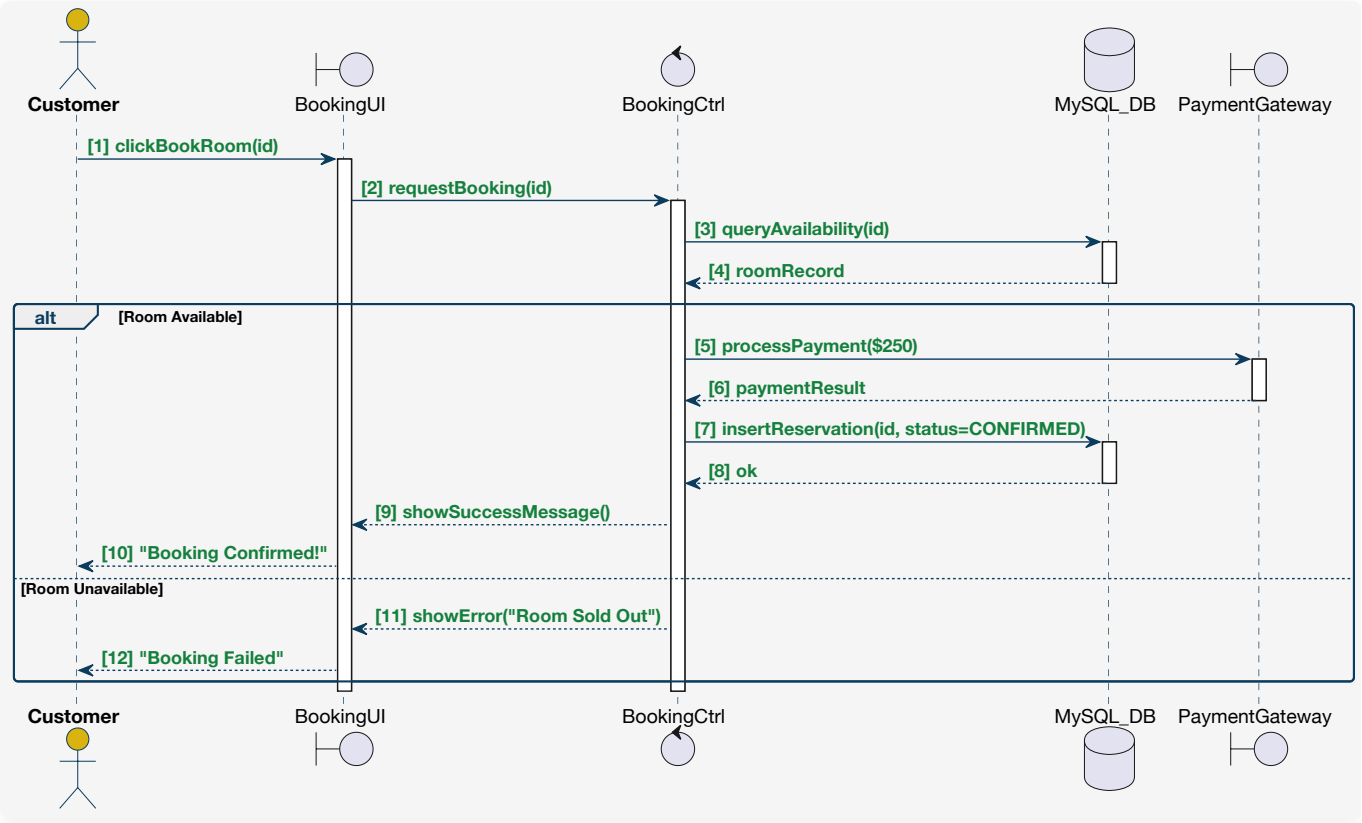


圖 4.5.10：修正後之標準循序圖。貫徹 BCE 分層、互斥條件守衛、先寫入後確認時序與清晰閉合的啟動焦點。

4.5.7 互動實作：運用 AI 演進循序圖模型 (Pair Hands-On Activity)

- **實作目標：** 在外送平台結帳循序圖提示詞中引入一項分散式異常邊緣案例，並審查 AI 產出的時序互動邏輯：
  i. *選項 A (第三方金流閘道逾時)：* 刷卡呼叫超過 5 秒未回應，觸發控制器重試或取消交易補償機制。
  ii. *選項 B (數位錢包與信用卡雙軌混合支付)：* 先扣錢包餘額，餘額不足再呼叫外部信用卡；若信用卡失敗則執行錢包退款補償。
  iii. *選項 C (結帳瞬間庫存售罄)：* 在金流扣款前鎖定庫存失敗，優雅中斷交易並回報使用者。
- **架構檢核清單：**
  - 分層架構：UI 是否維持透過 Controller 調用，未直接存取資料庫或外部 API？
  - 分支互斥性：成功路徑與異常失敗路徑是否使用 `alt` 片段清晰劃分？
  - 背景解耦：非同步背景通知（如廚房印單、推播訊息）是否正確使用非同步箭頭 (`->>`)？

🤦 觀念檢核測驗 (CCQ 5) — BCE 架構職責分工

問題

在 BCE (Boundary-Control-Entity) 架構循序圖中，當顧客透過結帳前端點擊『確認下單』按鈕時，該事件應先傳遞給哪一種類型的物件處理？

- A. 直接傳遞給 `Order` 實體物件 (Entity) 立即寫入資料庫
- B. 直接呼叫 `PaymentGatewayAPI` 邊界物件 (Boundary) 立即刷卡扣款
- C. 傳遞給 `OrderController` 控制物件 (Control) 協調業務邏輯驗證與調用
- D. 直接傳遞給 `KitchenPrinter` 邊界物件列印工單

Interactive Activity (線上作答)

▶ 點擊展開參考答案與解析

4.6 流程與活動圖模型 (Activity Diagrams)

循序圖擅長追蹤物件之間的特定訊息時序，而**活動圖 (Activity Diagram)** 則將鏡頭拉遠，以工作流程與 Token 流程控制的巨觀角度，描繪跨角色、跨系統之演算法邏輯與並行業務流程。

4.6.1 活動圖 vs. 傳統流程圖之四大優勢

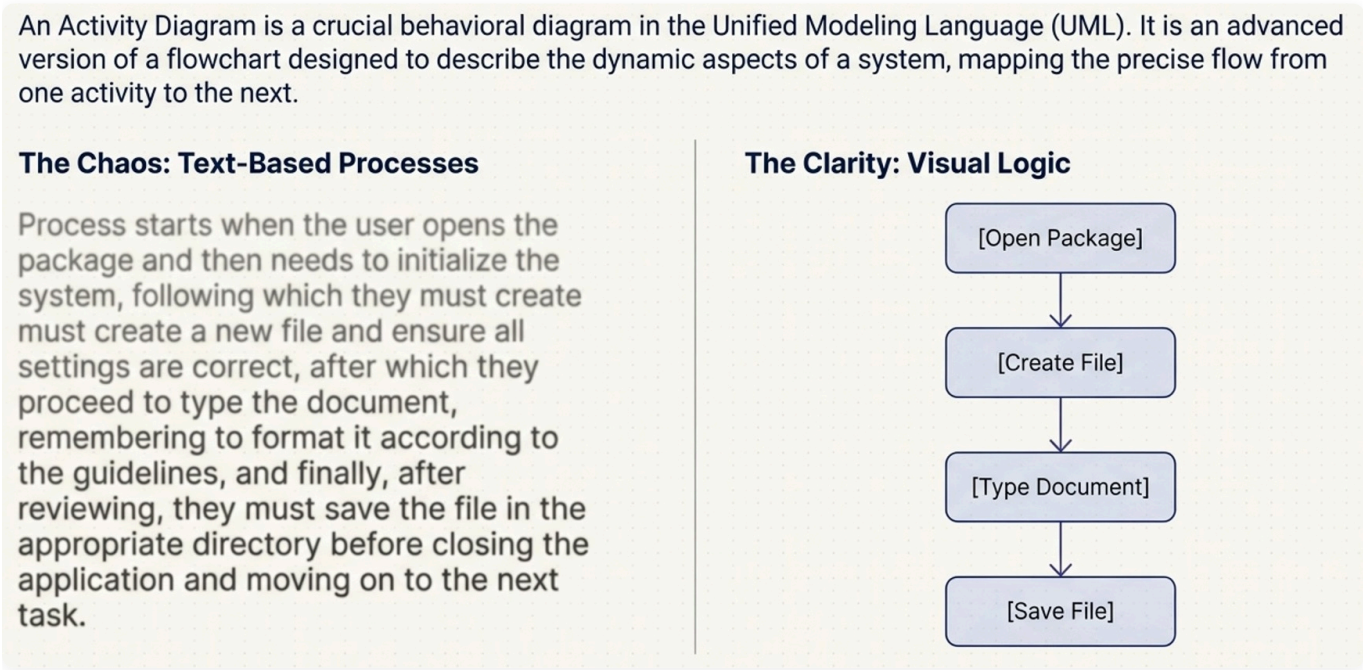


圖 4.6.1：活動圖之視覺邏輯。引入 Petri-Net Token 語意、物件流與泳道機制，徹底超越傳統流程圖。

評估維度	傳統流程圖 (Flowchart)	UML 活動圖 (Activity Diagram)
並行處理支援	無 (本質上屬於嚴格單執行緒流程)。	**原生支援同步條 (Fork/Join)**，可精確塑模多執行緒並行運算。
組織職責歸屬	無 (所有步驟平鋪於單一全局畫布)。	**泳道分區 (Swimlanes)** 明確指派各步驟之責任歸屬單位與系統邊界。
資料物件狀態流	僅有模糊的平行四邊形 I/O 標記。	**顯式物件節點 (Object Nodes)** 攜帶狀態釘針，如 `[已付款]`、`[已備餐]`。
執行數學語意	僅為非正式的視覺連接線。	**基於 Petri-Net 權標 (Token) 推進機制**，具備嚴謹數學模型基礎。

4.6.2 核心節點：動作、決策/合併與分岔/結合

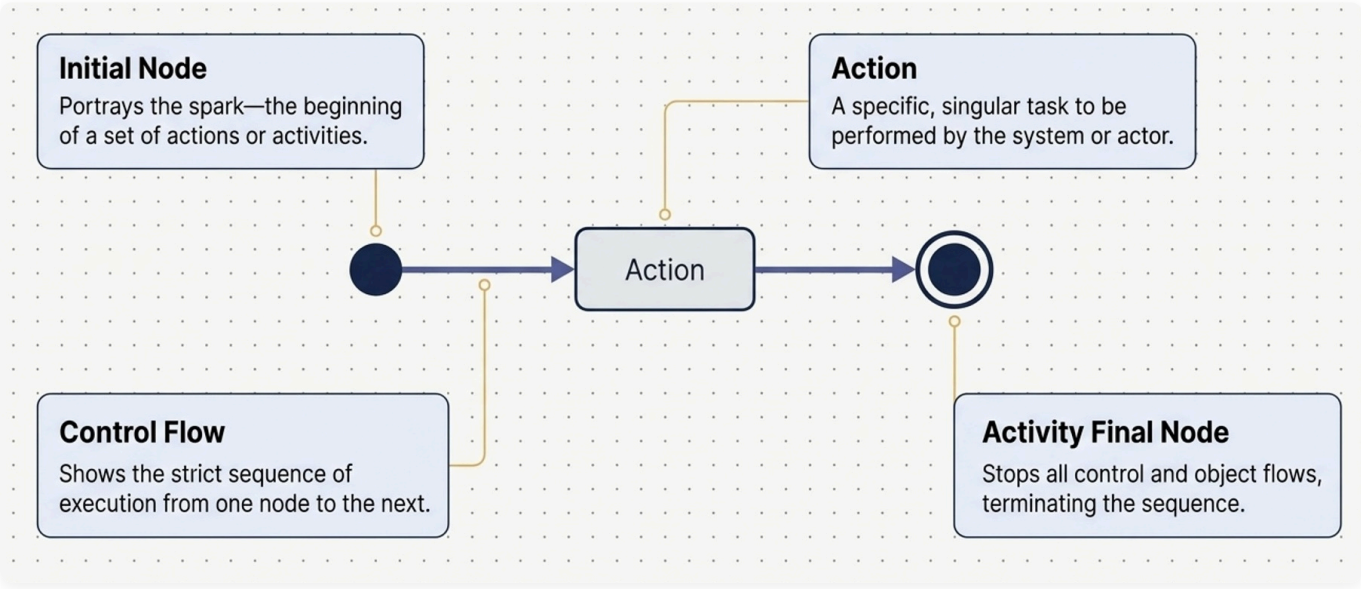


圖 4.6.2：活動圖核心語法節點。初始節點、動作狀態、決策/合併菱形與活動終止節點。

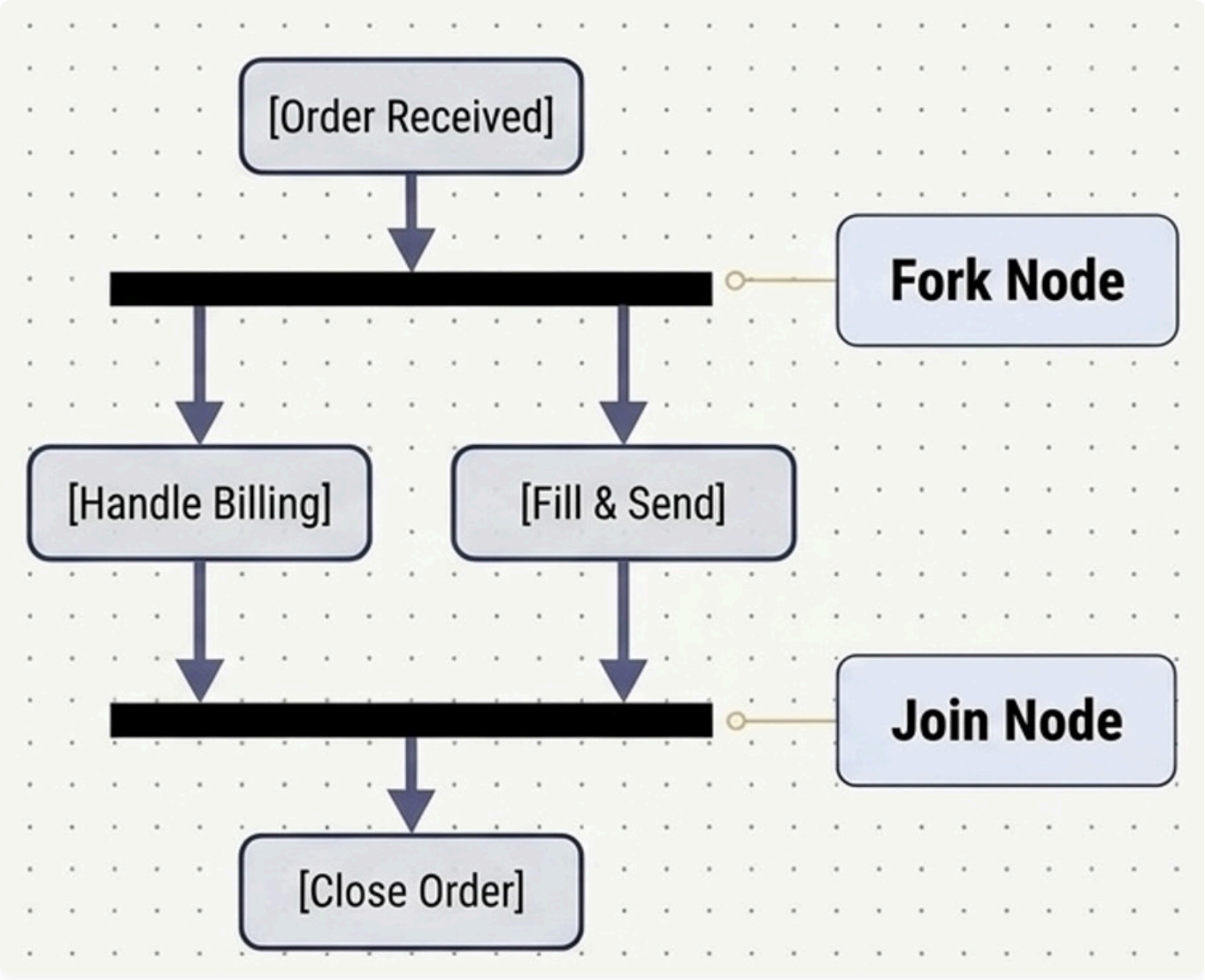


圖 4.6.3：Fork 與 Join 同步控制條。Fork 將單一 Token 複製切分為多個並行活動；Join 則等待所有並行路徑皆完成匯流後方能向下推進。

核心辨析：Fork/Join vs. Decision/Merge

- **決策與合併 (Decision / Merge ◇ 菱形)**：代表**條件互斥分支**。一個流入的 Token，依據布林守衛條件 `[guard]`，在多條路徑中**嚴格選擇其一**推進。
- **分岔與結合 (Fork / Join ▮ 同步條)**：代表**多執行緒並行**。一個流入的 Token 被複製分配至所有流出路徑，所有並行活動**同時並發執行**；Join 條則具備阻塞同步語意，必須等待每一條並行路徑的 Token 全部到達，才融合成單一 Token 向下推進。

4.6.3 泳道 (Swimlanes) 與責任分區

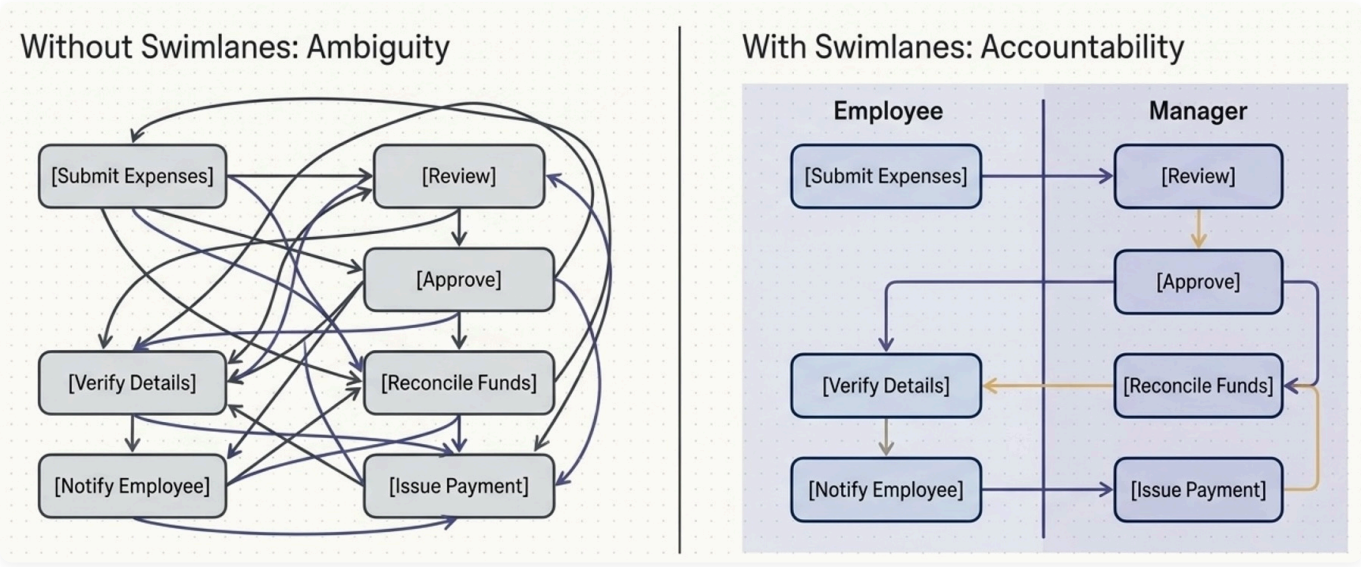


圖 4.6.4：泳道 (Swimlanes/Partitions) 劃分。將活動分配至不同部門、使用者或微服務系統。

泳道將活動圖垂直或水平分割為不同欄位，明確各角色的權責邊界：

- **顧客泳道 (Customer Lane)：** 負責確認下單、以 OTP 驗證簽收餐點。
- **餐廳廚房泳道 (Kitchen Lane)：** 負責接單烹飪、打包餐點並將餐點交付外送員。
- **配送物流泳道 (Courier & Dispatch Lane)：** 負責媒合最佳外送員、行駛至餐廳取餐並配送到府。

4.6.4 物件節點與狀態資料流 (Object Nodes & Stateful Data Flows)

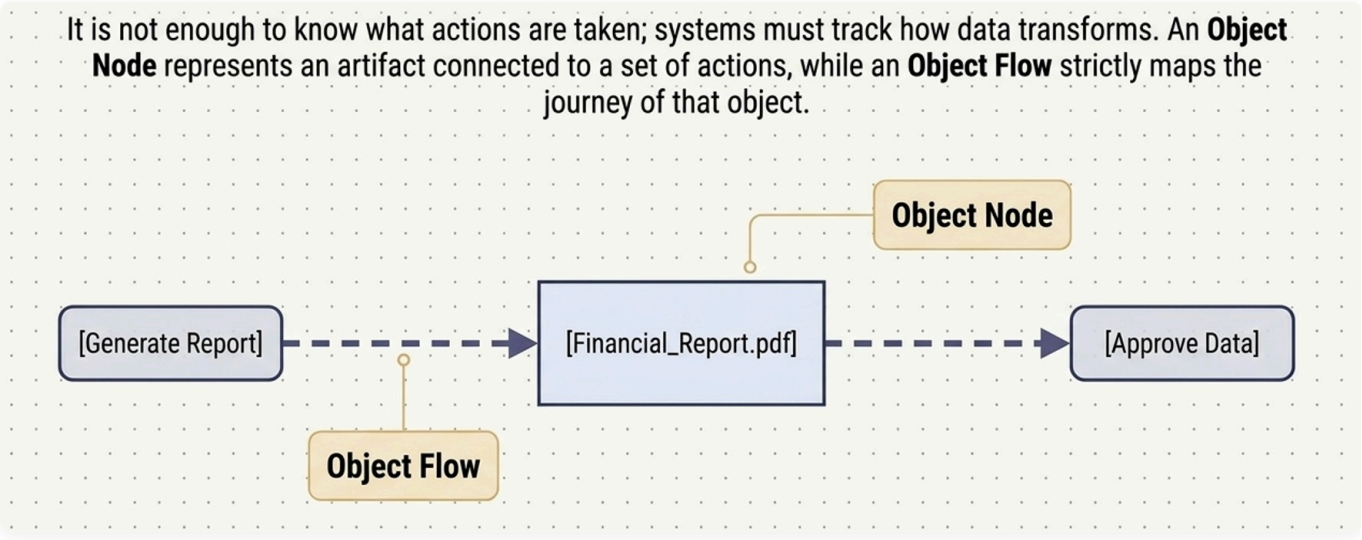


圖 4.6.5：物件流 (Object Flow) 與狀態釘針。展示實體物件在各動作執行間的生命週期狀態轉變。

除了純粹的控制流箭頭，活動圖亦可引入**物件節點 (Object Nodes)**（標註為 `類別 [狀態]` 的矩形框），展示資料實體如何在動作之間流轉與轉變狀態：`[訂單已成立]` ——▶ **烹飪餐點** ——▶ `[餐點已煮好]` ——▶ **打包餐點** ——▶ `[餐點待取餐]`。

4.6.5 外送平台多方協同活動圖實戰案例

外送平台活動模型圖 (Food Delivery Activity Model)

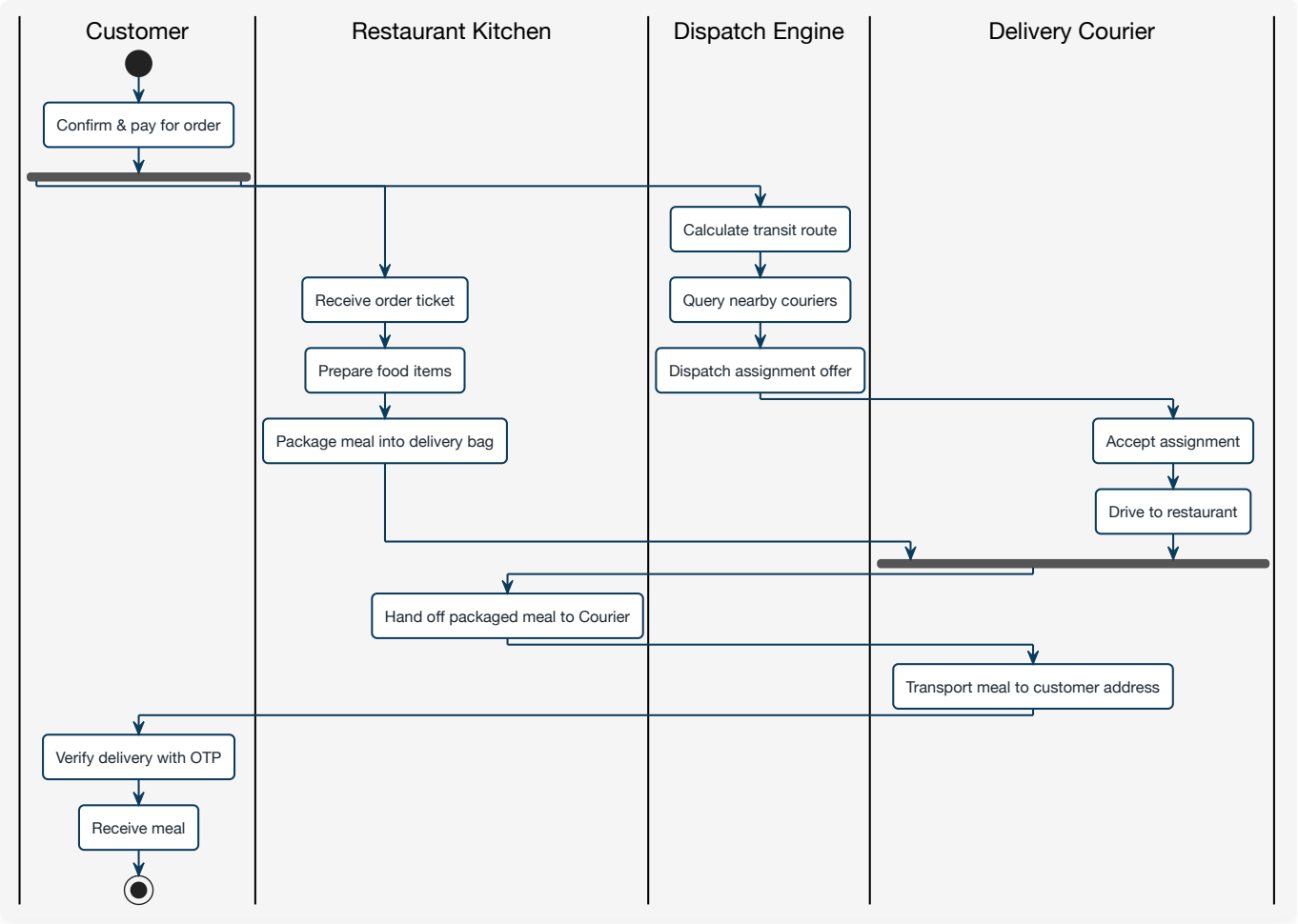


圖 4.6.6：外送平台 PlantUML 生成之泳道活動圖。展示顧客、餐廳廚房、調度引擎與外送員四大泳道之並行備餐、調度與 Fork/Join 結合點。

```
@startuml
|Customer|
start
:Confirm & pay for order;

fork
  |Restaurant Kitchen|
  :Receive order ticket;
  :Prepare food items;
  :Package meal into delivery bag;
fork again
  |Dispatch Engine|
  :Calculate transit route;
  :Query nearby couriers;
  :Dispatch assignment offer;
  |Delivery Courier|
  :Accept assignment;
  :Drive to restaurant;
end fork

|Restaurant Kitchen|
:Hand off packaged meal to Courier;

|Delivery Courier|
:Transport meal to customer address;

|Customer|
:Verify delivery with OTP;
:Receive meal;
stop
@enduml
```

4.6.6 診斷式案例實戰：抓出活動圖模型的流程破綻（外送訂單履約流程）

設計健全的業務工作流程，必須嚴格遵守 Token 守恆、同步平衡與泳道責任歸屬原則。

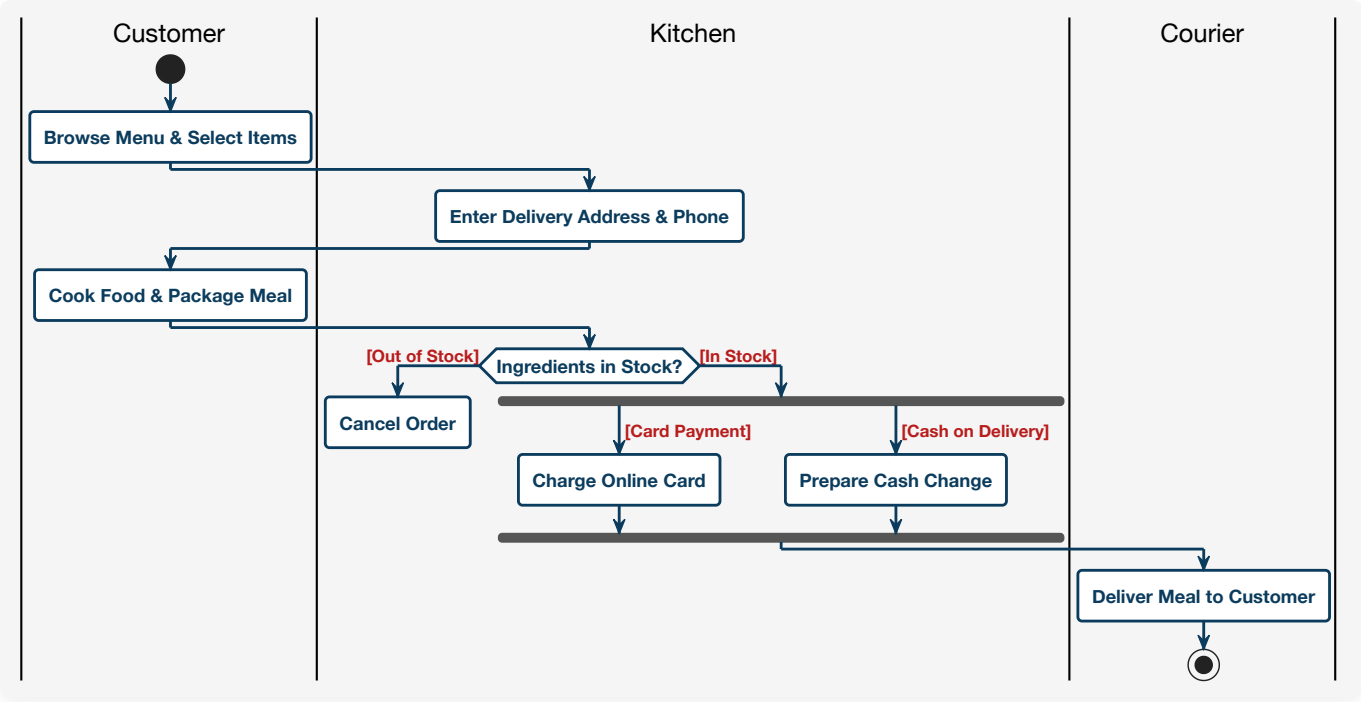


圖 4.6.7：外送履約流程活動圖草稿。包含分岔與分支混淆、泳道顛倒、懸掛流程與死鎖危機四大嚴重破綻。

架構診斷審查：四大致命反模式

1. **分岔同步條 vs. 條件決策菱形混淆 (Fork Bar vs. Decision Diamond)：**
   - *破綻：* 在廚房泳道中，竟然以 `fork` 分岔同步條分出 `[Card Payment]`（信用卡支付）與 `[Cash on Delivery]`（貨到付款）兩條帶守衛條件的分支。
   - *修正：* `fork` 分岔條的語義是「多執行緒並發」，所有流出分支會同時並行啟動！將互斥的付款方式掛在 fork 上，代表系統一邊刷顧客的信用卡、同時又吩咐外送員向顧客收現金！互斥條件二選一必須使用「決策菱形 (Decision Diamond, ◇)」，嚴禁使用同步條。
2. **泳道職責嚴重顛倒 (Swimlane Responsibility Inversion)：**
   - *破綻：* 動作職責完全錯置：`顧客 (Customer)` 泳道被指派去執行 `備餐與包裝餐點`，而 `餐廳廚房 (Kitchen)` 泳道卻被指派去執行 `輸入外送地址與電話`！
   - *修正：* 泳道代表該活動之法定執行實體。顧客負責填寫外送地址；餐廳廚房負責料理餐點與打包。
3. **懸掛控制流與遺漏終止節點 (Dangling Control Flow & Missing Final Node)：**
   - *破綻：* 當食材檢核為 `[Out of Stock]`（庫存不足）時，流程執行 `Cancel Order`（取消訂單）後直接中斷懸掛在半空中，未連接任何終止節點。
   - *修正：* UML 活動圖嚴禁懸掛箭頭。所有終止路徑必須明確引導至「活動終止節點 (Activity Final Node, ◉ 實心圓帶外圈)」或流程終止節點 (`⊗`)。
4. **非對稱同步之死鎖危機 (Deadlock Hazard on Asymmetric Synchronization)：**
   - *破綻：* 將互斥的決策分支直接匯入 `join` 結合同步條中。
   - *修正：* Join 條會永久阻塞等待所有輸入箭頭上的 Token 到達。若將互斥分支（永遠只有一個 Token）接進 Join，系統將因永遠等不到另一個 Token 而陷入永久死鎖 (Deadlock)！互斥分支必須用合併菱形 (Merge) 匯流，Join 條僅能用於同步並行執行緒。

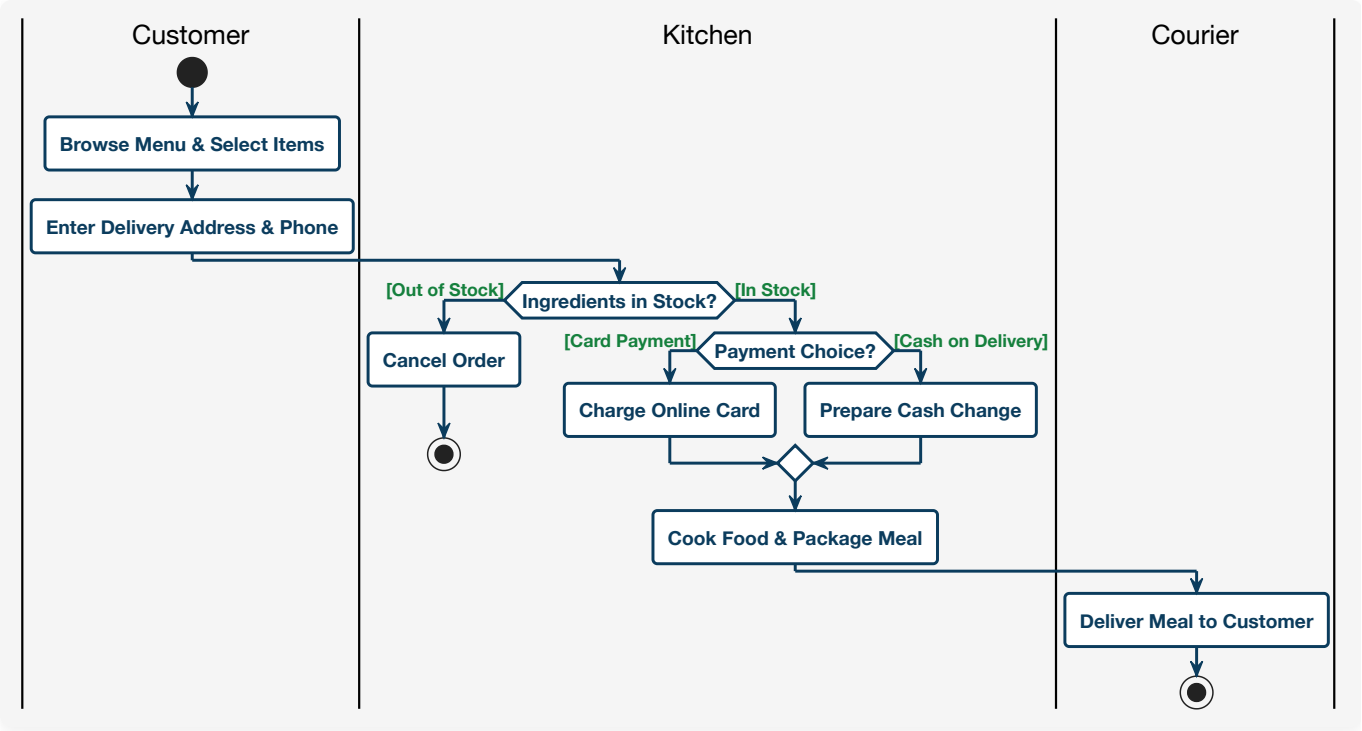


圖 4.6.8：修正後之標準活動圖。正確運用決策菱形、分明泳道責任、補齊終止節點並杜絕死鎖危機。

4.6.7 動態模型深度對比：狀態機圖 vs. 活動圖架構決策矩陣

狀態機圖與活動圖皆用於塑模系統的動態行為，但兩者回答的本質工程問題截然不同：

評估向度	狀態機圖 (State Machine / Statechart)	活動圖 (Activity Diagram / Workflow)
核心視角	**實體中心 / 反應式 (Entity-Centric)：** 描繪「單一實體」從誕生到消滅之完整生命週期。	**流程中心 / 程序的 (Process-Centric)：** 描繪跨角色、跨系統之「端到端工作流程」。
主要節點	**狀態 (State)：** 物件等待事件到達或持續執行背景運算的持久靜止模式。	**動作 (Action / Activity)：** 正在被執行的具體運算步驟或業務任務（動詞）。
推進驅動力	**事件驅動 (Event-Driven)：** 需外來事件刺激觸發變遷（`Event [Guard] / Action`）。	**完成驅動 (Completion-Driven)：** 前一項任務完成後自動沿控制流向下推進。
並行模型	**正交區域 (Orthogonal Regions)：** 複合狀態內的並行子狀態軌道 (`	
最佳應用	物件生命週期 (Order, Account, TCP 連線)、IoT 韌體裝置、狀態機模式。	業務流程自動化 (訂單履約流程)、批次排程、跨部門協同工作流程。

4.6.8 互動實作：運用 AI 演進活動圖工作流程 (Pair Hands-On Activity)

- **實作目標：** 在外送履約活動圖提示詞中引入一項實際營運之流程變異，並審查 AI 產出的 Token 控制流：

i. *選項 A (外送員拒單重新派單迴圈)：* 外送員可拒絕接單，調派引擎需進入 `repeat...repeat while` 迴圈重新搜尋下一位外送員。
ii. *選項 B (廚房品管檢驗重做迴圈)：* 餐點打包前進行 QA 檢驗，若未達標準需退回重新製作。
iii. *選項 C (顧客門市自取直通分支)：* 若訂單為自取 (Takeaway)，直接略過所有外送員搜尋與配送泳道，通知顧客到店自取。

- **架構檢核清單：**
  - 泳道歸屬：每個動作是否皆正確歸入權責角色，未出現越俎代庖之角色顛倒？
  - 迴圈終止性：`repeat` 迴圈是否具備明確的跳出條件，避免陷入無窮迴圈？
  - 避免死鎖：互斥分支是否嚴格採用 Merge 菱形匯合，絕未誤用 Join 同步條？

🤦 **觀念檢核測驗 (CCQ 6) — 活動圖同步條 vs. 決策菱形**

問題

在 UML 活動圖中，**分岔/結合 (Fork/Join 同步條)** 與 **決策/合併 (Decision/Merge 菱形)** 的核心差異為何？

- A. Fork/Join 用於將單一流程切分為多條「並行同時執行」的執行緒；Decision/Merge 則依據布林條件由多條路徑中「嚴格選擇其一 (互斥)」執行。
- B. Fork/Join 用於類別繼承；Decision/Merge 用於物件實體化。
- C. Fork/Join 只能處理資料流；Decision/Merge 只能處理錯誤例外。
- D. Fork/Join 必須等待人工確認；Decision/Merge 由系統計時器自動觸發。

Interactive Activity (線上作答)

▶ 點擊展開參考答案與解析

4.7 行為與狀態機圖模型 (State Machine Modeling)

許多核心業務實體（如電商訂單、網路連線 Session、物聯網溫控閥門）其行為具備高度的**狀態相依性 (State-Dependent Behavior)**。狀態機模型以單一實體為核心，捕捉其在整個生命週期中如何因應事件而改變狀態。

4.7.1 同一事件，不同反應

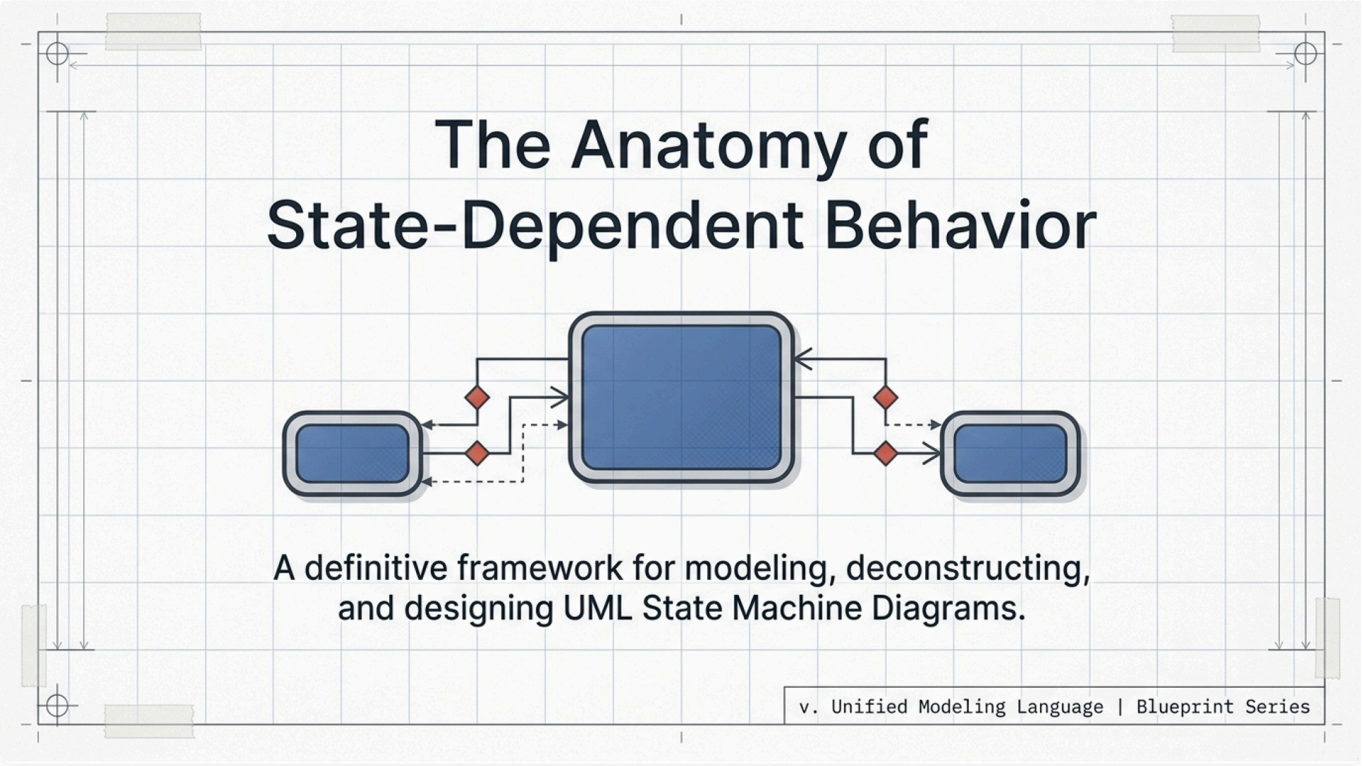


圖 4.7.1：狀態相依行為解剖學。物件對事件的反應取決於其當前處於何種內部狀態。

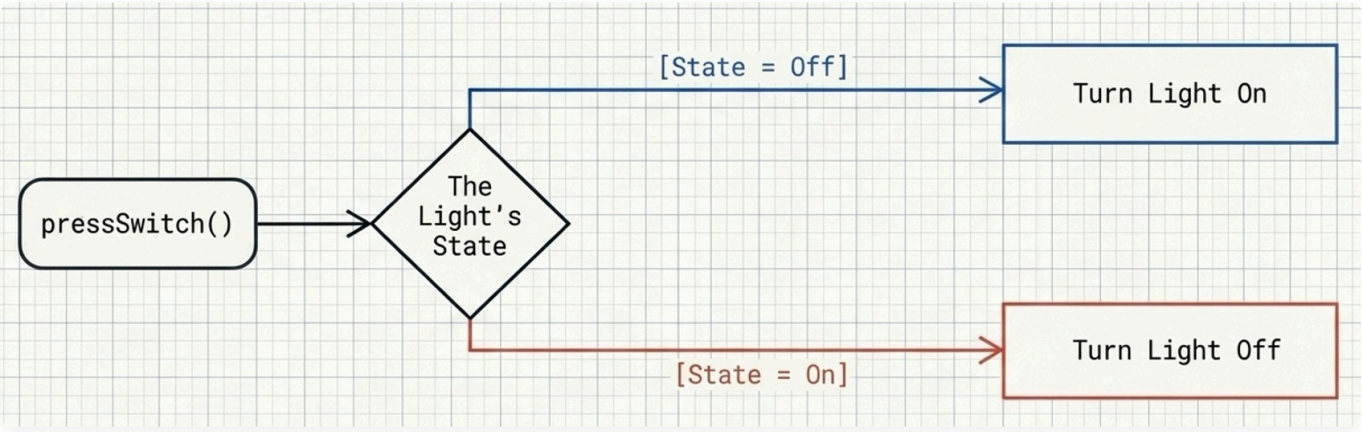


圖 4.7.2：相同事件在不同狀態下的相異反應。例如對未付款訂單觸發「取消」會成功退回購物車，但對「運送中」訂單觸發取消則會被系統拒絕。

狀態機的核心定理在於：**系統的反應不僅取決於輸入事件，更受控於當前狀態。**

4.7.2 五元素變遷語法與觸發因子

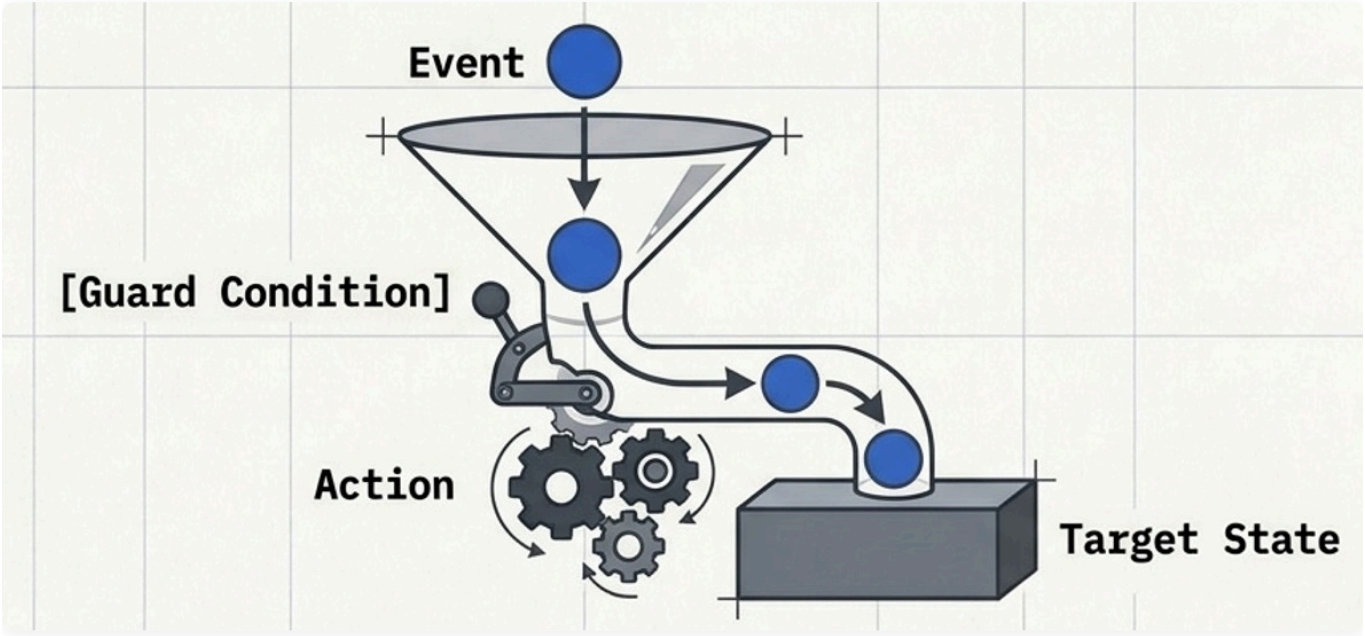


圖 4.7.3：狀態變遷力學公式。Trigger [Guard] / Effect 語法結構。

UML 狀態變遷以帶箭頭的直線連接來源狀態與目標狀態，完整語法公式為：

```
觸發事件 (Trigger) [布林守衛條件 (Guard)] / 執行動作 (Effect/Action)
```

1. **Trigger (觸發事件)**：引發變遷的事件（如 `cardInserted`、`timeoutExpired`）。
2. **Guard (守衛條件)**：以中括號括起之布林條件（如 `[balance >= orderAmount]`），僅在評估為 true 時變遷方能觸發。
3. **Effect / Action (觸發動作)**：變遷跨越時執行之瞬間原子性操作（如 `/ deductAccountBalance()`）。

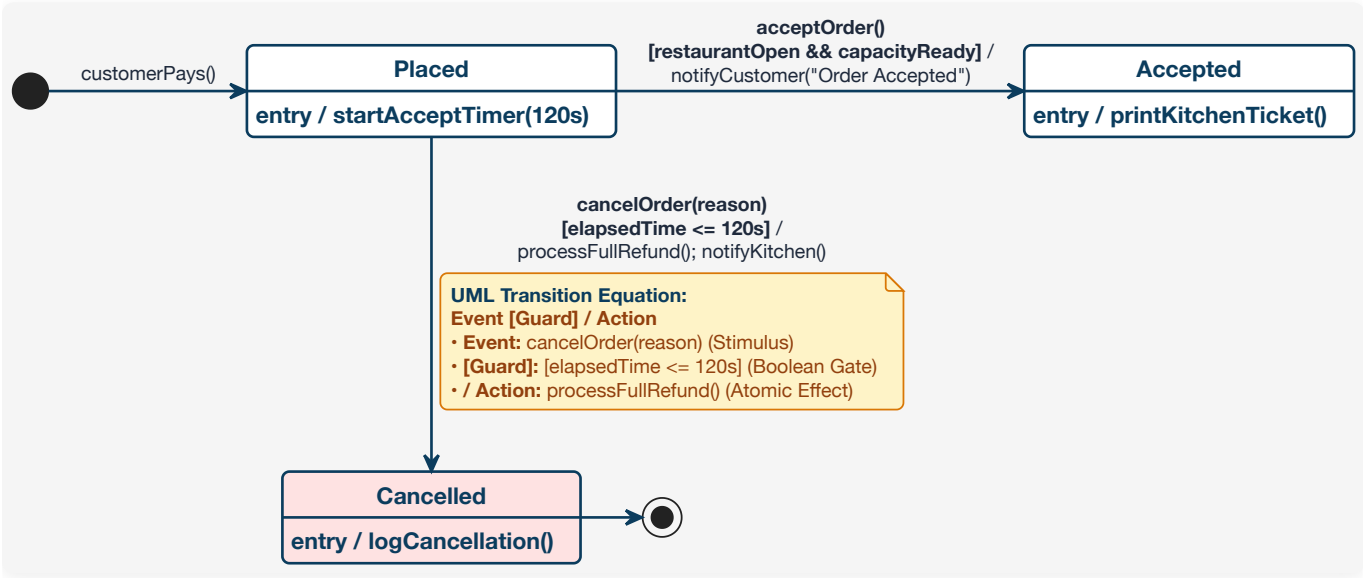


圖 4.7.4：變遷實戰：外送平台訂單案例。套用五元素公式落實「下單 2 分鐘內允許免費取消」之業務規則。

在外送平台中：

- **具體變遷語法：** `cancelOrder(reason) [elapsedTime <= 120s] / processFullRefund(); notifyKitchen()`。
- **刺激事件 (Trigger)：** 顧客在手機 App 點擊「取消訂單」。
- **守衛條件 (Guard)：** 系統嚴格檢查 `[elapsedTime <= 120s]`。若下單已超過 2 分鐘，守衛條件評估為 **false**，變遷被強制攔截，狀態絕不改變。
- **動作效果 (Action)：** 瞬間原子操作執行 `/ processFullRefund()`，並非同步通知廚房停止備餐。
- **目標狀態：** 訂單進入 `Cancelled` 並觸發 `entry / logCancellation()`。

4.7.3 動作 (Action) vs. 活動 (Activity) 核心語義辨析

Feature	Action	Activity
Nature	Atomic Computation (Operation call, destroy object)	Non-atomic computation
Duration	Instantaneous	Ongoing (Runs to completion or indefinitely)
Interruptibility	NEVER interruptible	YES, can be interrupted by events
Triggers	On Entry, Do, On Event, On Exit	Sustained state presence

圖 4.7.5：動作 (Action) 與活動 (Activity) 之本質差異。Action 瞬間完成且不可中斷；Activity 持續耗時且可被外來事件隨時終止。

特性維度	動作 (Action: `entry`, `exit`, `/`)	活動 (Activity: `do / ...`)
時間長度	**零耗時 (瞬時完成 Instantaneous)**	**持續耗時 (Takes Time)**
中斷性	**不可中斷 (Atomic 原子操作)**	**可中斷 (Interruptible)**，外部變遷一觸發即刻被終止
宣告位置	`entry /`, `exit /` 或變遷線上之 `/ effect`	僅能在狀態方塊內部以 `do /` 宣告

邊界執行：進入動作 (Entry) 與離開動作 (Exit)

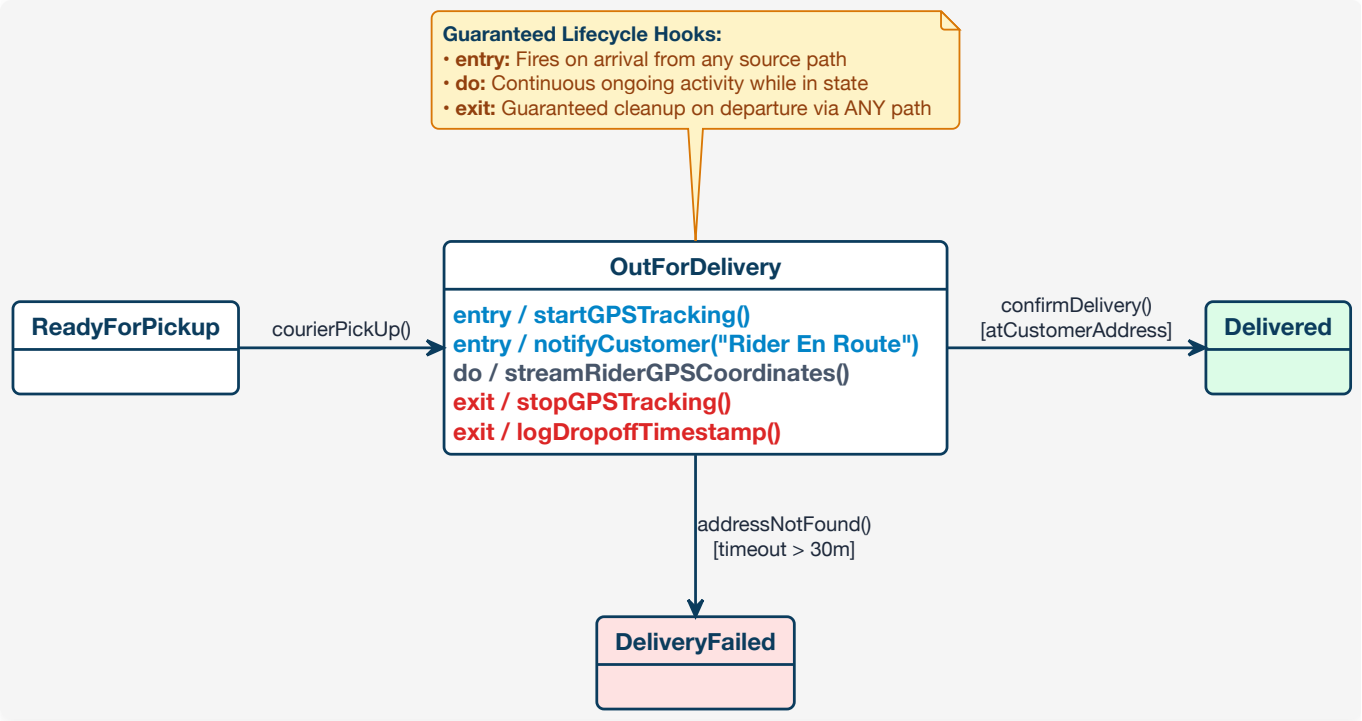


圖 4.7.6：邊界執行：進入與離開動作在《外送平台》的保證執行架構。

將初始化與清理邏輯綁定於狀態邊界，可徹底消除在多條流出變遷線上重複撰寫相同清理代碼的冗餘：

- **進入動作 (`entry / startGPSTracking()`):** 只要一進入 `OutForDelivery`（外送中）狀態，系統保證瞬間啟動外送員 GPS 定位並通知顧客。
- **持續活動 (`do / streamRiderGPSCoordinates()`):** 在外送途中持續串流回傳座標，一旦到達或訂單取消即被中斷。
- **離開動作 (`exit / stopGPSTracking()`):** 無論訂單是正常送達 (`confirmDropoff()`) 或異常失聯 (`addressNotFound()`)，離開狀態的瞬間**保證絕對執行**關閉 GPS 模組並記錄時間戳記。

4.7.4 複合狀態 (Composite States) 與歷史假狀態 (History)

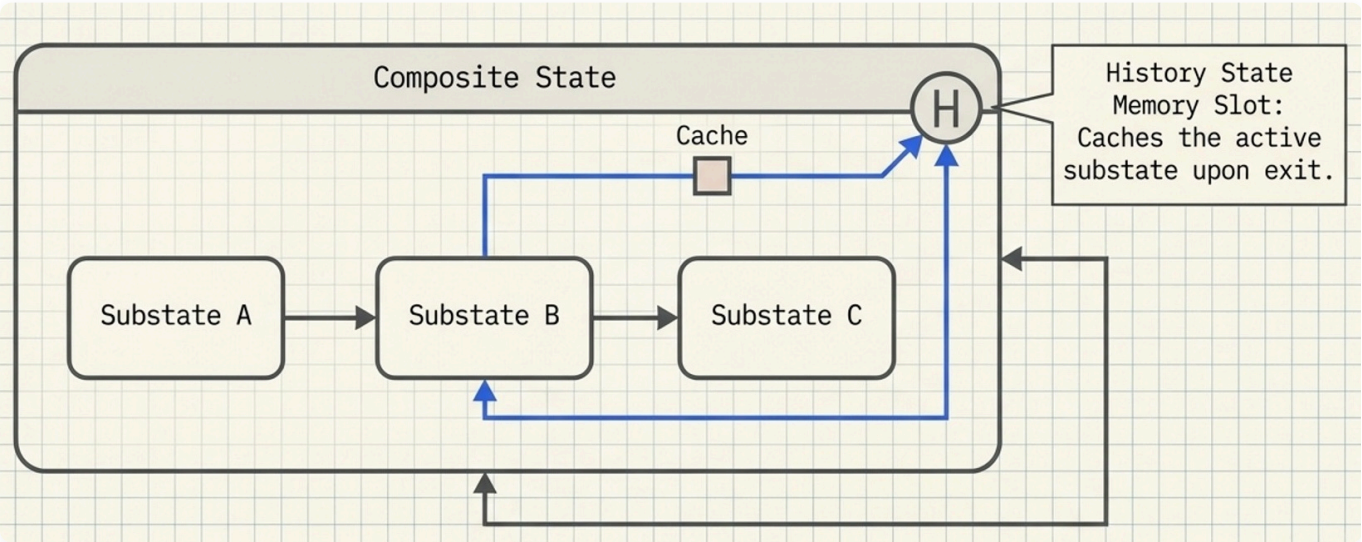


圖 4.7.7：複合狀態與歷史假狀態。以巢狀結構解決狀態爆炸問題，並透過 H 與 H 記號保存中斷前的內部狀態。

- **複合狀態 (Composite State)**：將多個相關子狀態封裝於一個父狀態方框中。任何連接至父狀態外框的變遷（如 `emergencyCancel()` 附加於 `OrderFulfillment`），對所有巢狀子狀態自動生效，徹底杜絕「狀態爆炸」問題。
- **並行正交區域 (Concurrent Orthogonal Regions, `||`)**：在複合狀態內部以虛線劃分多個同時並發推進的獨立子狀態軌道。

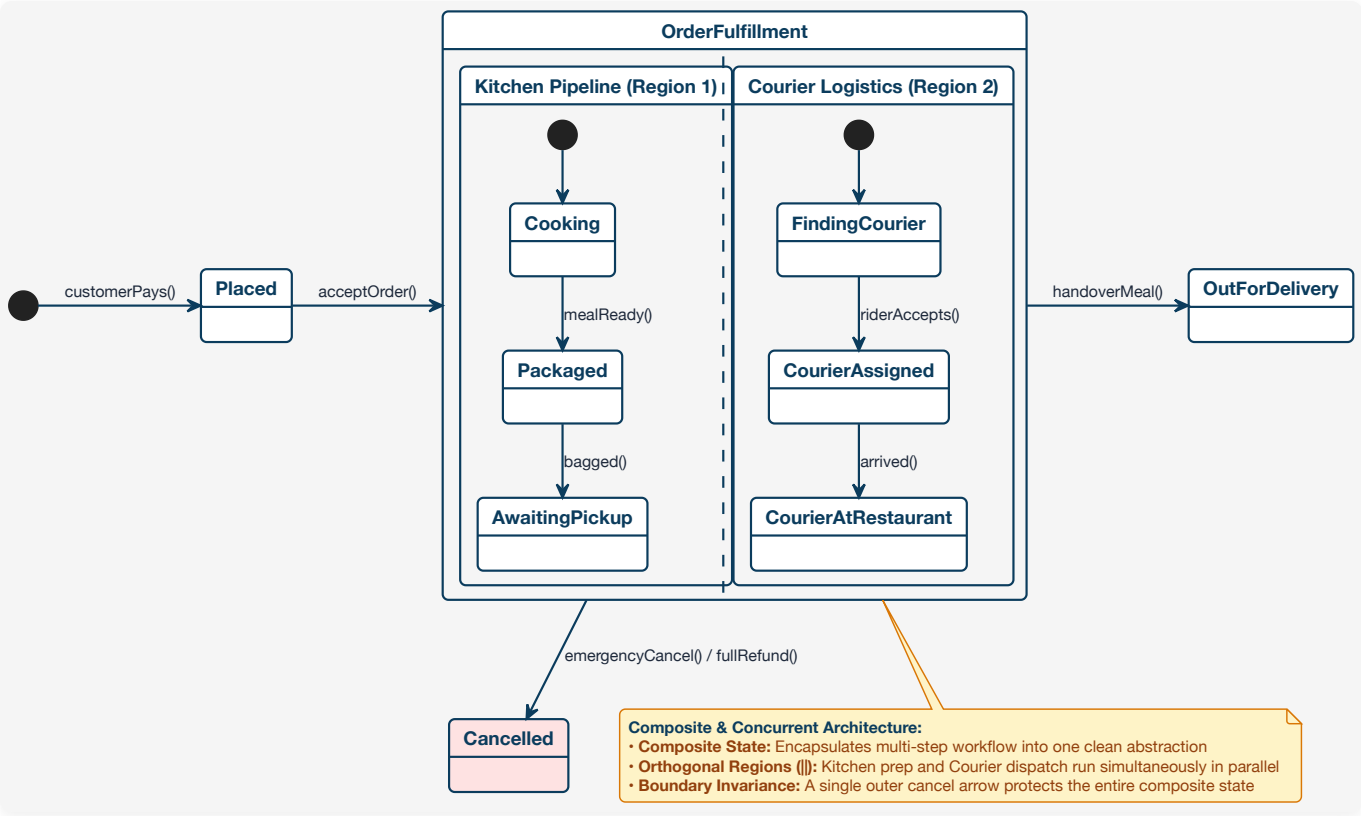


圖 4.7.8：外送平台履約之複合與並行狀態。同時並發推進「廚房備餐」與「外送調派」兩條正交子狀態軌道。

在 `OrderFulfillment`（訂單履約）超狀態內部，兩條業務軌道各自獨立推進：

1. **廚房管線 (區域 1)：** `Cooking` → `Packaged` → `AwaitingPickup`。
2. **物流管線 (區域 2)：** `FindingCourier` → `CourierAssigned` → `CourierAtRestaurant`。

兩者在背景同時執行，互不阻塞。

透過歷史狀態記憶恢復進度 (`H` / `H*`)

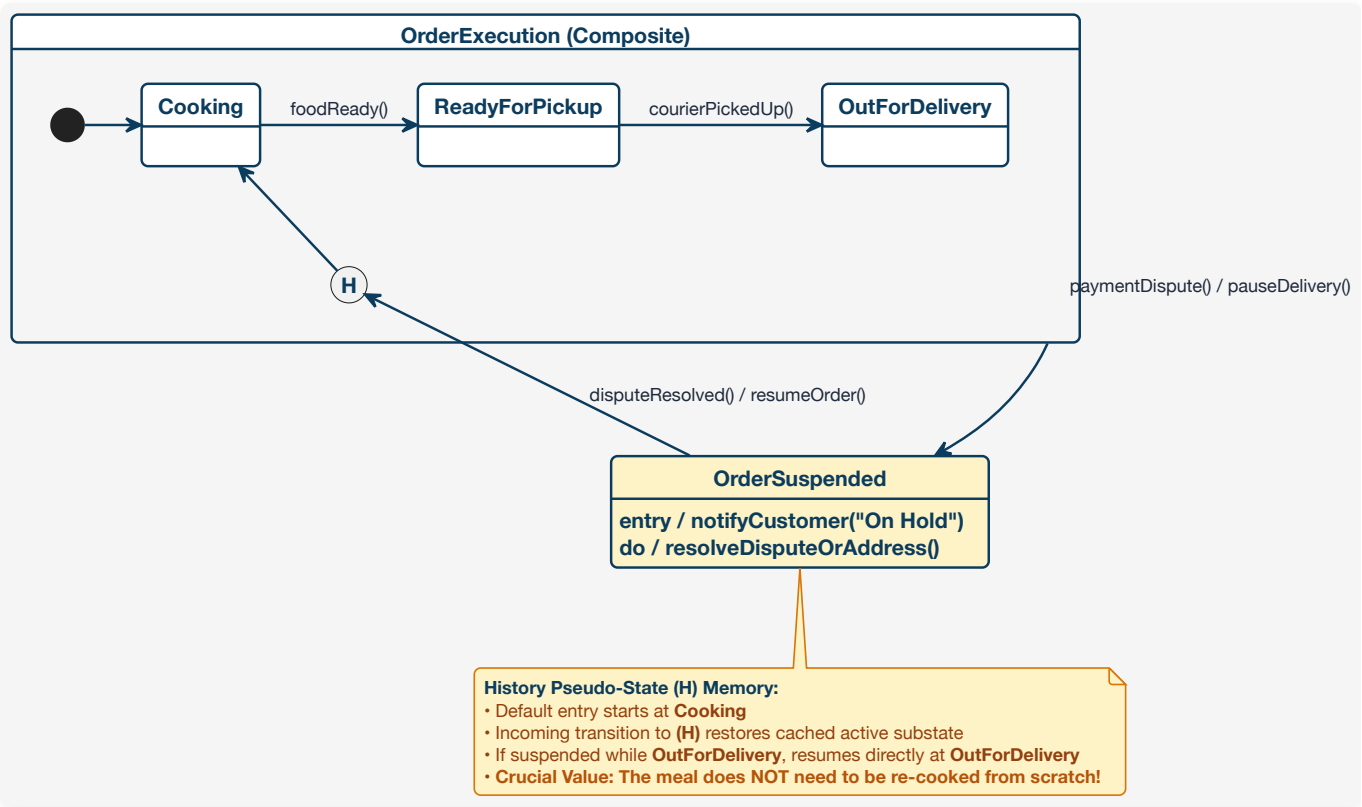


圖 4.7.9：歷史狀態記憶機制。在突發爭議中斷後精準恢復狀態，避免重頭開始備餐。

當訂單處於 `OutForDelivery`（外送中）時，若突然觸發信用卡爭議或地址確認風控，訂單會被中斷並轉移至 `OrderSuspended`（訂單暫停）。

- **無歷史狀態：** 爭議解除後重新進入履約狀態，系統只能回到預設初始狀態（`Cooking`），導致餐廳浪費食材、被逼重做一份餐點！
- **具備深歷史狀態（`H*`）：** 恢復變遷直接指向 `(H)` 節點，系統立即恢復至中斷前的 `OutForDelivery` 狀態，外送員直接繼續配送原餐點！

4.7.5 外送平台訂單生命週期狀態機實戰案例

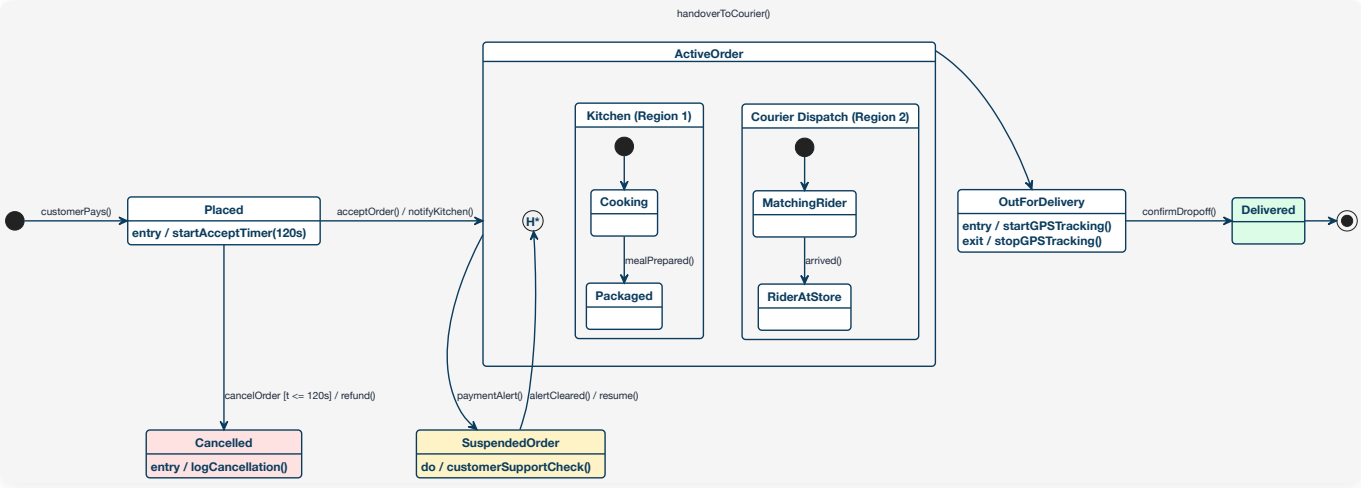


圖 4.7.10：外送平台訂單架構統一藍圖。整合守衛變遷、並行正交履約、深歷史暫停與邊界 Entry/Exit 動作之企業級藍圖。

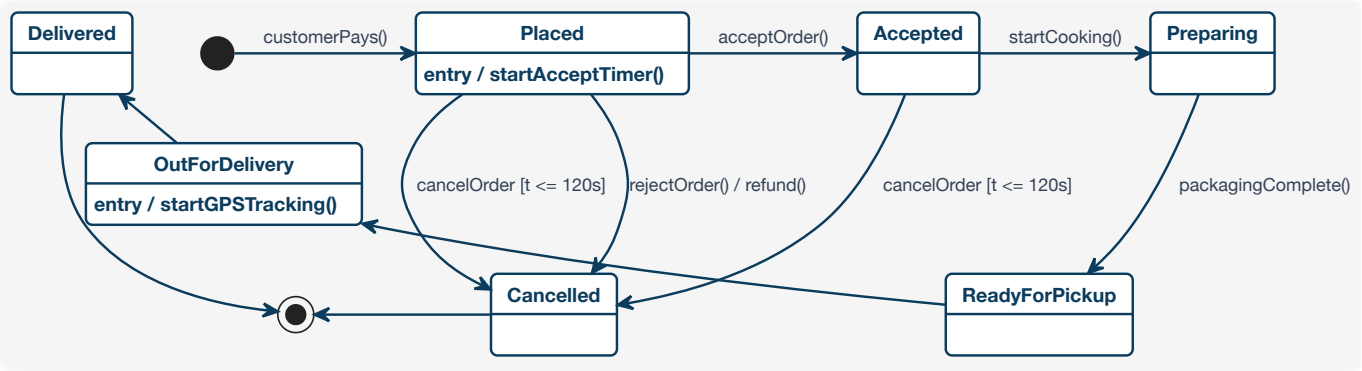


圖 4.7.11：外送平台基準訂單狀態機圖。描繪 Placed, Accepted, Preparing, ReadyForPickup, OutForDelivery, Delivered 與 Cancelled 等核心狀態。

```
@startuml
[*] -right-> Placed : customerPays()

state Placed {
  Placed : entry / startAcceptTimer()
}
state OutForDelivery {
  OutForDelivery : entry / startGPSTracking()
}

Placed -right-> Accepted : acceptOrder()
Accepted -right-> Preparing : startCooking()
Preparing -down-> ReadyForPickup : packagingComplete()
ReadyForPickup -left-> OutForDelivery : courierPickedUp()
OutForDelivery -left-> Delivered : confirmDropoff()
Delivered -down-> [*]

Placed -down-> Cancelled : rejectOrder() / refund()
Placed -down-> Cancelled : cancelOrder [t <= 120s]
Accepted -down-> Cancelled : cancelOrder [t <= 120s]
Cancelled -left-> [*]
@enduml
```

4.7.6 診斷式案例實戰：抓出狀態機圖的行為破綻（訂單生命週期）

狀態機圖規範了系統在執行時期的法定狀態與變遷約束。若狀態語意不清或缺少守衛，極易引發競態衝突與死鎖。

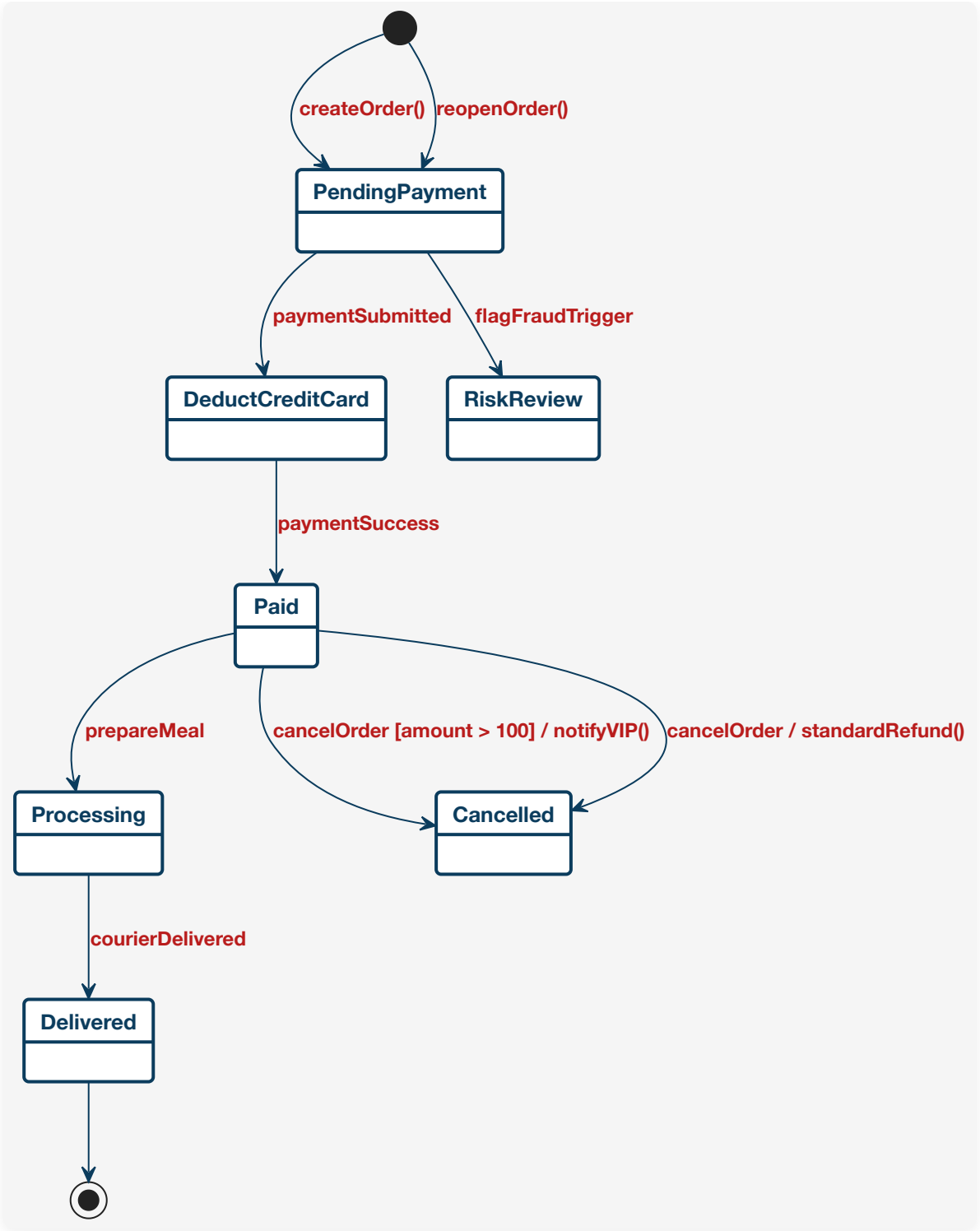


圖 4.7.12：訂單生命週期狀態機草稿。包含動作誤植、黑洞陷阱、非確定性變遷與終止狀態違規四大破綻。

架構診斷審查：四大致命反模式

1. **瞬態動作誤植為持久狀態 (Action vs. State Confusion)：**
   - *破綻：* 狀態圖中出現了一個名為 `DeductCreditCard`（扣信用卡款）的狀態方塊。
   - *修正：* 狀態 (State) 代表實體處於某種持久滿足的條件或靜止等待模式（如 `PendingPayment`, `Paid`）。扣款是瞬間完成的原子動作，耗時為零，必須標記為變遷上的 Effect 動作（`pay / deductCreditCard()`），絕不能畫成狀態方塊。
2. **黑洞陷阱狀態 (Black Hole / Trap State)：**

- *破綻：* `RiskReview`（風控審查）狀態只有流入箭頭，但流出箭頭數量為零。
- *修正：* 訂單一旦進入 `RiskReview` 即永久被困死在系統中，無法通過審查、無法取消、也無法終止。任何中間狀態皆必須具備至少一條引導至後續或終止狀態的流出路徑。

3. **非確定性變遷衝突 (Non-Deterministic Race Conflict)：**
   - *破綻：* 從 `Paid` 出發有兩條 `cancelOrder` 變遷，一條帶有 `[amount > 100]`，另一條完全無守衛條件。
   - *修正：* 若顧客取消一筆 $150 的訂單，兩條變遷同時被激活！UML 狀態機嚴禁非確定性競態。從同一個狀態出發、共享相同觸發事件之變遷，其守衛條件必須嚴格互斥（例如：`[amount > 100]` 與 `[amount <= 100]`）。
4. **違規由終止狀態引出變遷 (Illegal Outgoing Transition from Final State)：**
   - *破綻：* 從終止狀態 `[*]` 拉出一條 `reopenOrder()` 箭頭回到 `PendingPayment`。
   - *修正：* 終止狀態代表該物件實體生命週期的徹底終結。UML 語法規範嚴禁由終止狀態引出任何箭頭！若訂單允許重新開啟，代表先前根本未真正終結。

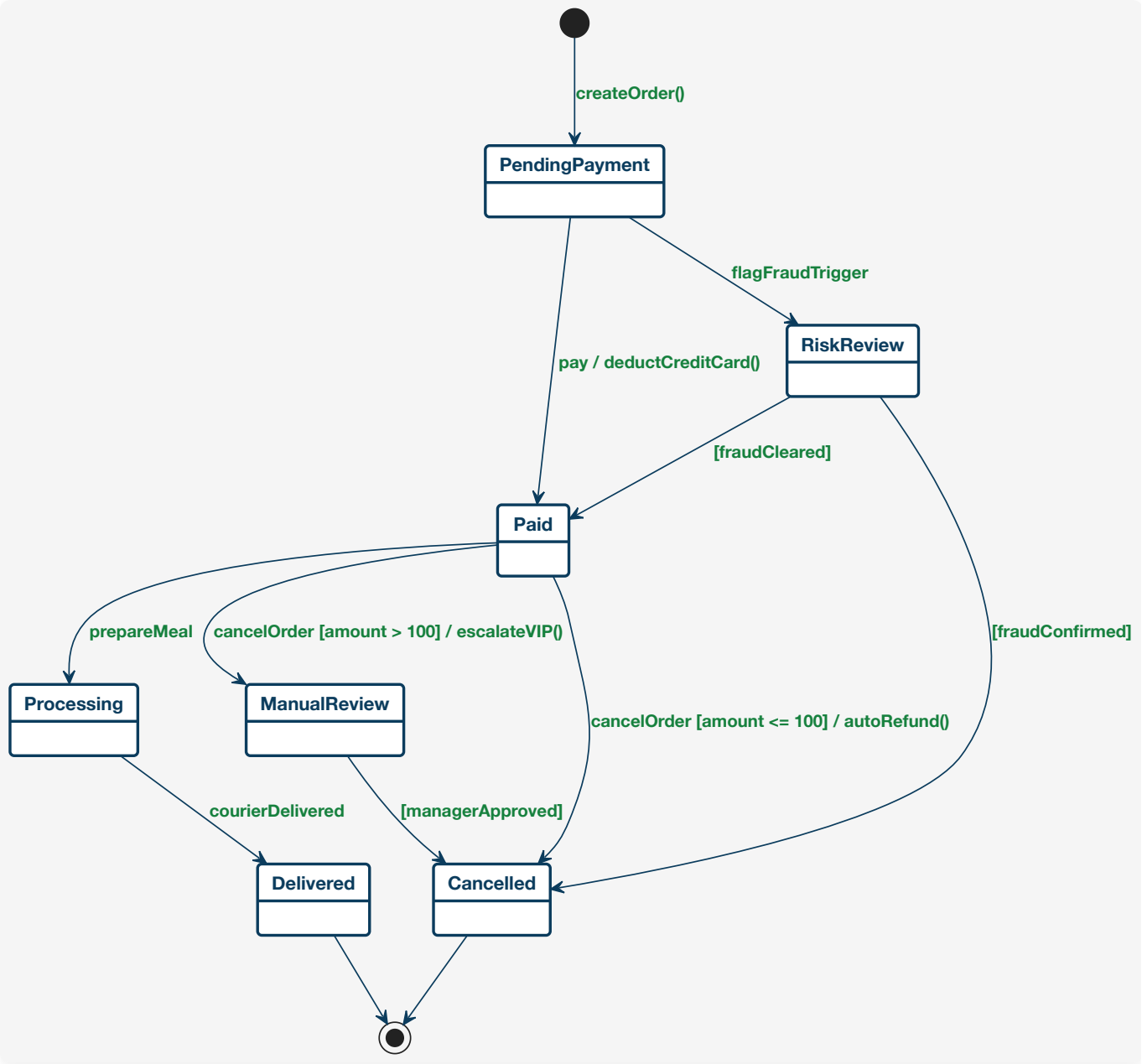


圖 4.7.13：修正後之標準狀態機圖。瞬態動作轉化為變遷效果、消除黑洞陷阱、規範互斥守衛並嚴守終止狀態不可逆性。

4.7.7 互動實作：運用 AI 演進狀態機模型 (Pair Hands-On Activity)

- **實作目標：** 在外送訂單狀態機提示詞中引入一項真實世界的營運例外狀態，並審查 AI 產出的生命週期約束：
  i. *選項 A (外送員途中車禍/拋錨重新指派)：* 外送途中若騎手拋錨，訂單變遷退回至 `ReadyForPickup`（`[retries < 3]`），重新調派下一位外送員，避免廚房重做。
  ii. *選項 B (餐點製作中顧客申請延期配送)：* 顧客在 `Preparing` 狀態下申請變更送達時間。
  iii. *選項 C (送達現場聯絡不上顧客退回餐廳)：* 送達現場等待超過 10 分鐘，狀態轉移至 `FailedDelivery`，再轉移至 `ReturnedToStore` 後終止。
- **架構檢核清單：**
  - 狀態命名紀律：狀態是否皆以被動狀態、條件或形容詞命名，未出現動詞動作？
  - 非法變遷防堵：是否嚴格禁止在餐點已出發配送（`OutForDelivery`）時直接隨意取消訂單？
  - 終止路徑完備：所有異常終止狀態是否皆有箭頭正確連接至終止節點 `[*]`？

觀念檢核測驗 (CCQ 7) — 狀態機動作 vs. 活動

問題

在 UML 狀態機圖中，**動作 (Action，如 transition 上的 `/ refund()` 或 `entry /`) 與 活動 (Activity，如 `do /`) 最根本的執行語義差異為何？**

- A. Action 是瞬間完成、不可中斷的原子性操作；Activity 是耗時進行的持續運算，且可被外來事件隨時中斷。
- B. Action 只能寫 Python；Activity 只能寫 SQL。
- C. Action 只能在進入狀態時執行；Activity 只能在離開狀態時執行。
- D. Action 專門處理例外失敗；Activity 專門處理成功路徑。

Interactive Activity (線上作答)

▶ 點擊展開參考答案與解析

4.8 PlantUML 文字宣告式塑模 (Architecture-as-Code)

傳統採用閉源 GUI 繪圖軟體（如 Visio、Enterprise Architect）維護系統圖表的做法，在現代 CI/CD 與敏捷開發中已被證明是重大維運負擔。以純文字撰寫架構的 **PlantUML**，實現了「架構即代碼 (Architecture-as-Code)」的工程願景。

4.8.1 為何捨棄 GUI 拖拉？純文字塑模的工程價值

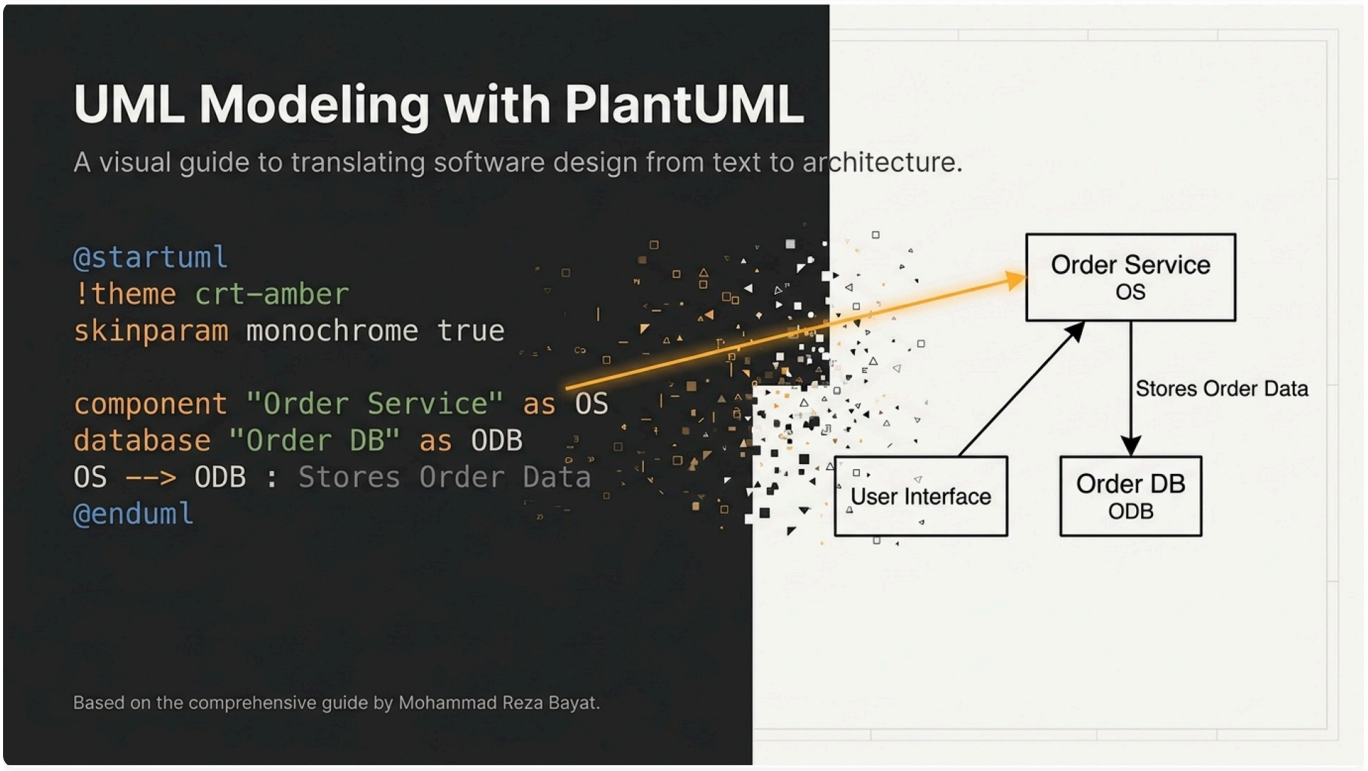


圖 4.8.1：PlantUML 代碼驅動架構。純文字源碼可進行版本控制、PR 審查與自動化生成。

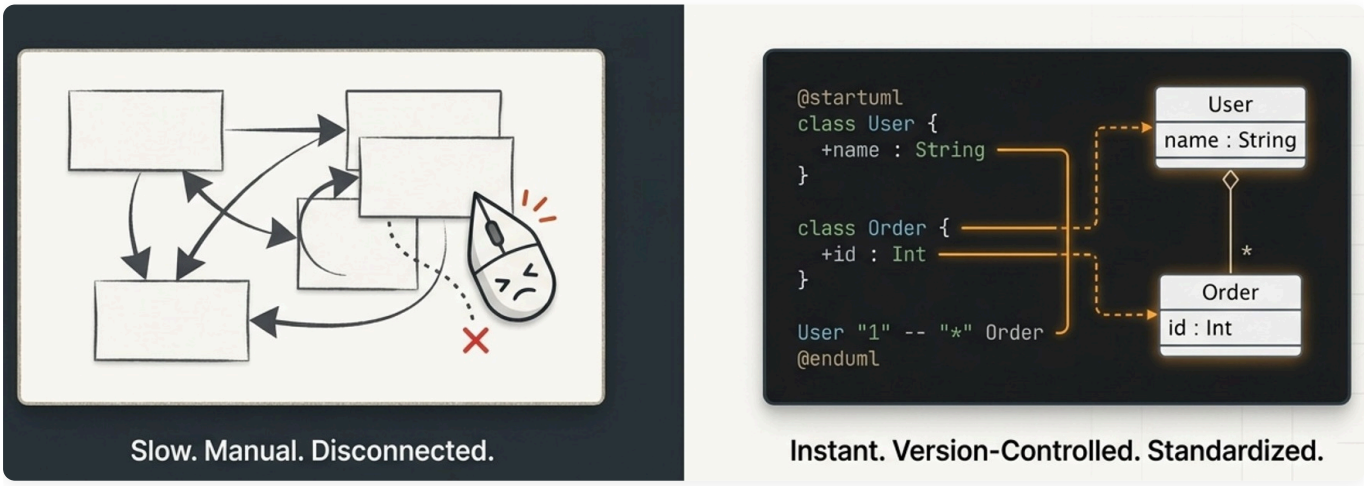


圖 4.8.2：停止拖拉滑鼠，開始編寫架構。文字宣告式建模徹底解決二進位圖片衝突與手動排版浪費時間的困境。

評估維度	傳統 GUI 圖形拖拉軟體	PlantUML (架構即代碼)
儲存格式	私有二進位檔案或極度冗長的 XML	純文字檔案 (`.puml` / `.md`)
Git 版本控制	二進位衝突無法合併、Diff 毫無意義	原生支援分支合流、清晰行級 Diff 與 PR Review 審查
CI/CD 自動化	需人工手動截圖匯出	GitHub Actions 背景無頭自動編譯向量 SVG/PDF 手冊
AI 輔助合成	AI 無法直接操作滑鼠拖拉生成	LLM 能直接生成完整正確的 PlantUML 純文字代碼

4.8.2 五大核心圖表 PlantUML 跨模型語法矩陣

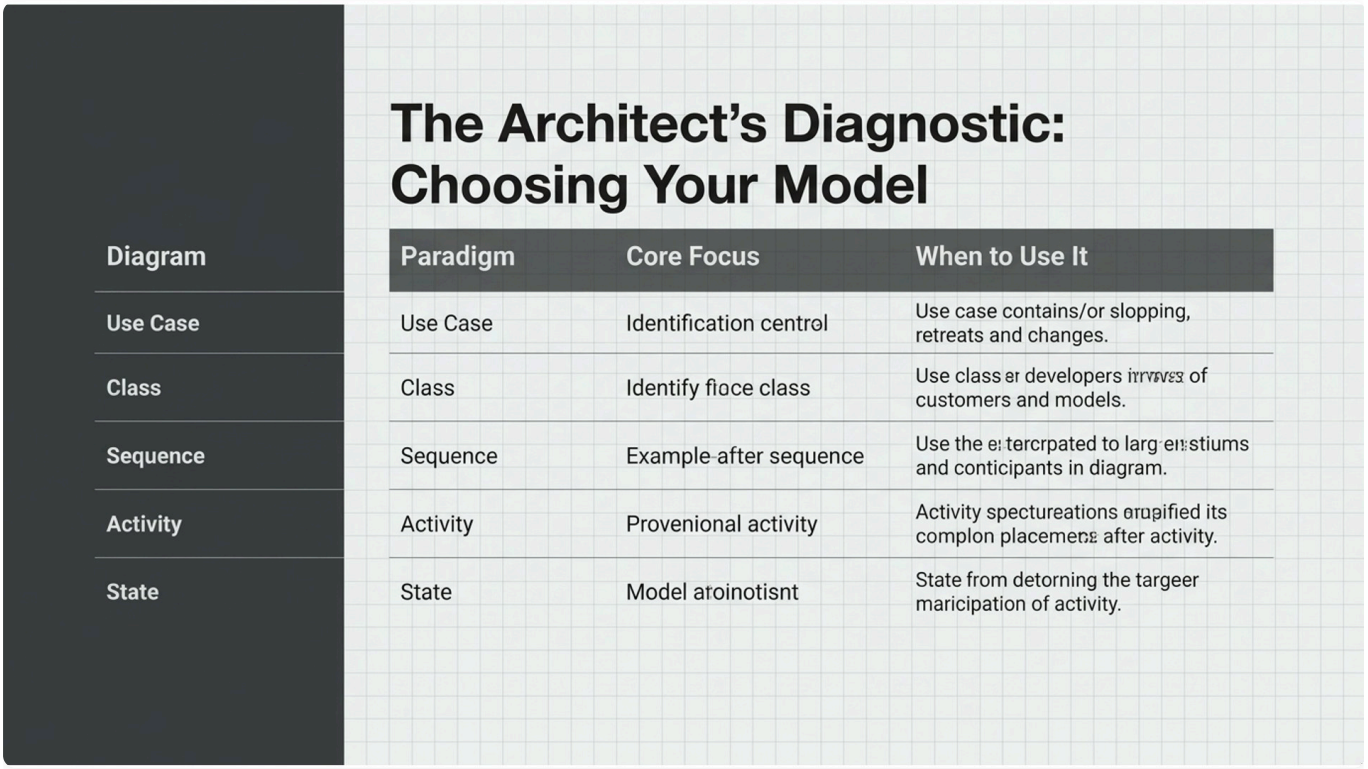


圖 4.8.3：PlantUML 跨模型語法速查指南。統一在 @startuml 與 @enduml 區塊內宣告類別、循序、活動、狀態機與使用案例圖。

```
@startuml
skinparam monochrome true
skinparam shadowing false

title 外送平台結帳與扣款循序圖 (Food Delivery Checkout)

actor 顧客 as Customer
participant "結帳前端UI" as UI <<Boundary>>
participant "訂單控制器" as OrderCtrl <<Control>>
participant "第三方金流閘道" as PaymentAPI <<Boundary>>
database "PostgreSQL" as DB <<Entity>>

Customer -> UI : 點擊『確認下單』submitOrder()
activate UI

UI -> OrderCtrl : 建立訂單與扣款 checkout(cartId)
activate OrderCtrl

OrderCtrl -> PaymentAPI : 授權扣款 authorizePayment(amount)
activate PaymentAPI
PaymentAPI --> OrderCtrl : 授權成功 token (200 OK)
deactivate PaymentAPI

alt 扣款授權通過 [paymentApproved]
    OrderCtrl -> DB : 建立 Order 實體記錄並儲存
    activate DB
    DB --> OrderCtrl : 訂單已建立 (ID: #8820)
    deactivate DB
    OrderCtrl --> UI : 結帳成功畫面通知
    UI --> Customer : 顯示訂單追蹤動態
else 扣款授權失敗 [else]
    OrderCtrl --> UI : 顯示刷卡失敗警示 (餘額不足)
    UI --> Customer : 提示更換信用卡
end

deactivate OrderCtrl
deactivate UI
@enduml
```

🤦 觀念檢核測驗 (CCQ 8) — 文字宣告式塑模工程價值

問題

相較於傳統封閉式 GUI 圖形拖拉繪圖軟體，採用如 **PlantUML** 等文字宣告式塑模工具的最大工程優勢是什麼？

- A. 完全不需要學習 UML 規範即可自動產生圖表。
- B. 純文字原始碼可直接納入 Git 版本控制、在 Pull Request 中進行 diff 審查，並整合至 CI/CD 自動化建置。
- C. PlantUML 能自動產生完整的商業生產微服務與雲端資料庫。
- D. PlantUML 只能繪製類別圖，無法繪製動態流程圖。

Interactive Activity (線上作答)



▶ 點擊展開參考答案與解析

4.9 模型選擇決策框架與課程總結 (Decision Framework & Recap)

4.9.1 六大核心模型架構決策矩陣

UML 核心圖表	核心回答之工程問題	關鍵視覺語法元素	最佳實務應用場景
1. 使用案例圖 (Use Case)	系統外部使用者可獲得「何種具備商業價值的服務」？	參與者小人、橢圓案例、系統邊界矩形框、`<<include>>`、`<<extend>>`	確立專案範疇、合約規格邊界與利害關係人協議共識。
2. 類別圖 (Class Diagram)	領域實體具備「何種靜態資料結構與彼此間的關聯繼承」？	三層隔間框、可見度符號 (`+`, `-`)、組合 (`◆`)、聚合 (`◇`)	物件導向領域模型設計、關聯式資料庫 Schema 映射。
3. 循序圖 (Sequence Diagram)	各物件如何「隨時間順序傳遞訊息調用完成特定情境」？	生命線、啟動條、同步/非同步箭頭、BCE 架構分工、複合片段 (`alt`, `loop`)	具體 API 交易呼叫追蹤、微服務調用鏈與分散式協定。
4. 活動圖 (Activity Diagram)	跨部門與組織的「演算法邏輯與並行工作流程」為何？	圓角動作節點、泳道分區、同步條 (Fork/Join)、決策菱形	業務流程自動化 (BPMN)、批次運算排程與並行流程。
5. 狀態機圖 (State Machine)	關鍵核心實體的「生命週期狀態因應事件如何變遷」？	圓角狀態方塊、觸發因子、守衛條件、Composite 狀態、歷史 `H`	具備狀態機特徵之物件 (訂單、連線協定、IoT 裝置)。
6. PlantUML 純文字代碼	如何讓視覺架構模型「享受代碼般的版本控制與自動化」？	純文字標記語法 (`@startuml ... @enduml`)	將架構代碼直接納入 Git 倉儲、CI/CD 自動編譯與 PR 審查。

4.9.2 五大模型演進實作解答與架構評分規準

活動解答 1：使用案例模型擴充 (Use Case Model Extensions)

- **架構方案與語義分析：**
  - **預約排程訂餐 (`<<extend>>`)：** `(Place Food Order) <.. (Schedule Future Delivery) : <<extend>>`，並附帶守衛條件 `[delivery_type == scheduled]`。屬於選擇性擴充功能，在結帳擴充點掛載，絕不污染基底案例。
  - **在線客服即時交談 (自主基底案例)：** 獨立的 `(Live Order Chat)` 使用案例，主要參與者 `Customer` 與次要參與者 `Support Agent`、`Courier` 協同。外部合作夥伴嚴格維持在系統邊界矩形框之外。
  - **多人團購分帳 (`<<extend>>`)：** 基底案例 `(Place Food Order)` 在進入購物車結帳前，由 `(Pool Group Cart)` 擴充注入。
- **PlantUML Delta 代碼：**

```
rectangle "Food Delivery Platform" {
  usecase "Place Food Order" as UC_Order
  usecase "Schedule Future Order" as UC_Sched
  usecase "Live Order Chat" as UC_Chat
  UC_Order <.. UC_Sched : <<extend>>
}
Customer --> UC_Order
Customer --> UC_Chat
UC_Chat --> Agent : <<assists>>
```

- **架構評分檢核：**
  - `[通過]` 外部協同角色 `Support Agent` 嚴格置於系統邊界之外。
  - `[通過]` 預約排程採用 `<<extend>>`，絕未誤用強制包含之 `<<include>>`。
  - `[通過]` 案例橢圓代表商業使用者目標，避免退化為 UI 按鈕點擊。

活動解答 2：領域類別模型深化 (Class Model Refinements)

- **架構方案與語義分析：**
  - **跨餐廳多店家購物車：** 主訂單 `Order` 拆解為餐廳專屬的子訂單 `SubOrder`。`Order` `"1"` `*--` `"1..*"` `SubOrder`（組合：主訂單刪除則子訂單同生共死）；`SubOrder` `"1..*"` `o--` `"1"` `Restaurant`（聚合：餐廳為獨立實體，生命週期獨立於訂單）。
  - **外送員評分與小費：** `DeliveryTask` `"1"` `o--` `"0..1"` `CourierReview`（弱聚合關聯，評分與小費為非強制之可選項目）。
  - **VIP 尊榮訂閱方案：** `Customer` `"1"` `o--` `"0..1"` `MembershipPlan`（採用策略模式解耦訂閱等級，避免多重繼承）。
- **PlantUML Delta 代碼：**

```
class Order {
  - orderId: UUID
  - total: Money
}
class SubOrder {
  - subTotal: Money
  - status: Status
}
class Restaurant {
  - name: String
  - address: String
}
class CourierReview {
  - rating: int
  - tip: Money
}
Order "1" *-- "1..*" SubOrder : aggregates
SubOrder "1..*" o-- "1" Restaurant : fulfills
SubOrder "1" *-- "1..*" OrderItem
DeliveryTask "1" o-- "0..1" CourierReview
```

- **架構評分檢核：**
  - `[通過]` 多重性邏輯：1 筆主訂單包含 `1..*` 筆子訂單；1 筆子訂單鎖定 1 間餐廳。
  - `[通過]` 關聯符號精確：實心菱形 (`*--`) 代表生命週期相依；空心菱形 (`o--`) 代表弱擁有引用。
  - `[通過]` 嚴格私有封裝 (`-`)，屬性型別明確。

活動解答 3：循序圖動態邊緣案例 (Sequence Interaction Edge Cases)

- **架構方案與語義分析：**
  - **第三方金流閘道逾時 (>5s)：** `CheckoutUI` 呼叫 `OrderController.submitPayment()`。控制器協調外部金流 `PaymentGateway.processPayment()`。在 `alt [金流於 5 秒內回應]` 內正常確認並派單；在 `else [逾時 > 5s]` 內，控制器觸發 `cancelTransaction()` 復原交易、記錄稽核日誌，並回傳帶有重試選項的 `PaymentTimeoutError`。
  - **雙軌混合支付流程：** 兩階段交易：先扣除 `Wallet Balance`（錢包餘額）；扣款成功再向信用卡請求差額。若信用卡刷卡失敗，觸發補償交易退回錢包扣款。
- **PlantUML Delta 代碼：**

```
UI -> Controller : submitOrder(cart)
activate Controller
Controller -> Gateway : charge(amount)
activate Gateway
alt #LightGreen Response within 5s
  Gateway --> Controller : 200 OK (txnId)
  Controller -> Order : markPaid()
  Controller --> UI : OrderSuccess
else #Pink Timeout > 5s
  Gateway --> Controller : [Timeout Error]
  Controller -> Gateway : voidTransaction()
  Controller --> UI : PaymentFailed(retryOption)
end
deactivate Gateway
```

- **架構評分檢核：**
  - `[通過]` UI 僅與 Controller 通訊，嚴禁直接穿透存取資料庫或外部 Gateway。
  - `[通過]` 成功與失敗路徑封裝於 `alt` 複合片段中，守衛條件清晰且互斥。

活動解答 4：活動圖工作流程演進 (Activity Workflow Evolution)

- **架構方案與語義分析：**
  - **外送員拒單重新派單迴圈：** 在 `|調派引擎|` 泳道內部，以 `repeat` 迴圈查詢最近外送員。發送通知將 Token 傳遞給 `|外送員|` 泳道。決策菱形評估：`if (接受接單?) then (yes) -> 確認指派; else (no) -> 記錄拒單; repeat while (仍有可用騎手)`。
  - **顧客門市自取直通分支：** 在 `|平台系統|` 設置決策菱形：`if (訂單類型?) then (外送) -> Fork 同步啟動廚房與外送員; else (自取) -> 僅通知廚房備餐; |顧客| :於櫃檯出示憑證取餐; -> Merge`。
- **PlantUML Delta 代碼：**

```
|Dispatch System|
repeat
  :Find Available Courier;
  |Courier|
  :Receive Dispatch Ping;
backward:Log Rejection;
repeat while (Courier accepts?) is (no)
->yes;
:Confirm Courier Assignment;
|Kitchen|
:Hand Off Sealed Package;
|Courier|
:Deliver to Customer;
stop
```

- **架構評分檢核：**
  - `[通過]` 動作泳道歸屬明確，未發生職責倒置。
  - `[通過]` `repeat...repeat while` 迴圈具備確定性的跳出路徑。
  - `[通過]` 所有終止分支皆匯入明確的 `stop` 活動終止節點。

活動解答 5：狀態機生命週期例外處理 (State Machine Lifecycle Exceptions)

- **架構方案與語義分析：**
  - **外送員途中拋錨重新指派：** `OutForDelivery → ReadyForPickup : courier_abandoned [retries < 3] / unassign()`。退回備餐完成狀態重新調派騎手，確保餐廳無需重做已打包好的餐點。
  - **送達現場聯絡不上顧客退回餐廳：** `OutForDelivery → FailedDelivery : unreachable [wait >= 10m] / markFailed()`。隨後 `FailedDelivery → ReturnedToStore : return_package / recordDisposal()`。終止變遷：`ReturnedToStore → [*]`。
  - **不可逆約束守衛：** 嚴禁直接由 `OutForDelivery → Cancelled`，餐點一旦在途取消，必須依循實體回收補償流程。
- **PlantUML Delta 代碼：**

```
state OutForDelivery {
  [*] --> InTransit
}
OutForDelivery --> ReadyForPickup : courier_dropped \
  [retries < 3] / reassign()
OutForDelivery --> FailedDelivery : unreachable \
  [wait >= 10m] / logFailed()
FailedDelivery --> ReturnedToStore : return_meal \
  / auditWaste()
ReturnedToStore --> [*]
```

- **架構評分檢核：**
  - `[通過]` 狀態命名維持被動條件與名詞/形容詞 (`FailedDelivery`, `ReturnedToStore`)。
  - `[通過]` 變遷完整標註事件觸發、`[守衛條件]` 與 `/動作`。
  - `[通過]` 異常生命週期路徑最終皆收斂至終止狀態 `[*]`。

4.9.3 全章核心觀念自我檢測：14 題精準填空測驗

```
候選字彙庫 (Word Bank):
[ 互動視角 (Interaction) ]
[ Ivar Jacobson ]
[ Boundary-Control-Entity (BCE) ]
[ <<include>> ]
[ <<extend>> ]
[ 組合關聯 (Composition, ◆) ]
[ 聚合關聯 (Aggregation, ◇) ]
[ 實心箭頭 (→) ]
[ alt 複合片段 ]
[ 泳道 (Swimlanes) ]
[ Join 結合節點 ]
[ 守衛條件 (Guard) ]
[ 正交區域 (Orthogonal Regions, ||) ]
[ 歷史狀態 (History, [H*]) ]
```

第一部分：結構與使用案例基石 (Structural & Use Case Foundations)

1. 在 Rational 軟體公司共同促成 UML 統一標準的「UML 三巨頭」為 Grady Booch、Jim Rumbaugh 與 **[Ivar Jacobson]**。
2. 在軟體系統塑模的四大基石視角中，分別為外部視角、**[互動視角 (Interaction)]**、結構視角與行為視角。
3. 在使用案例圖中，多個案例之間 mandatory 強制共用的子程序關聯標記為 **[`<<include>>`]**。
4. 在特定擴充點或異常條件下才會選擇性觸發插入的業務關聯標記為 **[`<<extend>>`]**。
5. 將系統切分為使用者介面、業務調度控制器與持久化領域實體的三層架構分析模式為 **[Boundary-Control-Entity (BCE)]**。
6. 在 UML 類別圖中，實心黑菱形 (`◆`) 代表 **[組合關聯 (Composition, ◆)]**，子物件生命週期與整體容器同生共死。
7. 在 UML 類別圖中，空心白菱形 (`◇`) 代表 **[聚合關聯 (Aggregation, ◇)]**，子物件具備獨立於整體的自主生命週期。

第二部分：動態流程與狀態行為 (Dynamic Workflows & Statecharts)

8. 在循序圖中，發送端呼叫後必須阻塞等待接收端返回之同步請求，其連線繪製為 **[實心箭頭 (→)]**。
9. 在循序圖中，用於塑模互斥條件分支（等同於程式碼 if-else）之結構化容器為 **[alt 複合片段]**。
10. 在活動圖中，用於區分各部門、系統服務或業務角色責任歸屬的垂直/水平分區為 **[泳道 (Swimlanes)]**。
11. 在活動圖中，用於等待所有並行執行緒全部到達匯流後方能繼續向下的同步粗線條為 **[Join 結合節點]**。
12. 在 UML 狀態機的變遷標籤中，以中括號 `[ ]` 包覆、必須為 true 變遷方能觸發之布林檢查式為 **[守衛條件 (Guard)]**。
13. 在複合狀態中，允許多個並行子狀態軌道在背景同時並發推進的區域稱為 **[正交區域 (Orthogonal Regions, ||)]**。
14. 當系統因突發中斷暫停、在爭議解除重新進入時能記憶並恢復至中斷前深層子狀態的假狀態節點為 *[歷史狀態 (History, [H])]*。

4.9.4 經典權威文獻與延伸閱讀

- **Booch, G., Rumbaugh, J., & Jacobson, I. (2005).** *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- **Fowler, M. (2003).** *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- **Cockburn, A. (2000).** *Writing Effective Use Cases*. Addison-Wesley.
- **Object Management Group (OMG). (2017).** *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.
- **PlantUML 官方標準手冊：** plantuml.com

附錄：課堂互動測驗 (CCQ 1–8) 詳細解答與架構復盤解析

CCQ 1: 四大塑模視角

- **正確答案：** C (結構視角)
- **架構復盤：** 定義實體資料如何組織、類別之間如何繼承與關聯，且完全不依賴執行時期之時序順序，為結構視角的典型職責。外部視角關注環境邊界；互動視角關注執行期訊息傳遞；行為視角關注動態狀態機與流程。

CCQ 2: UML 奠基先驅與三巨頭

- **正確答案：** C (Ivar Jacobson)
- **架構復盤：** Ivar Jacobson 於 1986 年在 OOSE 方法中首創使用案例 (Use Cases)，將軟體架構牢牢錨定於使用者目標。Grady Booch 奠定 Booch Method 物件設計，Jim Rumbaugh 開創 OMT 強調物件關係語法，Martin Fowler 則著有傳世經典《UML 精華》。

CCQ 3: 使用案例關聯 (包含 vs. 擴充)

- **正確答案：** B (優惠券套用為條件性擴充行為；扣款處理為強制包含必備子程序)
- **架構復盤：** `<<include>>` 代表基底案例成立所必需之強制共享邏輯（無金流扣款則訂單無法成立）；`<<extend>>` 則代表選擇性條件觸發邏輯，僅在擴充點被激發時插入執行（顧客可自由選擇是否輸入折價券代碼）。

CCQ 4: 類別關係 (組合 vs. 聚合)

- **正確答案：** B (OrderItem 隨 Order 同歸於盡，Courier 則擁有獨立之生命週期)
- **架構復盤：** 組合 (◆) 代表強烈生命週期相依，整體被刪除則內部組件必被 Cascade Delete 銷毀；聚合 (◇) 代表弱擁有同儕關係，外送員在送完訂單後依然存在於系統中。

CCQ 5: BCE 架構職責分工

- **正確答案：** C (傳遞給 OrderController 控制物件協調整體驗證與調用)
- **架構復盤：** BCE 架構強調關注點分離。前端邊界物件 (UI) 嚴禁直接操作資料庫 Entity 或呼叫第三方 API，必須透過 Control 物件協調整體商業邏輯、資料庫交易與邊界調派。

CCQ 6: 活動圖同步條 vs. 決策菱形

- **正確答案：** A (Fork/Join 同步切分並行執行緒；Decision/Merge 依據守衛條件互斥二選一)
- **架構復盤：** Fork/Join 為並行多執行緒節點，Token 同步複製與等待匯集；Decision/Merge 則是條件分支節點，Token 依據布林條件由多條路徑中選擇唯一的一條向下執行。

CCQ 7: 狀態機動作 vs. 活動

- **正確答案：** A (Action 瞬間完成不可中斷；Activity 耗時運算且隨時可被事件中斷)

- **架構復盤：** OMG UML 規範定義 Action（轉移觸發、entry/、exit/）為原子性瞬間操作；Activity（`do /`）代表一段正在背景持續進行的運算，一旦有任何出境轉移事件到達，該活動會被立即中斷中止。

CCQ 8: 文字宣告式塑模工程價值

- **正確答案：** B (純文字代碼可直接納入 Git 版本控制、在 PR 中進行 diff 審查與自動化建置)
- **架構復盤：** 架構即代碼 (Architecture-as-Code) 透過文字宣告圖表，完美克服傳統 GUI 軟體產出無法比對差異、無法跑 CI/CD 且無法與現代 LLM 自動生成的根本缺陷。